Precondition Tools

15 Minutes

Timedoor academy PRO

Precondition Tools

There are several **tools** we need to prepare before we dive into the next material which are **Text Editor**, **Node.js**, and **Javascript Package Manager.**





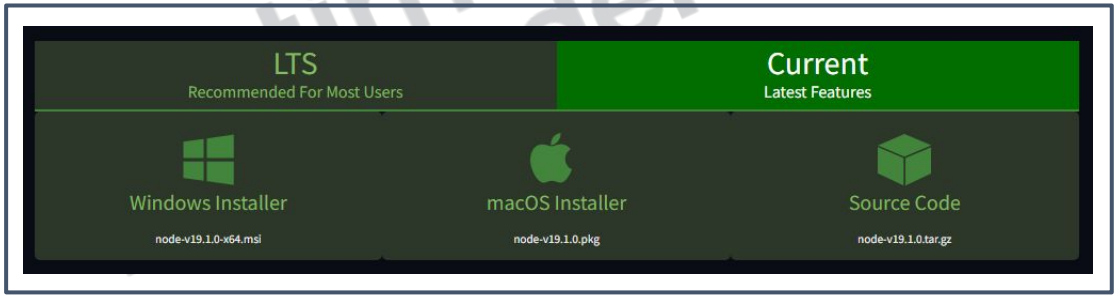


timedoor academy PRO

Node.js and Javascript Package Manager

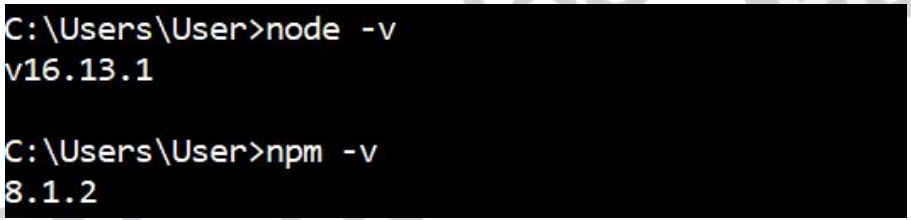
To be able to use **Vue Js** in your computer, we need to first install **Node.js** into the **computer** that will be used. Follow the next steps to **install Node.js**.

- Go to this link: **https://nodejs.org/en/download/**
- Select **Current (Lates Features)** then select the appropriate **OS** used.



- Wait until the **file is downloaded**, then do the **installation** until it's finished.

- After the installation is done, do a **command prompt** by opening and running the `node -v` and `npm -v` prompts. When the **two codes** are **running**, then the **installed version** of **node** and **npm** will appear as follows.



The version node and **npm** of others may be different, so don't worry if the **version of node** and **npm** that you are using is different from the **example above**.

Text Editor

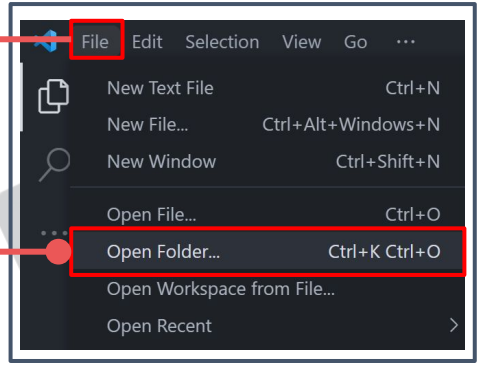
Text Editor that we will use is **Visual Studio Code (VS Code). VS Code** is a **Text Editor** that is very **popular** and **free** to use. **VS Code** can run in the following operating systems which are **Windows**, **MacOs**, **or even Linux.**

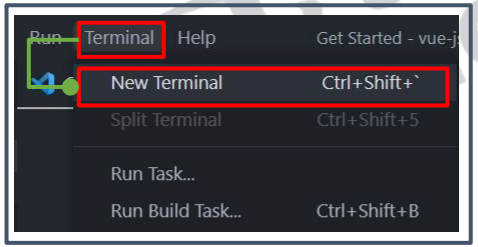
Visit the following link to download the editor:
https://code.visualstudio.com/Download

timedoor academy

Vue Installation

- Create a new **folder** in your **computer** named **vue-js-project.**
- Open **VS Code** then select **file -> open folder.** Select the **vue-js-project** folder made from before.

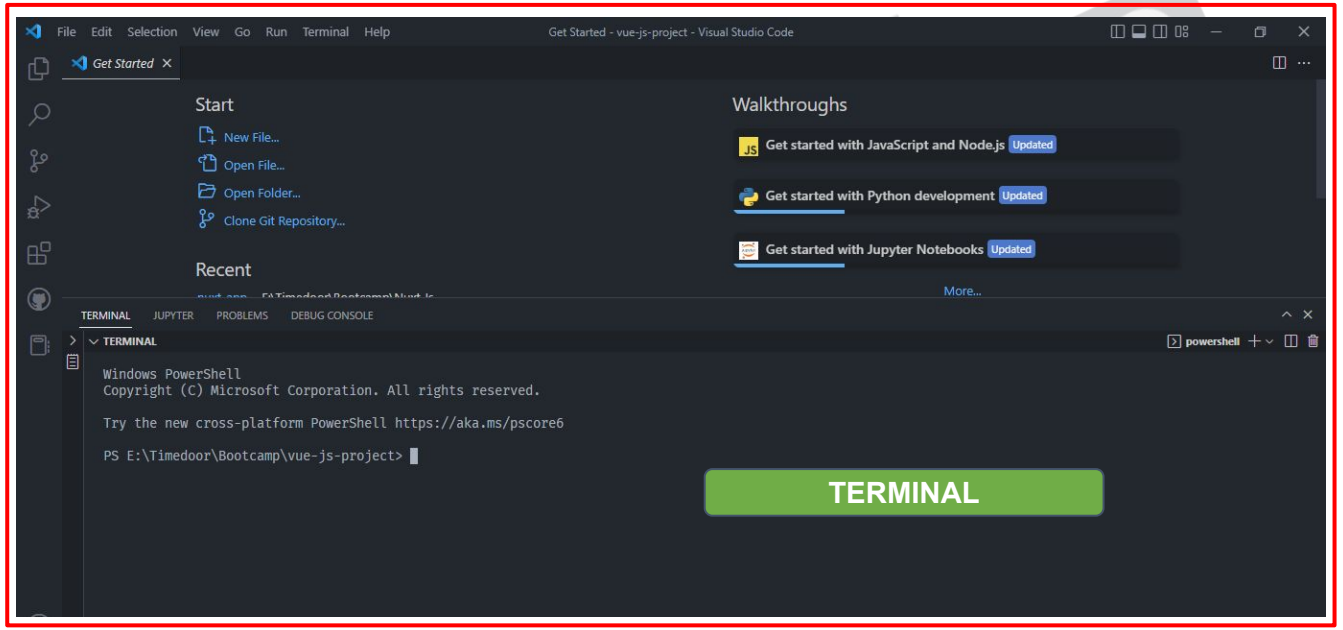




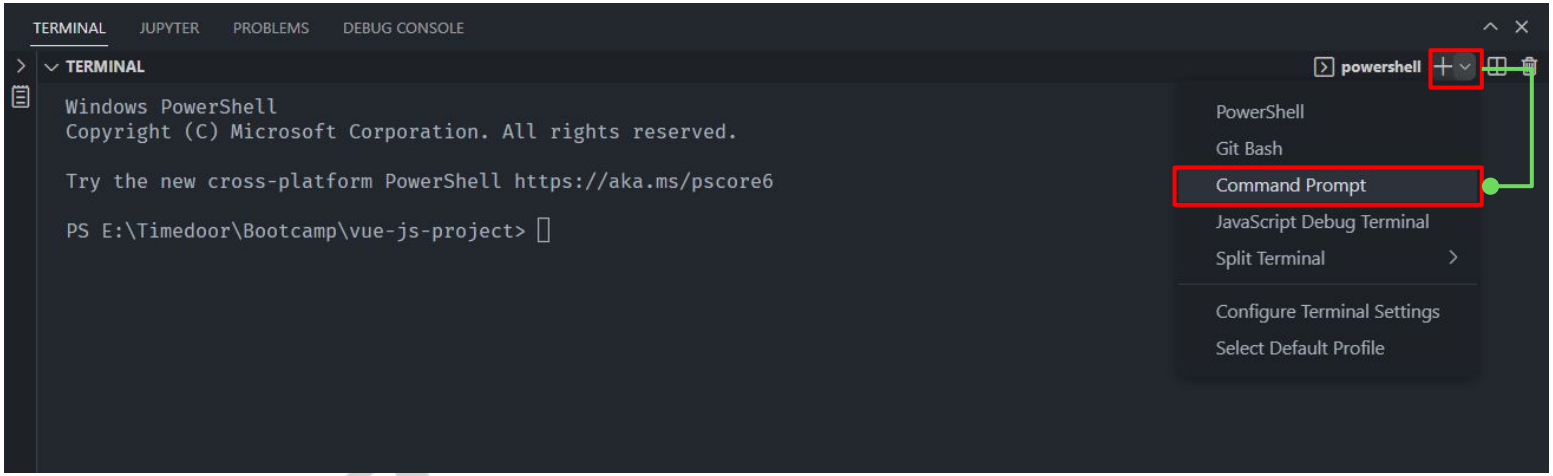
- Afterwards, in **VS Code,** select the menu **Terminal -> New Terminal.**

timedoor academy

The display of the **VS Code** will look like this. Through the **VS Code terminal,** we will **install Vue CLI**



- Click the **launch profile** part in the **terminal** then select **Command Prompt.**



- Afterwards, run this code: `npm init vue@latest.` When we run this code, there will be a few things we have to do.
  Go to the next slide.

- Type “**y**” then press **enter** in the next process.

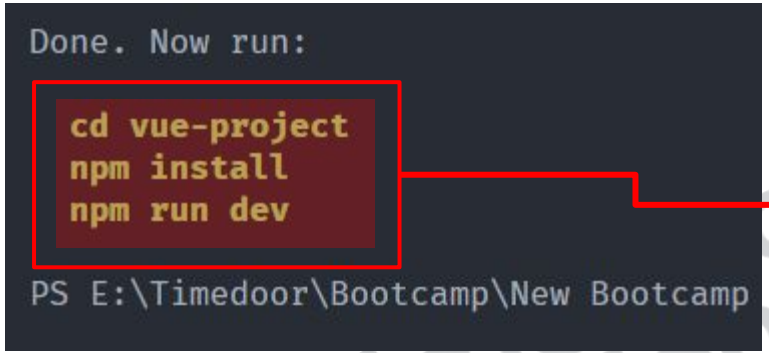
```
Need to install the following packages:
  create-vue@latest
Ok to proceed? (y) y
```

- Then select **No** to all of these options.

```
Vue.js - The Progressive JavaScript Framework

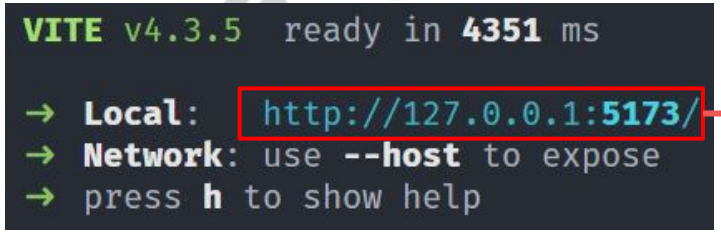
√ Project name: ... vue-project
√ Add TypeScript? ... No / Yes
√ Add JSX Support? ... No / Yes
√ Add Vue Router for Single Page Application development? ... No / Yes
√ Add Pinia for state management? ... No / Yes
√ Add Vitest for Unit Testing? ... No / Yes
√ Add an End-to-End Testing Solution? » No
√ Add ESLint for code quality? ... No / Yes
```

- Now, we just need to run the following codes in the terminal



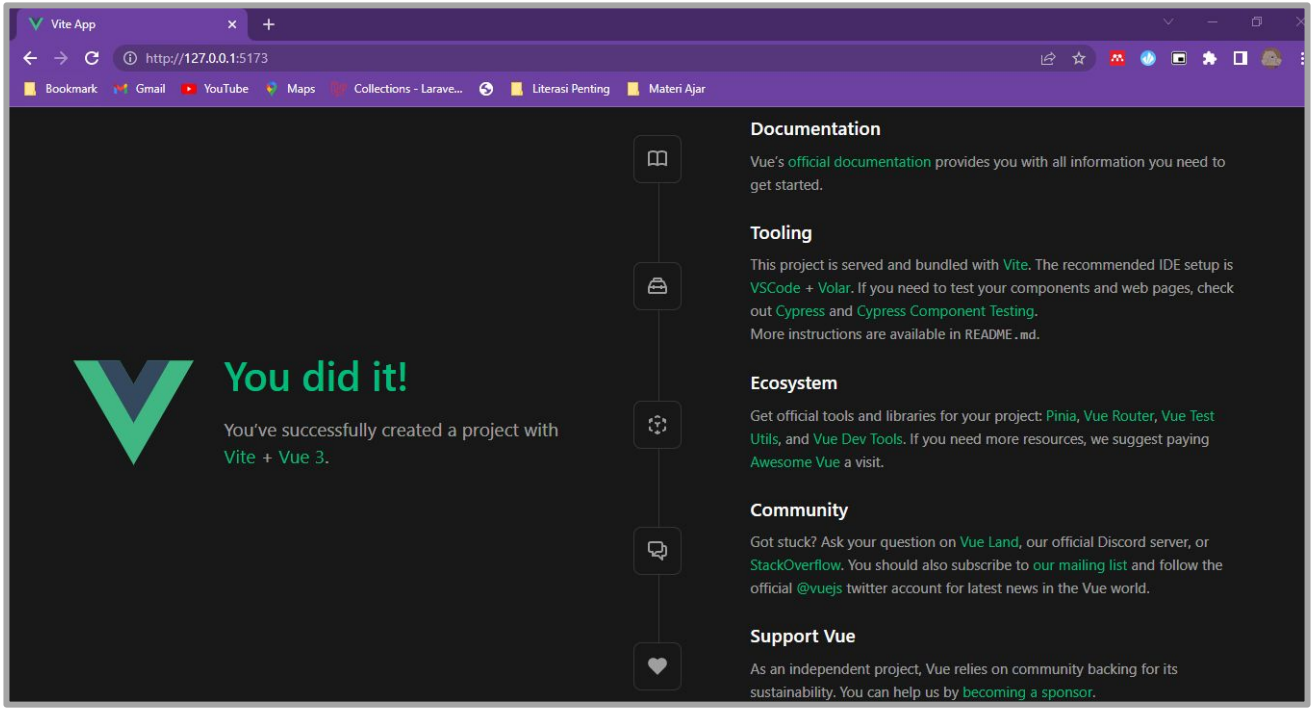
Follow the steps accordingly to each terminal

- After all the code is run, a screen like the following will appear.



Open the following link in the **browser**.

The appearance of the **website** page will be like the following

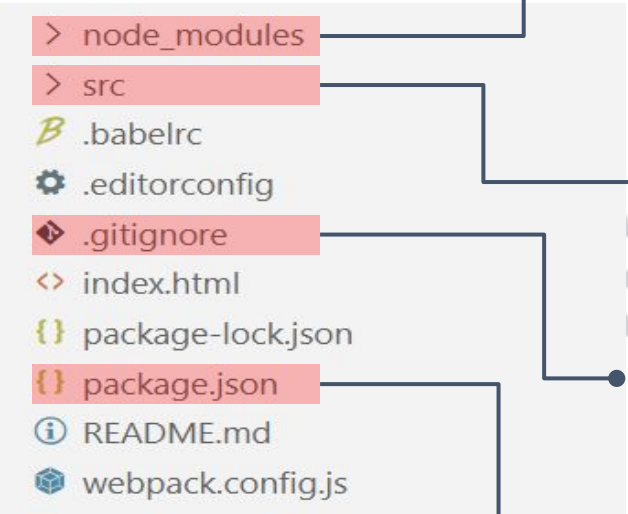


Folder Structure

15 Minutes

Timedoor academy PRO

Folder Structure



The **node_modules** folder is used to store the many dependencies that are needed for the project.

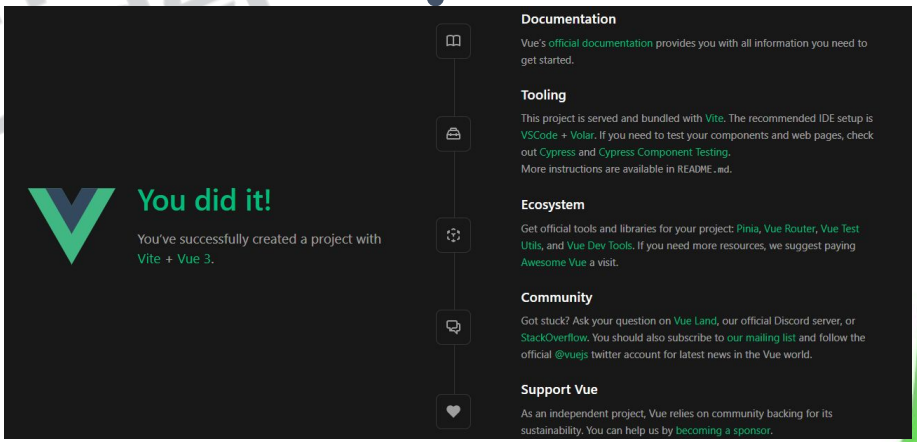
The **src** folder consists of several other folders such as **assets, components, and many more.** The **src** folder is used to store the syntax or code with the file format of **.vue**.

The **.gitignore** file is a file associated with github. This file can be used to set which files will be **pushed to repository**.

In the **package.json** file, there are lists of dependencies that can be used for project development needs.



The **index.html** file is the main page accessed by **Vue Js** when it runs.



The **main.js** file is used to link and configure all **Vue app** that will be used

main.js

```
import './assets/main.css'

import { createApp } from 'vue'
import App from './App.vue'

createApp(App).mount('#app')
```

This section connects with a **tag** with an **id** named **app** contained in **index.html.**

Because of that the ** ** tag in the **index.html** file is important because it will be used to link with the main.js file.

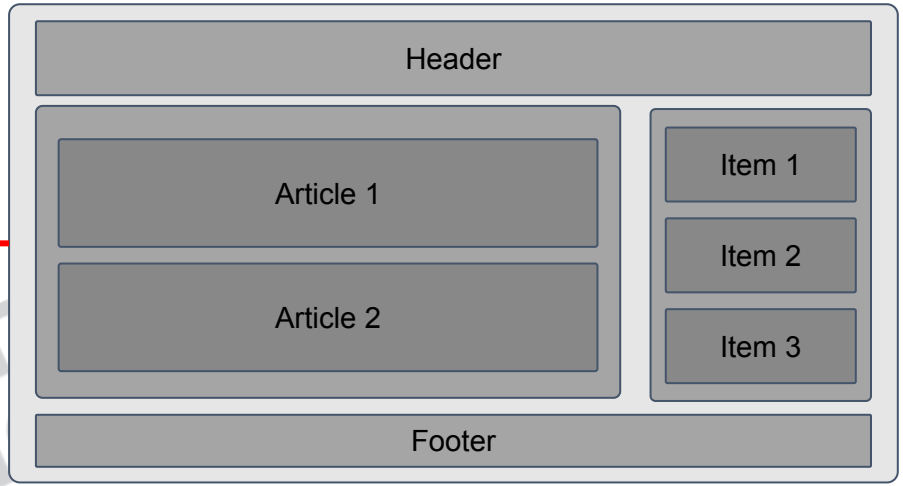
Component in Vue

15 Minutes

timedoor academy PRO

Component is an important feature in **Vue Js** that is used to create and store **HTML** elements and can be used repeatedly.

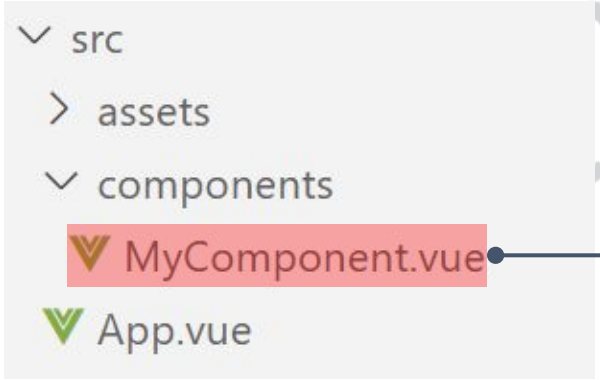
A website consists of several **component** like the following example



A website can be **divided into several components** like components for **header, footer, main, and aside.** Each component can also consist of other components. This is often called **nested component**.

All components will be stored in the **src folder**. In order for the file structure to be tidier, all **subsequent components** will be placed in the **components folder**





File names should consist of two syllables, use the **PascalCase** system for naming **component** files and the file format used is **.vue**

By the standard, every component in **Vue CLI** will contain three parts which are **<template></template>**, **<script></script>**, and **<style></style>.**

MyComponent.vue

```
<template>
    <div>

    </div>
</template>

<script>
    export default {

    }
</script>

<style>

</style>
```

The HTML code will be writted in this part. **All HTML tags** that will be used must be put in between the ** tag.**

The Javascript required for the **component** to be created is placed in between the **<script></script>**

The style code (CSS) required for the **component** is placed in between the **<style></style> tag**

medoor academy PRO

component.vue

```
<script>
  export default {
    data() {
      return {

      }
    },
    methods: {

    },
    computed: {

    },
    components: {

    }
  }
</script>
```

Components in Vue can also use **data, methods, computed, watch** and so on as follows.

The usage of data property in a component must be a function. This is because components can be used repeatedly. If the data is not in the form of a function, every change in one component will affect the other components.

In order for the **components** created to appear on **web pages**, these **components** need to be called on **App.vue**.

```
<template>
  <my-component></my-component>
</template>

<script>
  import MyComponent from "./components/MyComponent.vue"
  export default {
    components: {
      MyComponent
    }
  }
</script>
```

Component calls are made using the **kebab-case** system, which is separated by a **dash**.

Components must always be ***imported*** in the file that will use the **component**.

In **components,** add a **component** that has been ***imported***.

timedoor academy PRO

If more than one **component** is used, it can be written as follows

app.vue

```
<template>
  <first-component></first-component>
  <second-component></second-component>
</template>

<script>
  import FirstComponent from "./components/FirstComponent.vue"
  import SecondComponent from "./components/SecondComponent.vue"
  export default {
    components: {
      FirstComponent,
      SecondComponent
    }
  }
</script>
```

SLOT

Slots are used to insert elements in a component.

Parent Component

```
<template>
  <div>
    <app-child-component>
      <p>Hello World</p>
    </app-child-component>
  </div>
</template>
```

Child Component

```
<template>
  <div>
    <slot></slot>
  </div>
</template>
```

The inserted element will be displayed via the **<slot></slot> tag** added to the **child component.**

SLOT NAME

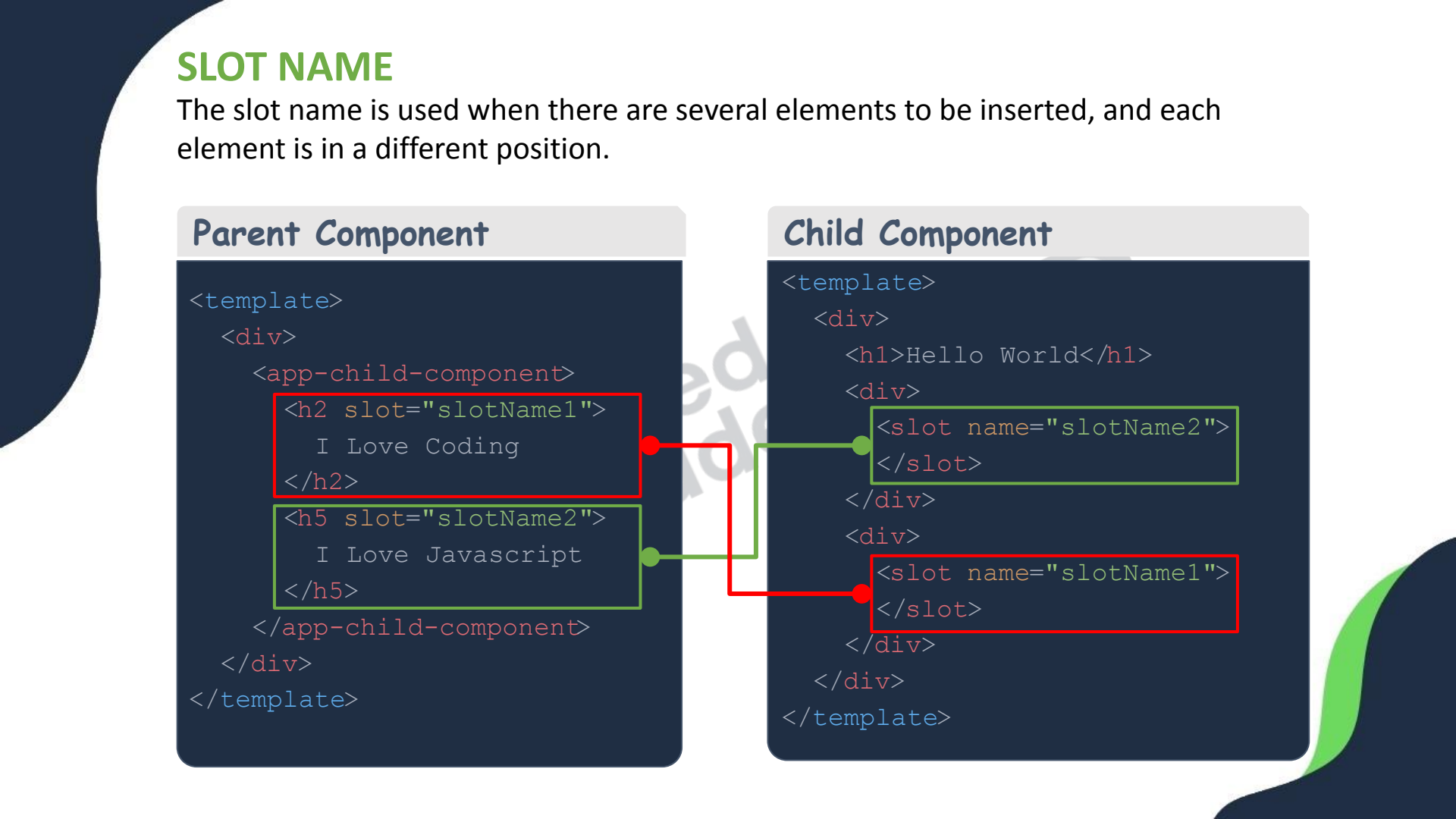
The slot name is used when there are several elements to be inserted, and each element is in a different position.

Parent Component

```
<template>
  <div>
    <app-child-component>
      <h2 slot="slotName1">
        I Love Coding
      </h2>
      <h5 slot="slotName2">
        I Love Javascript
      </h5>
    </app-child-component>
  </div>
</template>
```

Child Component

```
<template>
  <div>
    <h1>Hello World</h1>
    <div>
      <slot name="slotName2">
      </slot>
    </div>
    <div>
      <slot name="slotName1">
      </slot>
    </div>
  </div>
</template>
```



Practice Creating a Recipe Web Page

30 Minutes

Timedoor academy PRO

The Recipe Website will be used while learning **Vue.js**. For starters, please run the following command in the terminal to create a new project.

```
npm init vue@latest
```

Name it **“tasty-recipe”** in the **Project name**

```
Vue.js - The Progressive JavaScript Framework

? Project name: » tasty-recipe
```

Then select **“No”** to the rest of the options given.

```
cd tasty-recipe
npm install
npm run dev
```

Afterwards, run these three codes in the **terminal**.

Because the project created will use **Bootstrap**, please run the following code in the terminal to install **Bootstrap**.

```
npm install bootstrap@5.2.3
```

timedoor academy PRO

main.js

```
import './assets/main.css'
import "bootstrap/dist/css/bootstrap.css"
import { createApp } from 'vue'
import App from './App.vue'

createApp(App).mount('#app')
import "bootstrap/dist/js/bootstrap.js";
```

After installing Bootstrap, add the following two lines of code in the **main.js** file.

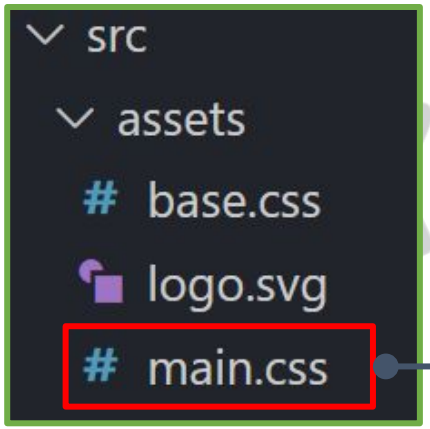
index.html

```
<head>
  <script src="https://kit.fontawesome.com/75f72bf635.js"
crossorigin="anonymous"></script>
  <link rel="preconnect" href="https://fonts.googleapis.com" >
  <link rel="preconnect" href="https://fonts.gstatic.com"
crossorigin>
  <link
href="https://fonts.googleapis.com/css2?family=Inter:wght@500&d
isplay=swap" rel="stylesheet">
  <title>Tasty Recipe</title>
</head>
```

- Add the following code to the tag in the **index.html file.**

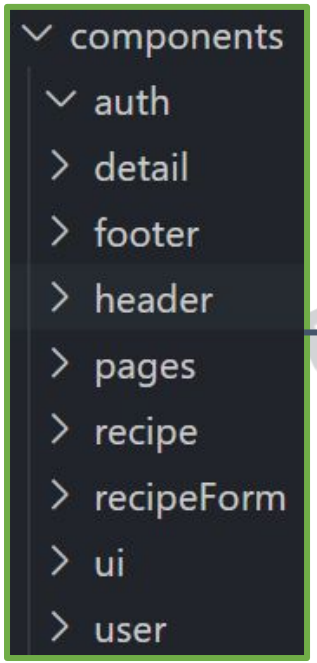
In the **src folder**, there is an **assets folder**. Replace the code contained in the **main.css** file with the code contained in the following link.

https://bit.ly/main-css-recipe



Replace the code contained in the following file.

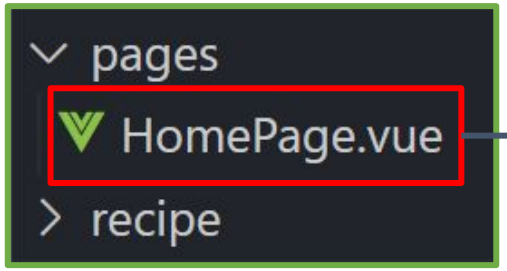
Now, we will create the necessary components to create a **Recipe Website.**



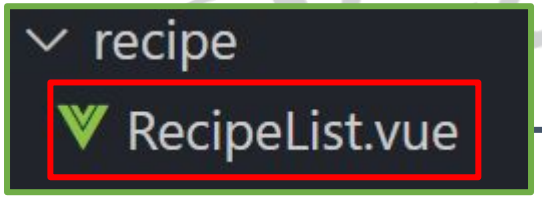
Add 9 folders to the **components** folder with names like the picture to the side.

timedoor academy PRO

Now we will create the main page first.



In the **pages** folder, add a new file called **HomePage.vue**



In the recipe folder, add two new files called **RecipeList.vue.**

timedoor academy PRO

In App.vue, add the code to call the HomePage.vue component as follows.

src > App.vue

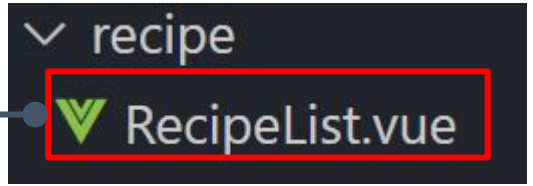
```
<template>
  <div>
    <home-page></home-page>
  </div>
</template>

<script>
import HomePage from "./components/pages/HomePage.vue"
export default {
  components: {
    HomePage
  }
}
</script>
```

Now we will fill ***2 components*** that has been made. Open the link below.

https://bit.ly/recipe-item-template

The code obtained from the following link will be placed in the **RecipeList.vue** component. **Pay attantion to the following slide**



Add the following code to the **RecipeList.vue** *component.*

src > components > recipe > RecipeList.vue

```
<template>
  <div class="recipe__list-recipe row" >
    <!-- Recipe Item Code -->
  </div>
</template>
```



Letakkan kode yang terdapat pada link slide sebelumnya pada bagian ini.

The RecipeList.vue *Component* is used to display the list of provided recipes. Meanwhile the **RecipeCard.vue** *component* is a template of each recipe.

Add the following code to the **HomePage.vue** *component.*

src > components > pages > HomePage.vue

```
<template>
  <div class="container-md my-5 py-5">
    <div class="recipe__title text-center">
      <h2>All Recipe</h2>
      <p style="font-size: 15px">
        Find and share everyday cooking inspiration on
         Allrecipes.Discover recipes, cooks, videos, and
         how-tos based on the food you love and the friends
         you follow.
      </p>
    </div>
    <!-- Recipe List Component -->
  </div>
</template>
```

Letakkan *component* **RecipeList.vue** pada bagian ini.

Make sure your **webpage** looks like this.

All Recipe

Find and share everyday cooking inspiration on Allrecipes.Discover recipes, cooks, videos, and how-tos based on the food you love and the friends you follow.

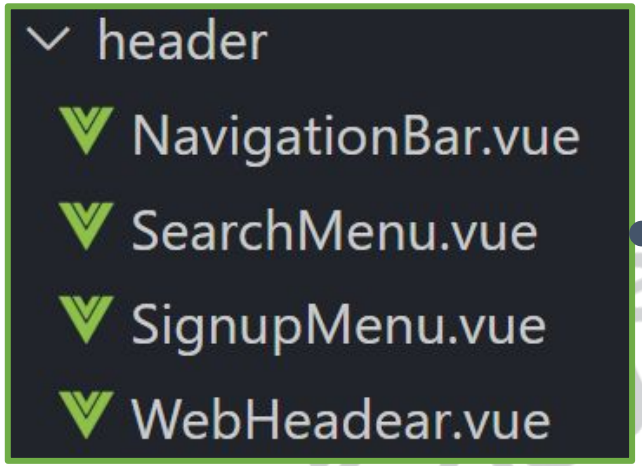


Dinner

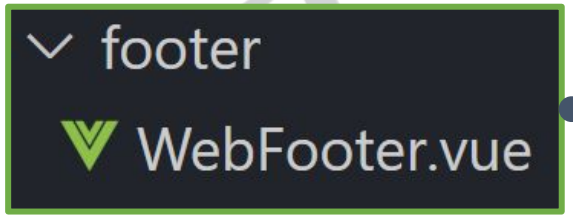
Beef Wellington

Recipe By Gordon Ramsay

Now we will display the **footer** and **header** on the recipe web page.



Add 4 new ***components*** in the **header folder** like the image to the side.



Then in the **footer folder**, Add 1 component like the image to the side.

Please enter the following link to obtain the code required for each components to create the **Header** and **Footer**.

https://bit.ly/header-footer-code

After that, move each piece of code and place it in the specified component.

https://bit.ly/tasty-recipe-logo

Download the logo in the link then place it in the **assets folder > images (Create the images folder first)**

Add the following code to the **NavigationBar.vue** *component.*

src > components > header > NavigationBar.vue

```
<template>
  <div
    class="header__navbar row
    justify-content-between align-items-center"
    style="width: 450px"
  >
    <!-- Component SearchMenu -->
    <!-- Component SignupMenu -->
  </div>
</template>
```

Panggil component **SearchMenu.vue** dan **SignupMenu.vue** disini.

Add the following code to the **WebHeader.vue** *component.*

src > components > header > WebHeader.vue

```
<template>
  <header
    class="shadow-sm container-fluid
        bg-white py-3 border-bottom fixed-top" >
    <div class="container-md d-flex
        justify-content-between align-items-center" >
      <img src="@/assets/images/Logo.png"  alt="Logo" />
      <!-- Component NavigationBar -->
    </div>
  </header>
</template>
```

Panggil component **NavigationBar.vue** disini.

In **App.vue**, call the **WebHeader.vue** and **WebFooter.vue** *component.*

src > App.vue

```
<template>
  <div>
    <!-- Component WebHeader.vue -->
    <home-page></home-page>
    <!-- Component WebFooter.vue -->
  </div>
</template>
```

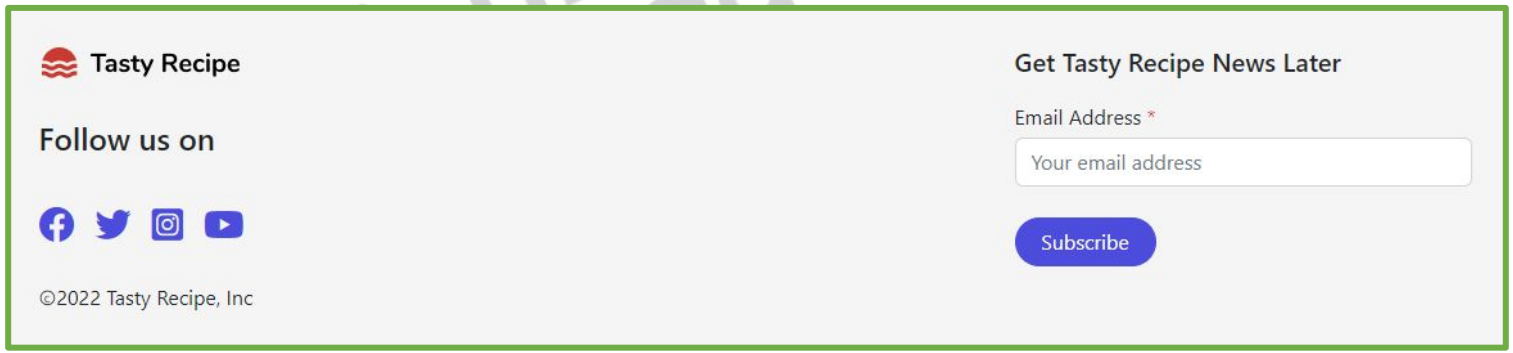
Panggil component **WebHeader.vue** dan **WebFooter.vue** disini.

Make sure your **webpage** looks like this.

Header



Footer



Composition API

15 Minutes

medoor academy PRO

Before we learn about **Composition API,** we will remember about **Option API** that was used when learning about **basicVue.js.**

Option API provides selections such as **data**, **methods**, **computed**, **mounted** to manage each code used. Because of that, this term is called **Option API**.

Option API

```
<script>
  export default {
    data() {
      return {

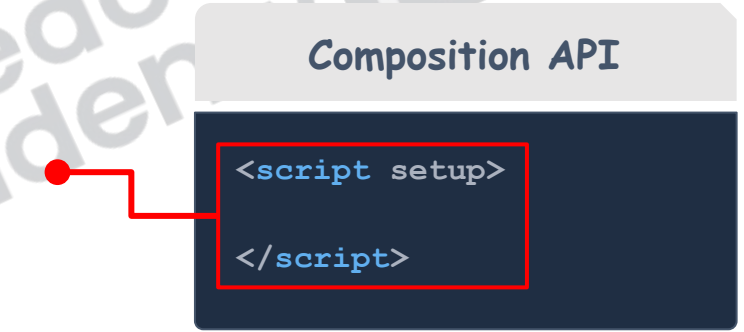
      }
    },
    methods: {

    },
    computed: {

    }
  }
</script>
```

Composition API is a new feature from **Vue.js version 3**. **Composition API** can overcome the limitations of the **Options API** contained in **Vue.Js**.

All code in the **Composition API** can be grouped based on their needs so that they are easy to maintain, test, and reuse.



Composition API has several important features which are as follows.

- **Reaktivitas**

  `ref` and **`reactive`**can be used to create reactive data within components. **`ref`** used to create reactive data with any data type while **`reactive`** is used to create data with object or array data type.

ref and reactive

```
<script setup>
import { reactive, ref } from 'vue'

const firstState = reactive({ count: 0 })
const secondState = ref(0)
// Call Reactive data
firstState.count++
// Call Ref data
secondState.value++
</script>
```

- **Lifecycle Hooks**

  The Composition API has **Lifecycle Hooks** which consist of functions that run at different stages of a component's lifecycle like **`onUpdated`**, **`onUnmounted`**, and **`onUpdated`.**

- **Computed Property**

  The computed property is recommended for use when there is complex logic involving reactive data.

Computed Property

```
<script setup>
import { computed } from 'vue'

const computedName = computed(() => {
  // return
})
</script>
```

Component in Composition API

We will now learn how to call a **component** with **Composition API.**

app.vue

```
<template>
  <first-component></first-component>
  <second-component></second-component>
</template>

<script setup>
  import FirstComponent from "./components/FirstComponent.vue"
  import SecondComponent from "./components/SecondComponent.vue"
</script>
```

The way is almost similar to **Option API**. Only in **Composition API**, the component needs to be *imported* first and directly used on **<template>**.

Composition API: Lifecycle Hooks

5 Minutes

timedoor academy PRO

Every **component** in **Vue** goes through several phases. **Lifecycle Hooks** help developers to run code at certain phases. The following are some of the **Lifecycles** contained in the **composition API**.

- **`onBeforeMount`:** This function is called right before the *component* is embedded in the DOM.
- **`onMounted`:** This function is called after the *component* is embedded in the DOM.
- **`onBeforeUpdate`:** This function is called before the *component re-renders* but not when the *component* is first *rendered*.

filmedoor academy PRO

- **`onUpdated`:** This function is called when the *component* is *re-rendered* but not when the *component* is first *rendered.*

- **`onBeforeUnmount`:** This function is called just before the *component* is detached from the DOM or is no longer part of the *rendered* web page.

- **`onUnmounted`:** This function is called after the *component* is detached from the DOM or is no longer part of the rendered web page.

- **`onErrorCaptured`:** This function is called when an error is found in the *child component*.

The functions described on the previous slide can be used in the setup function of the ***component***. An example of its use is as follows.

```
<script setup>
import { onMounted, onUnmounted } from 'vue'

onMounted(() => {
  //Code to run when the component is mounted
})

onUnmounted(() => {
  // Code to run when the component is unmounted
})
</script>
```

Challenge Calling Component in Composition API

15 Minutes

Ahmedoon Academy PRO

Challenge

After learning about the **Composition API**, now change all the component calling code in the **Tasty Recipe** project so that it uses the **Composition API** as follows.

Composition API

```
<template>
  <first-component></first-component>
</template>

<script setup>
  import FirstComponent from "./components/FirstComponent.vue"
</script>
```

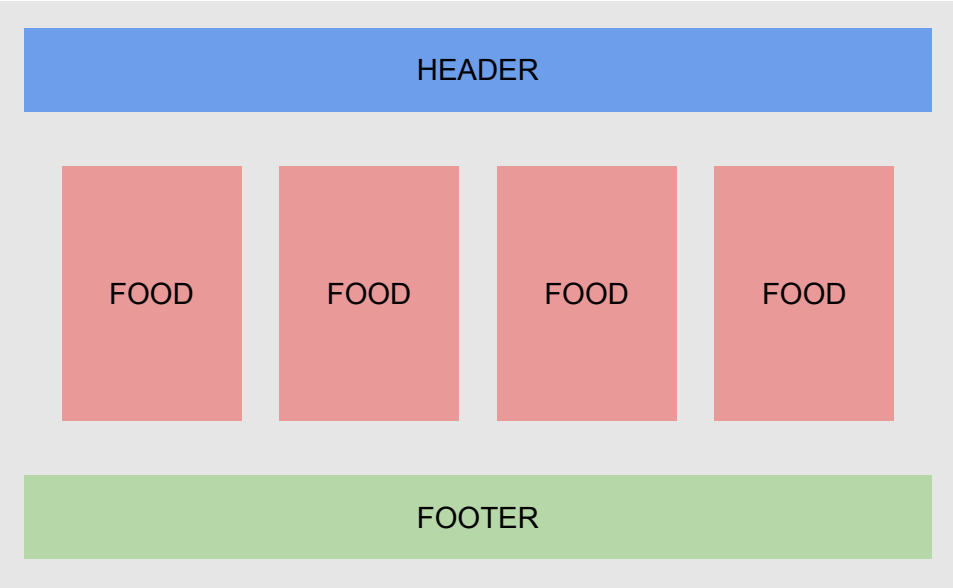
Props in Component

15 Minutes

timedoor academy PRO

Before we learn about **PROPS**, we need to understand about **parent** and **child components**.

Parent is a name for a **component** that uses another **component. While child** is the term for the **component** used.

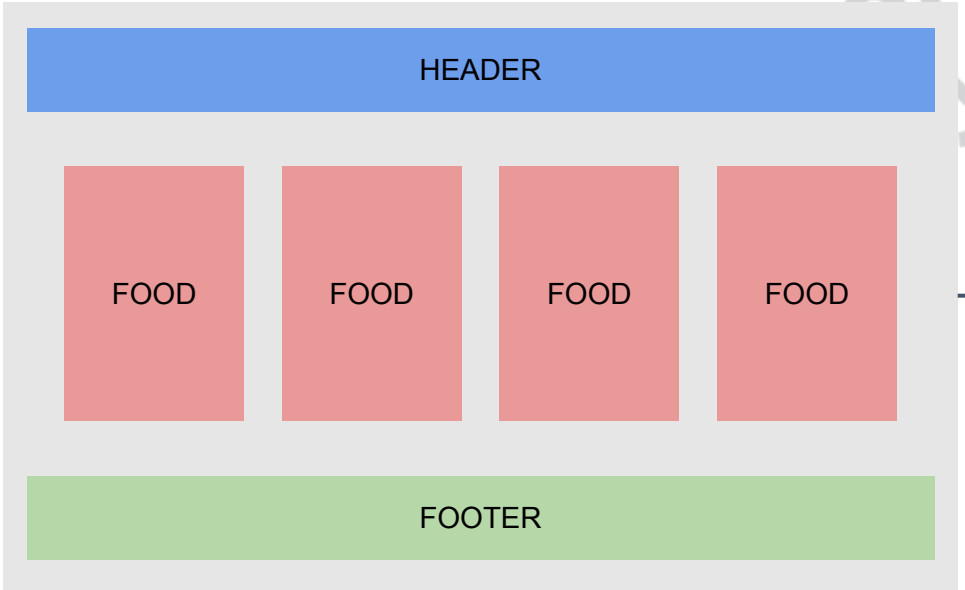


The picture beside is an example of a **component** that uses three **components** in it.

Now we try to divide the part that includes the **parent** and the part that includes the **child**.

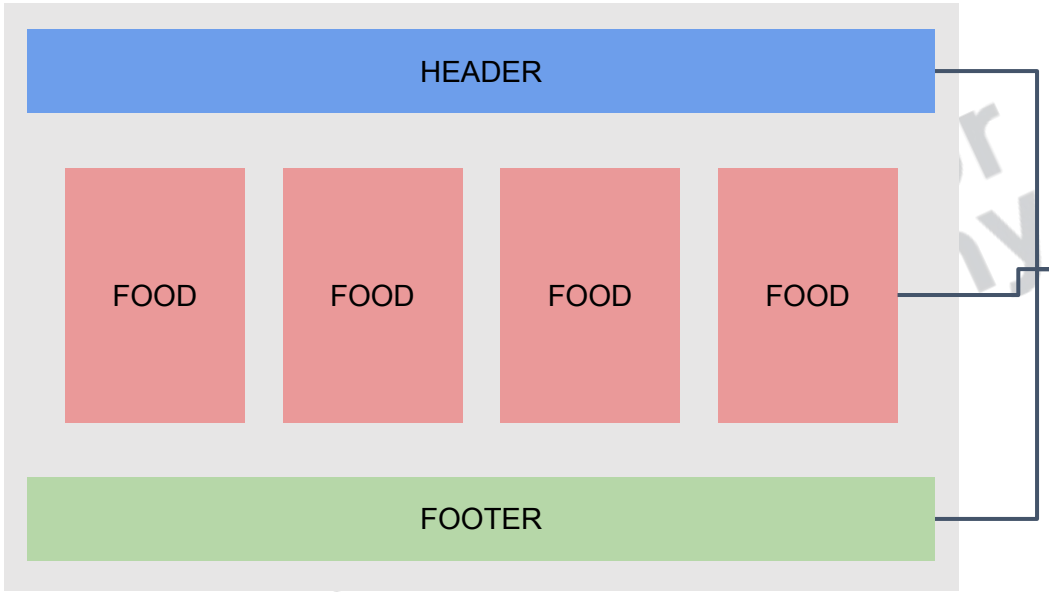
The following example consists of four *components*.

App Component | Header Component | Food Component | Footer Component



The App Component is the **main component.** This happens because the **the App Component** uses three other components which are **Header, Food**, and **Footer Components**. Because of that, the App Component is called the **parent**.

App Component | Header Component | Food Component | Footer Component

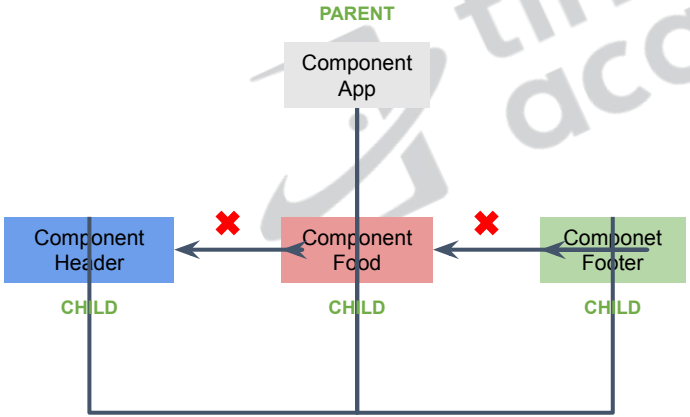


The Header,Food, and **Footer Components** are **Components** being used. So these **components** can be called a **Child.**

After understanding what are **parent and child to a component**, we will now learn about **Props**.

What are PROPS?

Props are property used to exchange data from the **parent component** to **the child component**.



Props can only be used to exchange data from **parent to child**. While data exchange between children cannot be done with props.

timedoor academy PRO

How to create Props?

Props are placed in the **component name** when called to the **parent component**.

Parent Component

```
<my-component :myProps="myPropsData"></my-component>
```

Props name

Props name will be used as a place to store data sent to **child components**

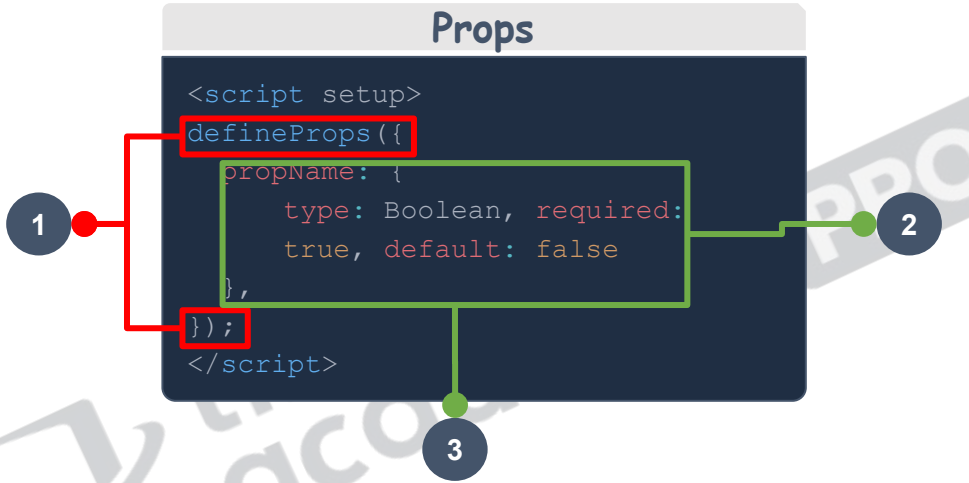
Data being sent

Is the data that is being sent to the **child component.**

Then how do child components receive data sent by parent components through Props?

Child components can receive data sent via props in the following way.

All props received must be **declared** in **defineProps({}).**



The props names that has been received which are **type data**, **required (optional)**, and **default value (optional).**

The props name that is declared must be the same as the **props name** used when the **component is called** in the **parent component.**

Parent Component

```
<my-component :myProps="myPropsData"></my-component>
```

Practice Using Props to Display Recipe Data

30 Minutes

Timedoor Academy PRO

We will now continue the **Recipe Web** made from before. Open the project in **Visual Studio Code.**



Add a new file in the **src folder** called **recipe.js.**

This file will be used to store the eeded **recipe data.**

https://bit.ly/42D9NA5

Copy the entire code from the recipe data that has been provided with the link and paste it in the **recipe.js file.**

timedoor academy PRO

Now, we will display all recipe data on the main page. In the **HomePage.vue** component, add the following code.

src > components > pages > HomePage.vue

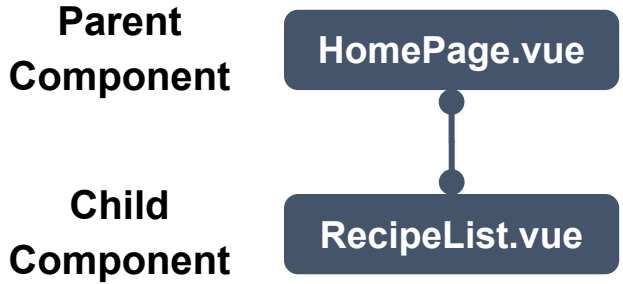
```
<script setup>
  import RecipeList from "../recipe/RecipeList.vue";
  import RECIPE_DATA from "../../recipe.js"

  const recipeList = RECIPE_DATA
</script>
```

Import the recipe data contained in the **recipe.js** file.

Save the recipe data in a variable named **recipeList**.

The structure of the **HomePage.vue** right now be will as followed.



The **HomePage.vue component** acts as a component that has recipe data needs to send that data to a child component, namely **RecipeList.vue.** In order to send data from the **parent component to the child component,** you can use **props.**

src > components > pages > HomePage.vue

```
<recipe-list
:recipes="recipeList"></recipe-list>
```

Add a **props** named **recipes** with the data sent which is the data saved in the **recipeList variable**.

Because the **RecipeList.vue** component acts as the **props receiver**, add the following code to the **RecipeList.vue component.**

src > components > recipe > RecipeList.vue

```
<script setup>
  defineProps({
    recipes: Array
  })
</script>
```

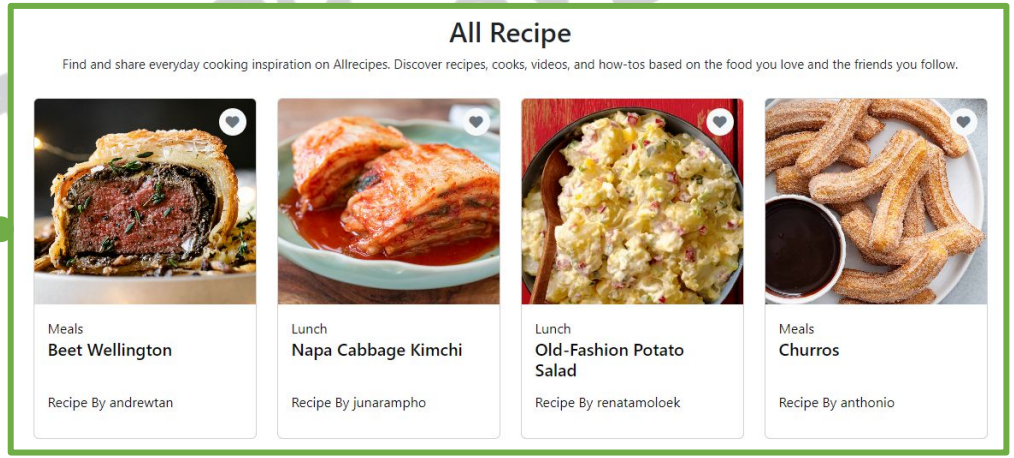
The props received is named **recipes** with an **Array** data type.

CHALLENGE

All the data is available in the **recipes props**. Now your task is to add the following code to the **RecipeList.vue component**.

- Use v-for to display the entire data in the recipes props.
- Use v-bind and expressions to display each data in the template that has been provided.

Expected end result.



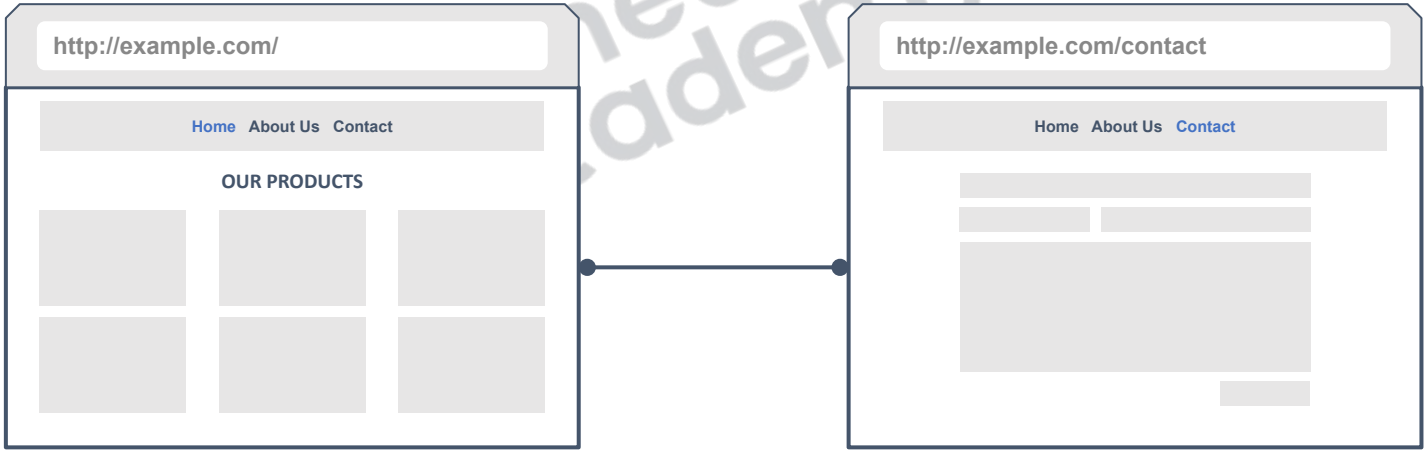
Routing Introduction

5 Minutes

timedoor academy PRO

ROUTING

Routing is the process of moving from one **page** to **another**. In **HTML**, we can use the **<a> t**ag to move pages by including the page we want to go to in the **href attribute**, whereas in **Vue Js** this can be done using the **Vue Router library.**

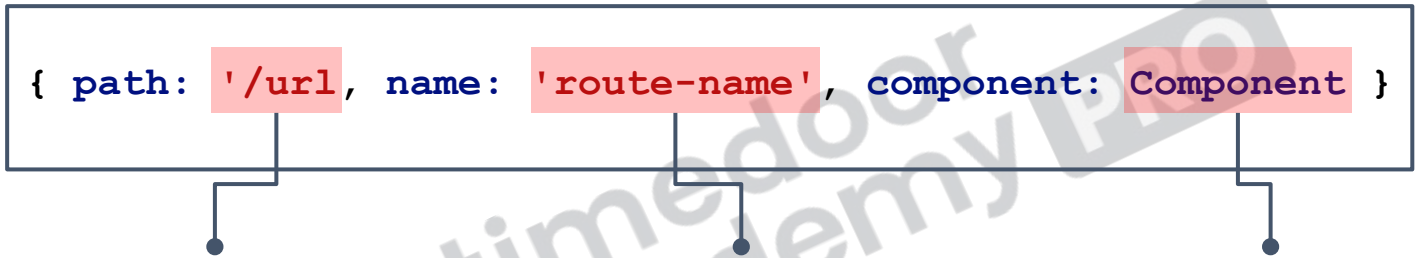




Vue Router takes advantage of the **url** in the **browser** to change **pages. Vue Router** will display the appropriate **component** with the **url** received.

When wanting to change **pages,** then we need to define what **url** will be **accessed** by **browser**.

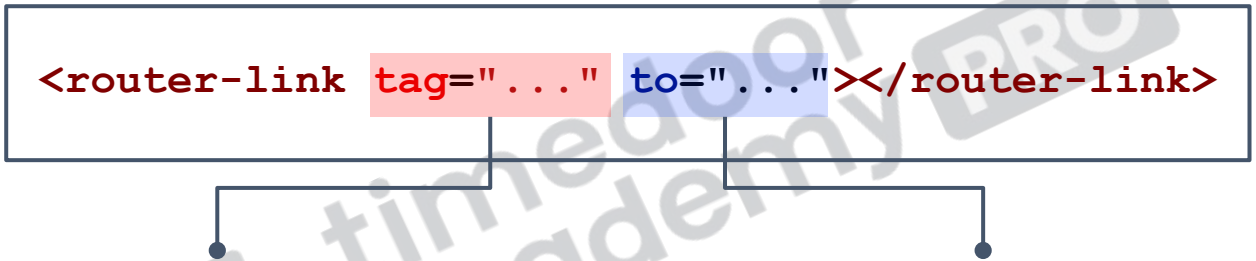
The **route** to be used needs to be **defined i**n advance in the following way.



A website will have **many routes,** so that **all routes** used later will be collected in one file called **route.js.**

router-link

`<router-link> </router-link>` is a command used to direct a page to another page. The function of `<router-link> </router-link>` is the same like the `<a> </a>` command in HTML.



The tag attribute can be filled with elements that are used to move pages. For example, we want to make a **button,** when the **button i**s clicked, the details page will open. Then the **tag atribut** can be written like the following `tag="button".`

The to attribute is filled with the **path or name** of the route to **be used.** On the **next slide,** we will discuss this further.

timedoor academy PRO

As we have learned before, the **route** needs to be **defined** first in the following way.

```
{ path: '/url', name: 'route-name', component: Component }
```

In the **code** above, there is a **property path** and **name.** The **to attribute** can be filled with the **path attribute** or **name** from a **route** that will be used.

```
to="/url"
```

How to write the to attribute when filled with the path property

```
:to="{ name: route-name }"
```

How to write the to attribute when filled with the property name

timedoor academy PRO

router-view

When using **Vue Router,** we need to add the **<router-view> </router-view> tag.** This tag functions to display the **component** from the directed page. This **tag** can be added in the **App.vue file.**

Installation

Installing the Vue Router library can be done using the following command.

```
npm install vue-router@4
```

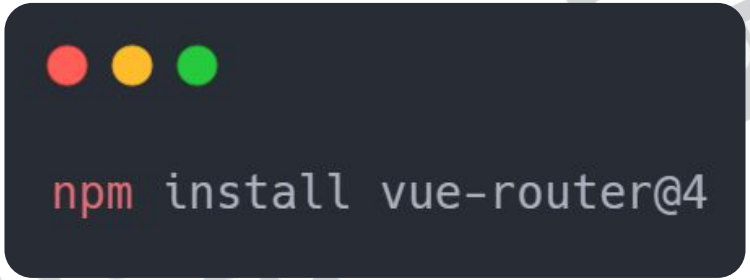
timedoor academy PRO

Practice Creating Home Page, Login and Sign Up Part 1

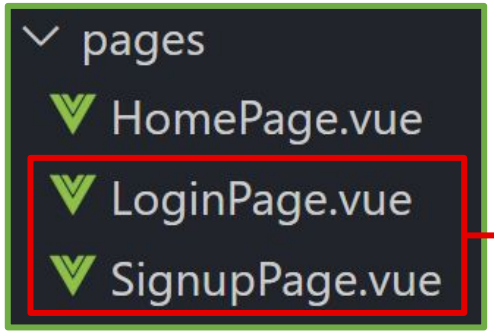
30 Minutes

Amedoor academy PRO

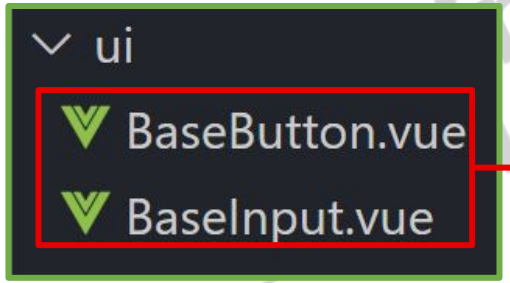
Now we will create Home, Login, and Sign Up pages with **vue-router**. Before that, please install **vue-router** by running the following command in the terminal.



After that, we will prepare the *components* needed to create the **Home, Login,** and **Sign Up** pages on the next slide.



Add two new *components* in the pages folder called **LoginPage.vue** and **SignupPage.vue**.



Add two new *components* in the ui folder called **BaseButton.vue** and **BaseInput.vue.** These two components will be used as the basic components for creating buttons and input forms.

medoor Academy PRO

Add the following code to the **BaseButton.vue component**. Slots are used to rename each button. So that each button can use a different name

src > components > ui > BaseButton.vue

```
<template>
  <button class="btn rounded-pill">
    <slot></slot>
  </button>
</template>
```

Add the following code to the **BaseInput.vue component**. The input tag requires an attribute whose value can be different for each input tag. There are five props that will be used which are **type**, **identity**, **placeholder**, **value**, and **readonly**.

src > components > ui > BaseInput.vue

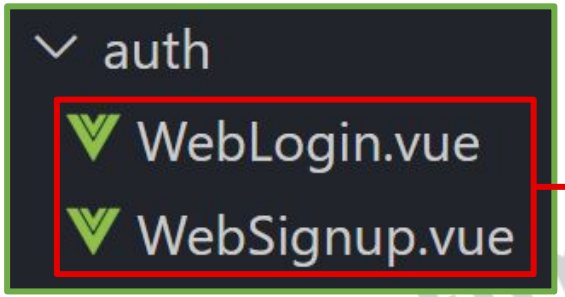
```
<template>
  <div>
    <label :for="identity" class="fw-semibold">
        Firstname <span style="color: #cb3a31">*</span>
    </label>
    <input class="form-control" :type="type" :id="identity"
        :placeholder="placeholder" :value="modelValue"
        :readonly="readonly === '1'"
    />
  </div>
</template>
```

In the **script tag**, add the following code.

src > components > ui > BaseInput.vue

```
<script setup>
  defineProps({
    type: { type: String, require: true },
    label: { type: String, require: true }.
    identiy: { type: String, require: true },
    placeholder: { type: String, require: false },
    readonly: { type: String, require: true, default: "0" }
  })
</script>
```

Now we will create a component that will be displayed on the Login and **Sign Up** pages.



Add 2 file components to the auth folder called bernama **WebLogin.vue** and **WebSignup.vue.**

Afterwards, copy paste the code provided by the following link into the **WebLogin.vue** and **WebSignup.vue** components.

https://bit.ly/3WaHbMg

Now, call the **WebLogin.vue** component in the **LoginPage.vue** and **WebSignup.vue** in the **SignupPage.vue**.

Add the following code to the **SignupPage.vue** component.

src > components > pages > SignupPage.vue

```
<template>
  <web-signup></web-signup>
</template>

<script setup>
    import WebSignup from "../auth/WebSignup.vue"
</script>
```

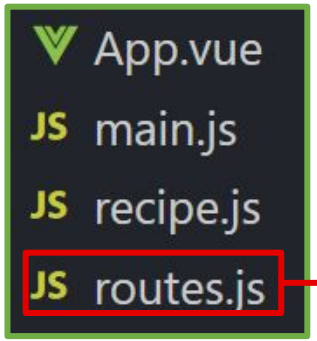
Add this code to the **LoginPage.vue** component.

src > components > pages > LoginPage.vue

```
<template>
    <web-login></web-login>
</template>

<script setup>
    import WebLogin from "../auth/WebLogin.vue"
</script>
```

Next, we need to prepare a *route* for the **Home**, **Login**, and **Sign Up** pages.



In the **src folder**, add a new file called **routes.js**. This file will be used to declare the routes for each page.

src > routes.js

```
export const routes = [

]
```

Add this code to the **routes.js file.**

Add the following code to the **main.js file**.

src > main.js

```
import { createRouter, createWebHistory } from 'vue-router'
import { routes } from './routes'

const router = createRouter({
    history: createWebHistory(),
    routes,
    scrollBehavior(to, from, savedPosition) {
        return { top: 0 }
    }
})

createApp(App).use(router)
createApp(App).mount('#app')
```

Add the following code to the **routes.js file**.

src > routes.js

```
import HomePage from "./components/pages/HomePage.vue"
import LoginPage from "./components/pages/LoginPage.vue"
import SignupPage from "./components/pages/SignupPage.vue"

export const routes = [
  { path: "/", name: "homePage", component: HomePage },
  { path: "/signup", name: "signup", component: SignupPage
},
  { path: "/login", name: "login", component: LoginPage }
];
```

The routes created are routes for the **Home**, **Login**, and **Signup pages.**

Then add **<router-view></router-view>** in **App.vue** so the structure of the code looks like the image below.

src > App.vue

```
<template>
  <div>
    <web-header></web-header>
    <router-view></router-view>
    <web-footer></web-footer>
  </div>
</template>

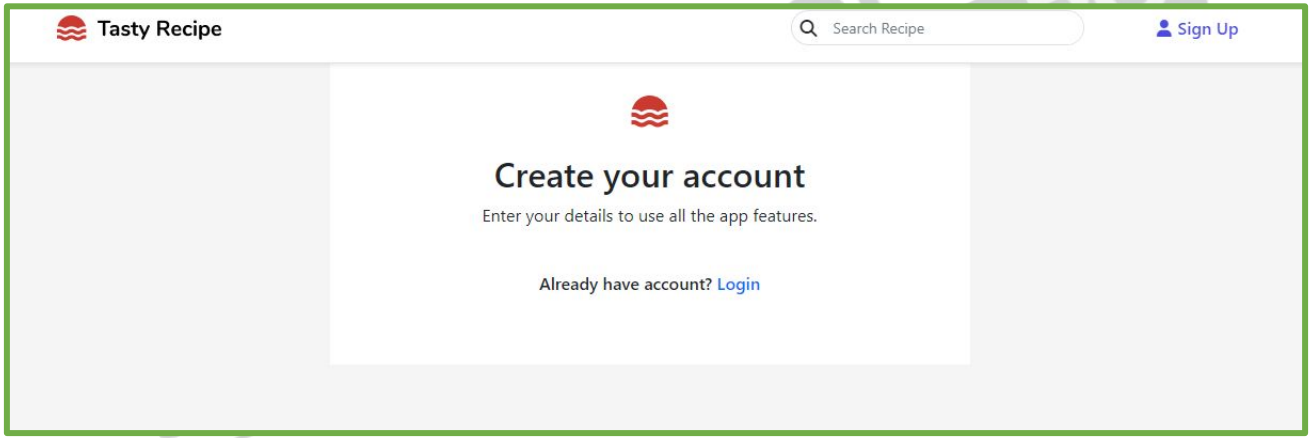
<script setup>
import HomePage from "./components/pages/HomePage.vue"
import WebHeader from "./components/header/WebHeadear.vue"
import WebFooter from "./components/footer/WebFooter.vue"
</script>
```

To make it easier to move pages, we will add **<router-link></router-link>** to the **sign up** button.

src > component > SignupMenu.vue

```
<template>
  <router-link
    to="/signup"
    class="header__signup col-7 col-sm-3 fw-semibold
        d-flex justify-content-evenly align-items-center
        text-decoration-none"
    style="color: #4c4ddc; text-align: right"
  >
    <i class="fa-solid fa-user"></i>
    <p class="mb-0">Sign Up</p>
  </router-link>
</template>
```

Now, try clicking the **Signup** button and make sure the web page will move to the **Signup** page. The display of the **Sign Up** page is as follows.



CHALLENGE

Your task is now as followed:

- Set the Login button located on the Sign Up page so that when it’s clicked, the Login page will open.
- Set the Signup button located on the Login page so that when it’s clicked, the Signup page will open.
- Set the logo contained in the Header so that when it’s clicked, it will lead to the Home page.

timedoor academy PRO

Practice Creating Home Page, Login and Sign Up Part 2

30 Minutes

Timeador academyPRO

Now we will improve the appearance of the **Login** and **Signup** pages. Pay attention to the following code snippet contained in the **WebSignup.vue component.**

src > components > auth > WebSignup.vue

```
<form class="mt-3">
        <div class="row">
          <div class="col-md-6"><!-- Firstname --></div>
          <div class="col-md-6"><!-- Lastname --></div>
        </div>
        <div class="my-4"><!-- Username --></div>
        <div class="my-4"><!-- Email --></div>
        <div class="my-4"><!-- Password --></div>
        <div class="my-4"><!-- Confirm Password --></div>
</form>
```

Each div will be filled with the **BaseInput.vue component.**

First call the **BaseInput.vue** component in the **WebSignup.vue component.**

src > components > auth > WebSignup.vue

```
<script setup>
import BaseInput from "../ui/BaseInput.vue";
</script>
```

The **BaseInput.vue** component needs 3 props that are required which are *type*, *id*, and *placeholder*. Now we will create an **input** for***firstname*** with the value of the props used are as follows.

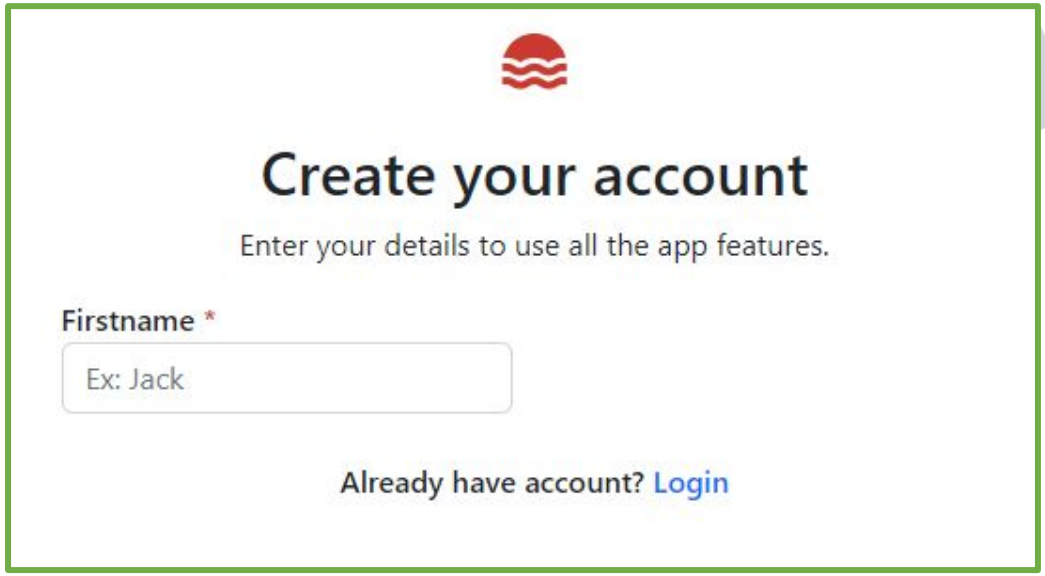
- type: “text”
- identity: “firstname”
- placeholder: “Ex: Jack”
- label: “Firstname”

In the **div** tag for **username**, add the following code.

src > components > auth > WebSignup.vue

```
<base-input
    type="text"
    identity="firstname"
    placeholder="Ex: Jack"
    label="Firstname">
</base-input>
```

The **Sign Up** page display will be as follows.

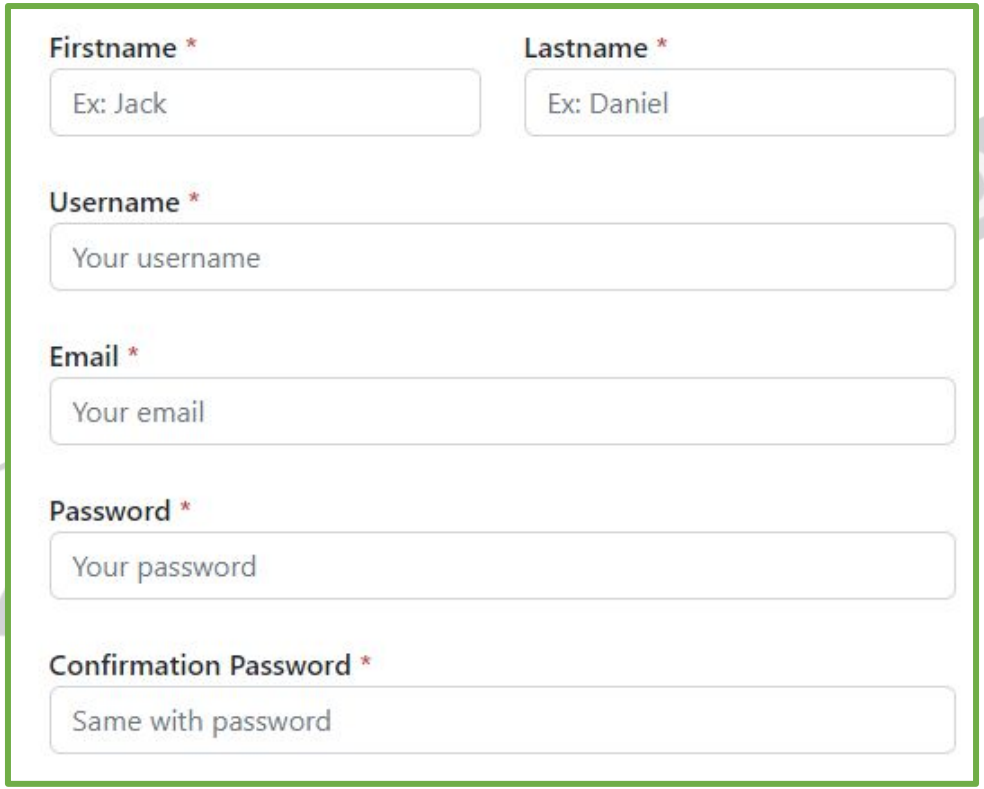


Now try to add another **input form** with the following conditions.

Input	Props
Lastname	type: “text”, identity: “lastname”, placeholder: “Ex: Daniel”, label: “Lastname”
Username	type: “text”, identity: “username”, placeholder: “Your username”, label: “Username”
Email	type: “email”, identity: “email”, placeholder: “Your email address”, label: “Email”
Password	type: “password”, identity: “password”, placeholder: “Your password”, label: “Password”
Confirm Password	type: “password”, identity: “confirmationPassword”, placeholder: “Same with password”, label: “Confirmation Password”

timedoor academy PRO

Make sure the end result looks like this.



On the **register** page, there will also be a feature to add a profile photo.

src > components > ui > BaseInput.vue

```
<label :for="identity" class="fw-semibold"
      >{{ label }} <span style="color: #cb3a31">*</span>
      <slot></slot>
</label>
```

<slot></slot> is filled with the code required for the feature to add a profile photo.

Add a new **prop** called **isImage**, whose default value is false.

src > components > ui > BaseInput.vue

```
isImage: { type: Boolean, require: true, default: false }
```

Then fix the **class attribute** in the **input tag** to be like the following.

src > components > ui > BaseInput.vue

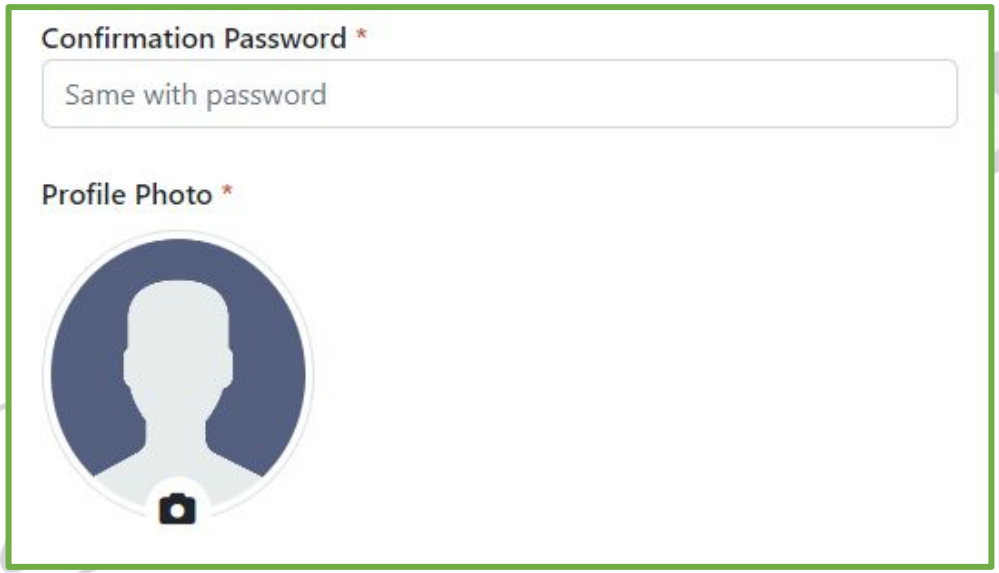
```
:class="[{ 'd-none': isImage}, 'form-control']"
```

Add the following code to the ***confirmation password*** input form.

src > components > auth > WebSignup.vue

```
<div class="my-4">
    <base-input type="file" identity="recipeImage"
        label="Profile Photo" isImage="true">
        <div>
            <div class="border p-1 mt-2 rounded-circle" >
                <img src="../../assets/images/user.png"
                class="rounded-circle"
                width="140" height="150" style="object-fit:
            cover"/>
            </div>
            <div class="text-center"
                style="transform: translateY(-24px)" >
                <i class="fa-solid fa-camera fs-5 p-2
                rounded-circle bg-white" ></i>
            </div>
        </div>
    </base-input>
</div>
```

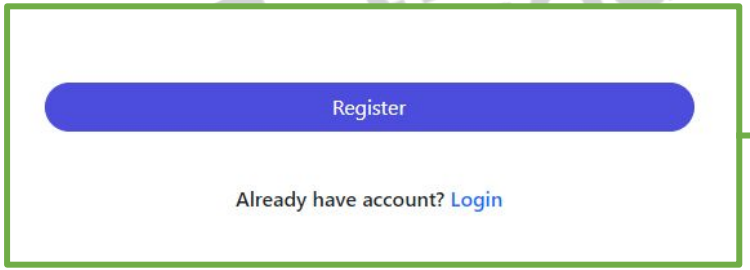
Make sure it looks like the following.



Finally, add the following code before the closing **</form> tag**

src > components > auth > WebSignup.vue

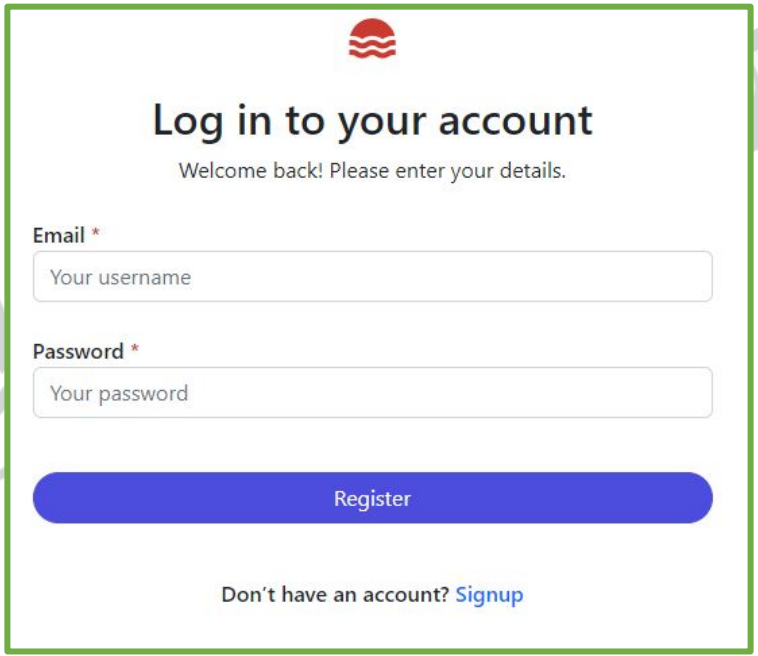
```
<base-button
    class="login w-100 my-3">Register
</base-button>
```



As a result, on the web page there will be a **Register** button like the following.

Challenge

Now the challenge that you have to do is to improve the appearance of the login page so that it looks like the following.



Introduction to Vuex

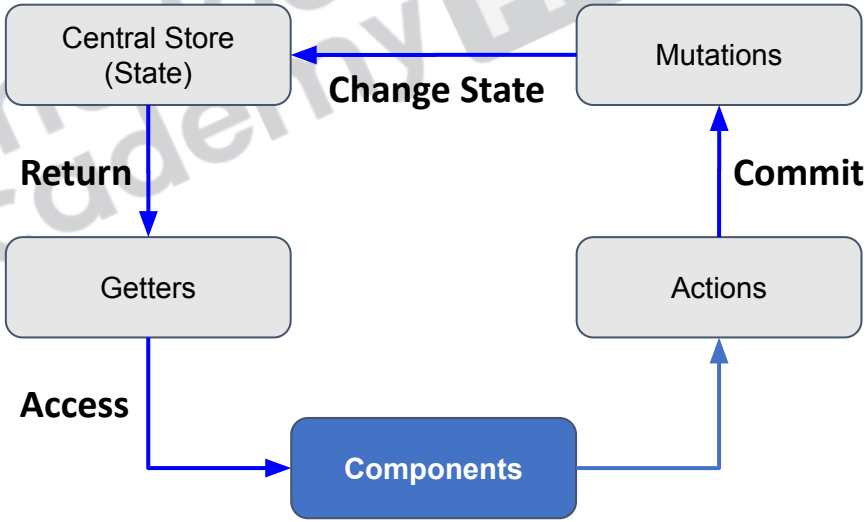
5 Minutes

timedoor academy PRO

What is Vuex?

Vuex is used when an **application** becomes larger and the more data you have, the more difficult it is to manage. **Vuex** is a ***state management pattern*** in **Vue.js,** it's job is to be a ***centralized store*** to all ***components*** in the **Vue.js** application that has been made.

Centralized State (***store***) is a place that will be used as data storage and can be used by all **component**.



Before using **Vuex**, run the following code in the terminal.

```
npm install vuex@next --save
```

```
import { useStore } from 'vuex'

export default {
  setup () {
    const store = useStore()
  }
}
```

In order to use **Vuex** in the **component**, **import useStore** from **vuex** as follows.

State

State is an object where all the **application** data resides.

```
export const store = new Vuex.Store({
  state: {
    color: ['red', 'blue', 'green'],
    price: 100000
  }
})
```

```
store.state.namaProperty
```

How to access ***state*** from ***component***.

Getter

Getter functions to access **state**. Through **getter,** data from the state can be processed first, the **state** will be retrieved like a **computed** function.

```
getters: {
    getterName(state) {
        // code here
      return data;
    }
}
```

```
store.getters.getterName
```

Cara memanggil ***getter*** dari ***component***.

timedoor academy

Mutation

Every change of data in ***state*** like **adding data, deleting,** or even **updating data** must go through a **mutation.**

```
mutations: {
    mutationName(state, payload) {
      state.stateName = newValue
    }
}
```

```
store.commit('mutationName', data)
```

How to call a ***mutation*** from ***component***.

Action

Action is similar to **mutation**, only **action** is used to make changes to data **asynchronously.**

```
actions: {
  actionName(context) {
    context.commit('mutationName')
  }
}
```

```
store.dispatch('actionName', data)
```

How to call ***action*** from ***component***.

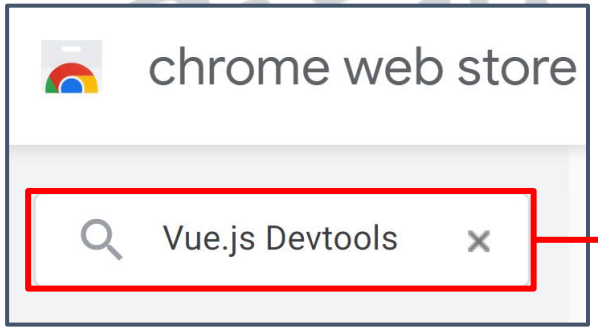
Chrome Extension for Vue.js

5 Minutes

Hamedoon academy PRO

Now, we are going to add **extensions** that will help us while developing our **Vue.js** application. Follow these steps.

1. In the **Chrome** *browser*, type/copy the following link : **https://chrome.google.com/webstore/category/extensions?hl=en-US**

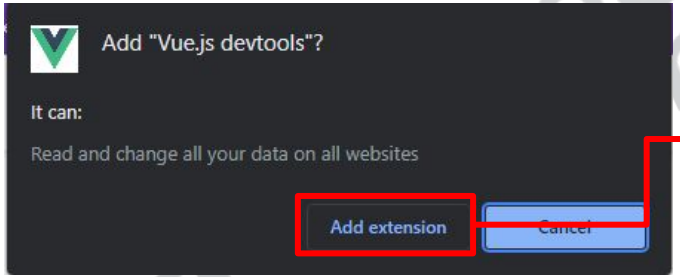
2. 

In the *search option*, type the keywords“**Vue.js Devtools**”

3

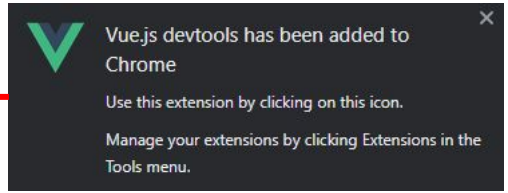


Click on“**Add to Chrome**” to add the *extension* to Chrome.



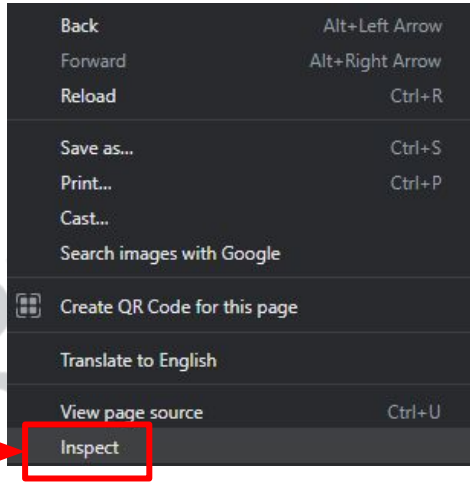
After that a notification will appear as follows. Click “**Add extension**”

If the following notification appears, then the *extension* has been successfully added.



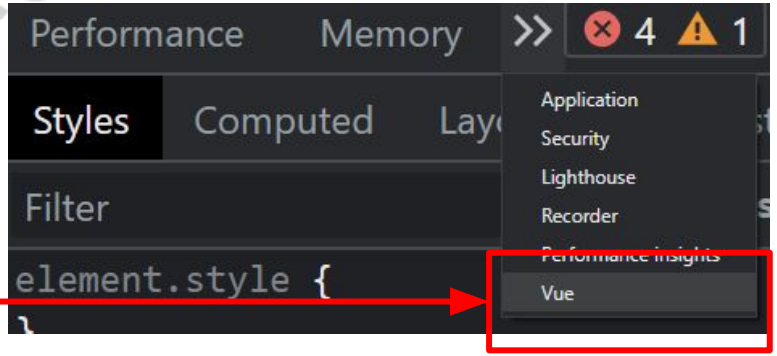
4

Right click on the **webpage** from the **Vue project** that's being worked on then select **Inspect**.



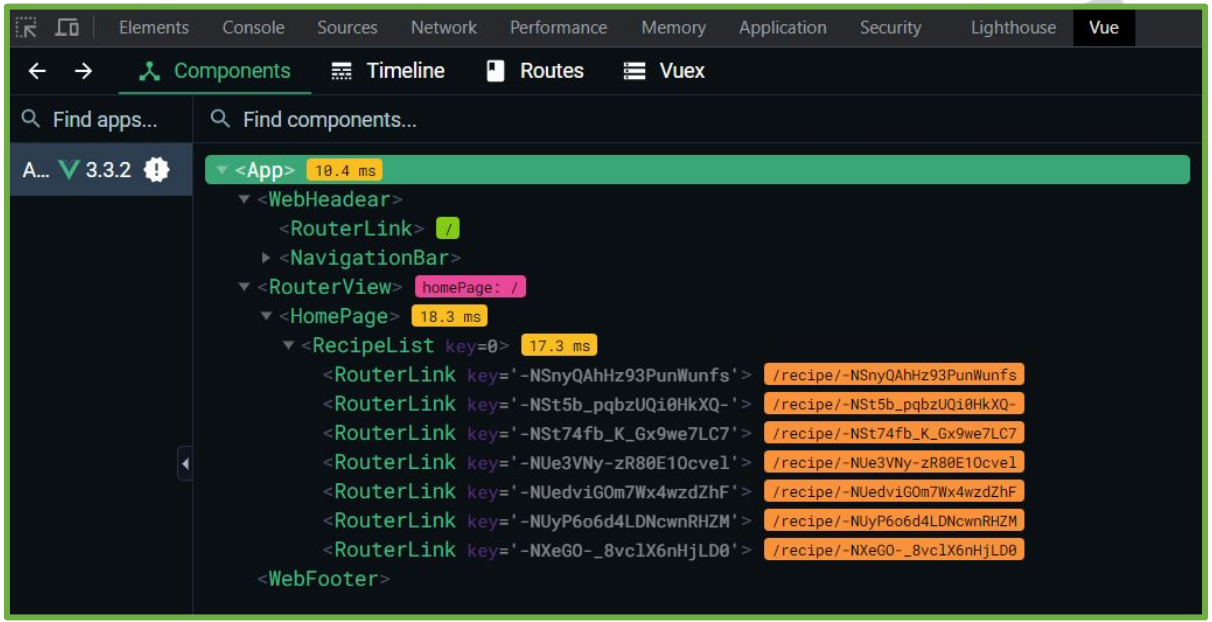
5

Then select **Vue**.



timedoor academy

After that, a display like the following will appear. With **Vue.js Devtools**, we can see the currently rendered component.



Async/Await in Javascript

15 Minutes

timedoor academy PRO

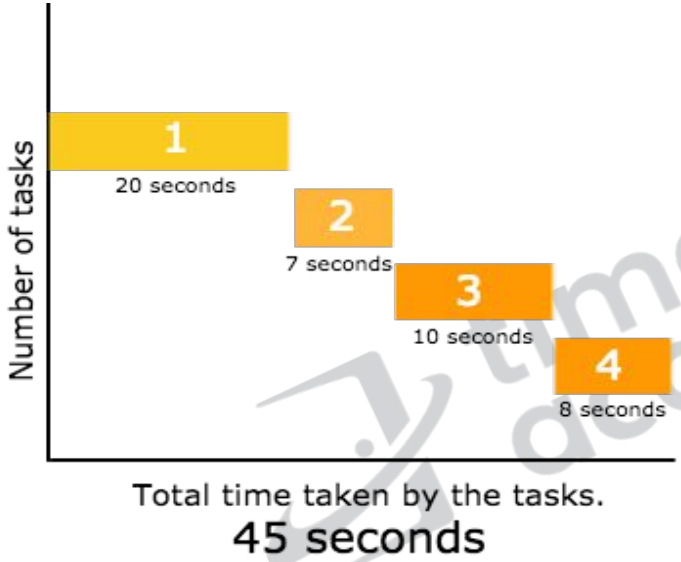
Do you remember about Actions in Vuex?

Action is used to change a **state asynchronously.** Do you know what **asynchronous** means?

By default, JavaScript code is processed line by line which means that each line of code will not be executed before the line of code currently running has finished processing.

The rogramming model that is run like this is known as ***synchronous*** or in short, ***sync***.

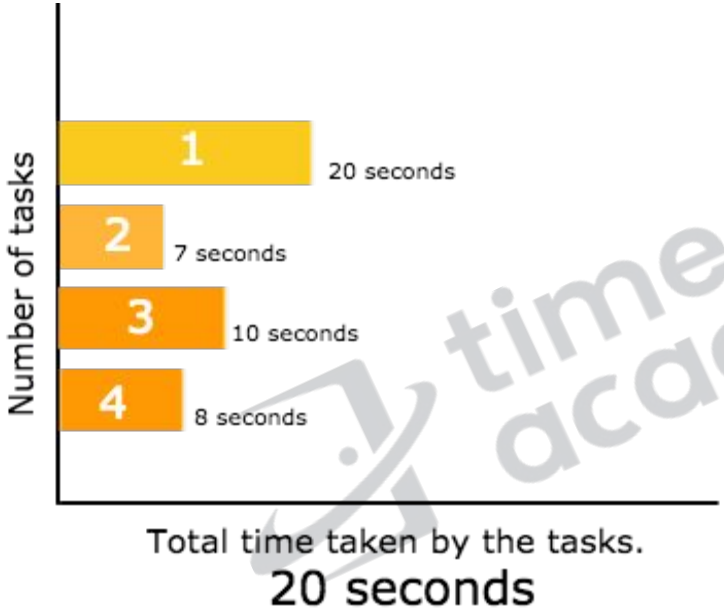
Synchronous



The illustration on the side shows that if the program is executed ***synchronously***, the task that is executed next must wait for the previous task to finish executing first.

tmedoor academy PRO

Asynchronous



Whereas **asynchronous or *async***, the next task can be executed without the need to wait for the previous task to finish executing first as illustrated on the side.

timedoor academy PRO

Async/Await is one way that can be used to handle *asynchronous* processes in javascript. `async` can be used in a function to indicate that the function is running *asynchronously*.

The syntax of **async** in a function is as follow.

```
async function name(parameter1, parameter2) {
    // statements
}
```

The `await` *keyword* is used in *async functions* to wait for the asynchronous process to finish.

The syntax to use `await` is as follow.

```
let result = await promise;
```

If you don't use **await**, then the next code will be executed before the **asynchronous** process finishes.

```
let promise = new Promise(function(resolve, reject) {
    setTimeout(function() {
      resolve('Promise resolved')
    }, 4000)
});

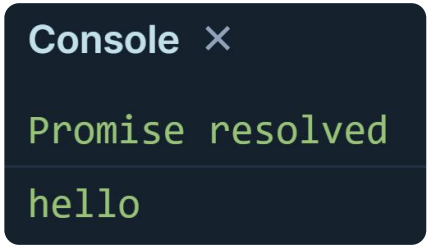
async function asyncFunc() {
    let result = await promise;

    console.log(result)
    console.log('hello')
}

asyncFunc()
```

Waiting for the *asynchronous* process to finish

In the example of using **async/await** above, **await** waits for the asynchronous process to finish, so the result will be as follows.



Error Handling

When using `async/await`, then `try/catch` block can be used to handle the *error* when the *asynchronous* process is running.

```
async function asyncFunc() {
    try {

        let result = await promise;
        console.log(result);
    }
    catch(error) {
        console.log(error);
    }
}
```

If the **async/await** process is successful, then the code in the **try block** will be executed. If it fails, then the **catch block** will be executed.

Firebase

15 Minutes

Timedoor academy PRO

What is **Firebase?**



Firebase is a **BaaS (*Backend as Service*)** provided by **Google** to speed up **developer** work. **Firebase** was first founded in 2011 by **Andrew Lee** and **James Tamplin**. In October 2014, the company was acquired by **Google**. Firebase has many features that can be used, one of which is the **Realtime Database** which will be used in this class.

Firebase Realtime Database

The **Firebase Realtime Database** is a **cloud-hosted database** and data is stored and executed as **JSON. Realtime database** will be synchronized in ***realtime*** to every connected user. This will make it easier to manage a database on a fairly large scale.

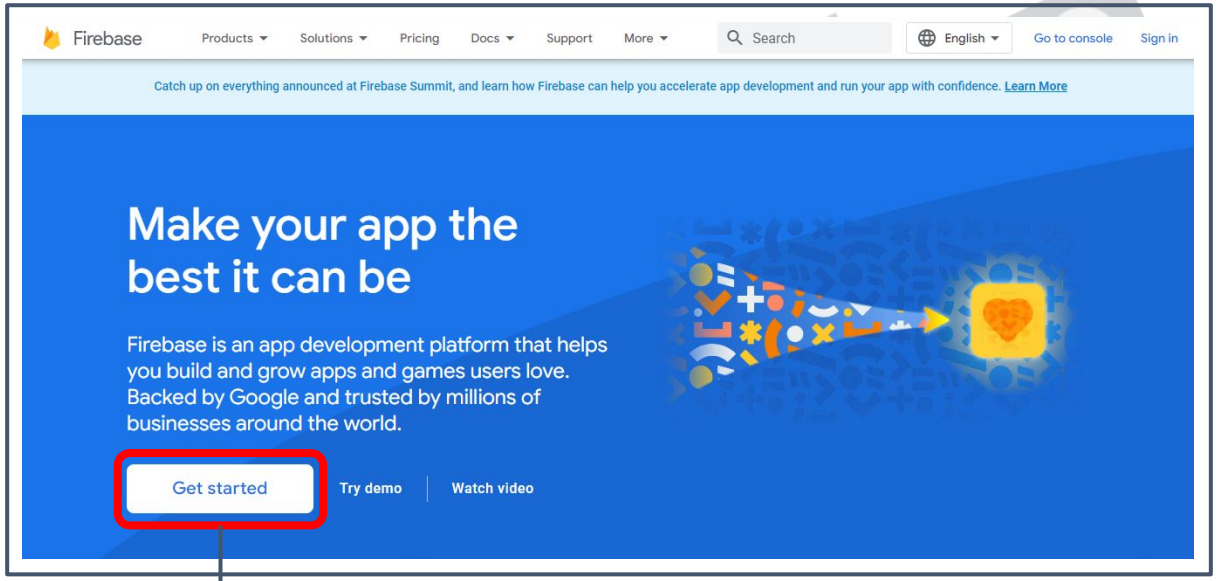
timedoor academy Pro

Firebase Configuration

Ahmed Nar Academy PRO

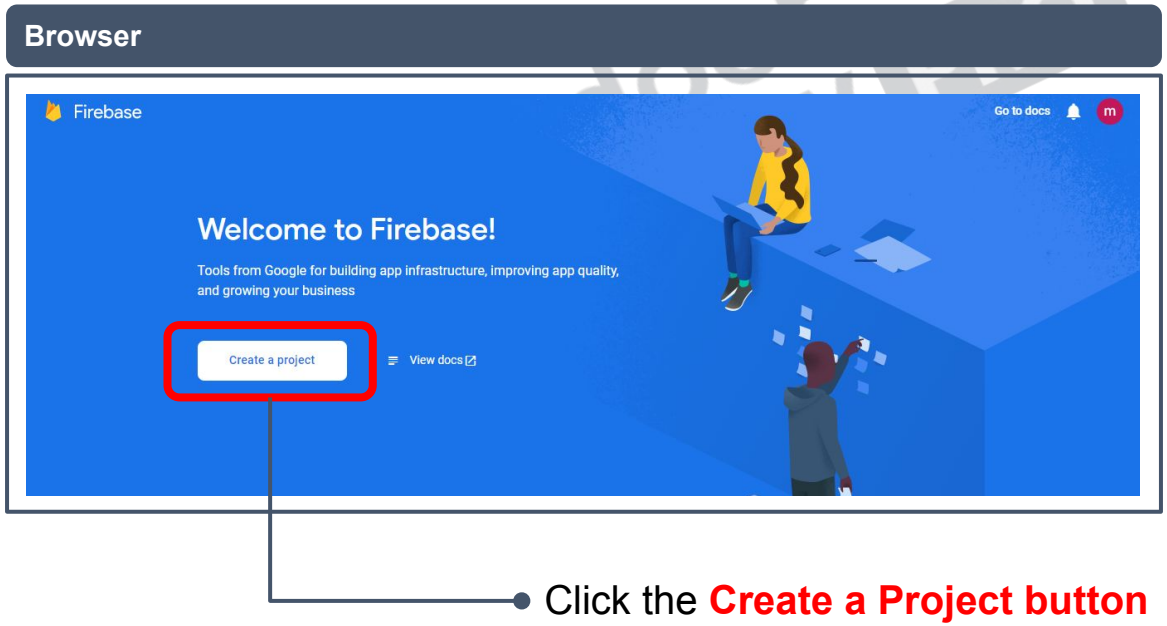
1 **First,** go to the following **Link:** https://firebase.google.com/ then click on the **Get Started button**

Browser



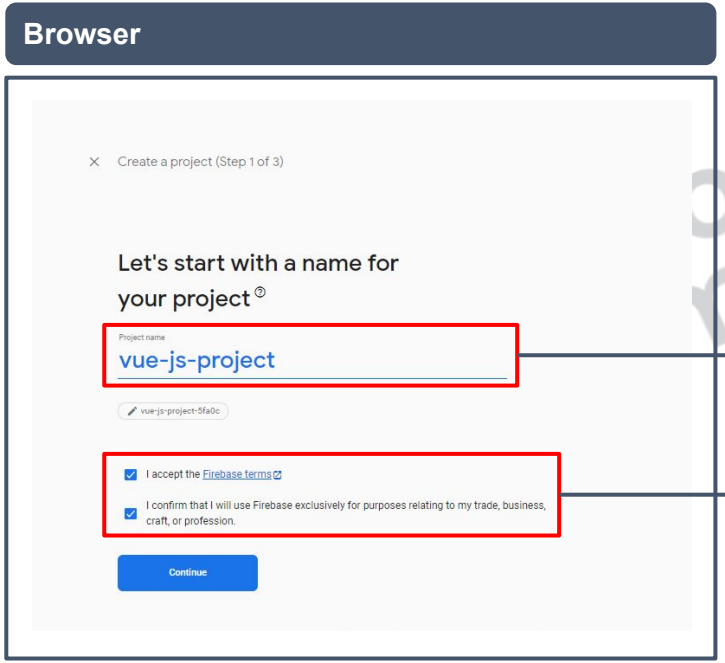
2 Log in using the **Google account** that you have.

3 After successfully logging in, a screen like the following will appear.



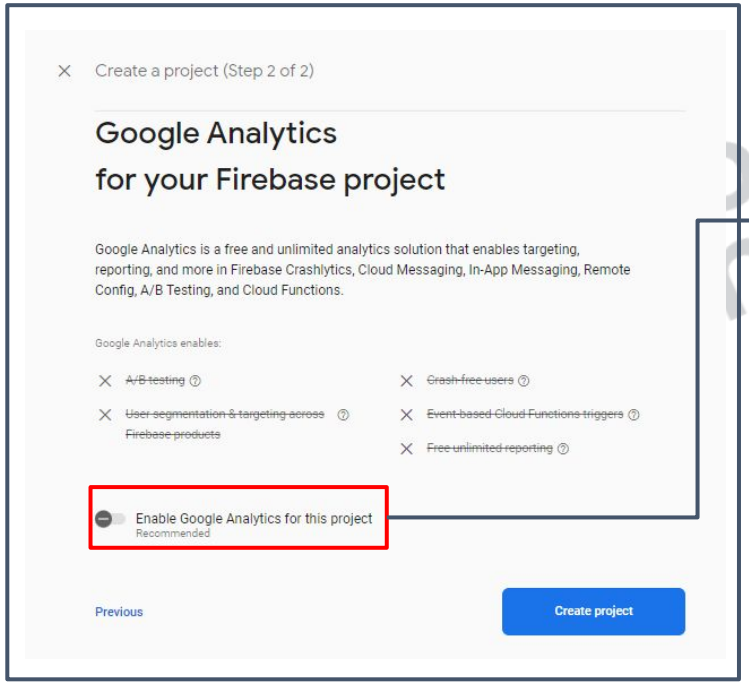
4

Next we need to fill in the data needed to create a new project.



Complete the following data then select **continue**

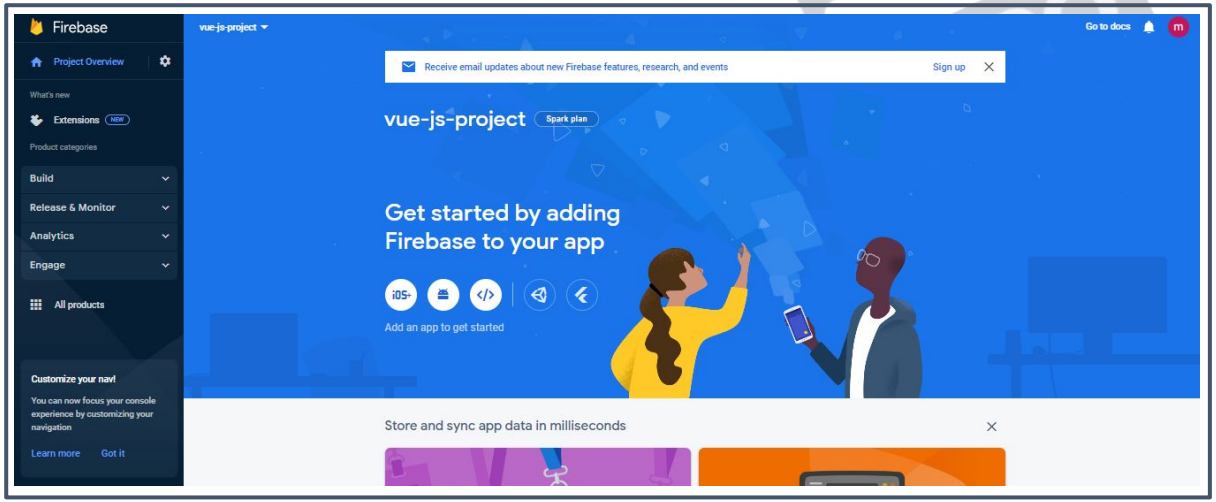
Browser



Make sure this part here is **disabled**. Then select **Create Project.**

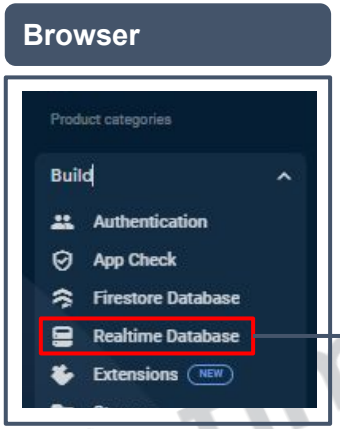
After the project is successfully created, the following screen will appear.

Browser

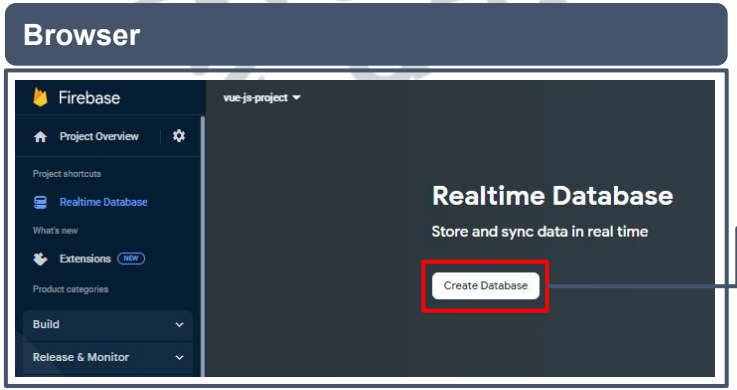


5

Now we will add a **realtime database** which will be used to store the data we need later.

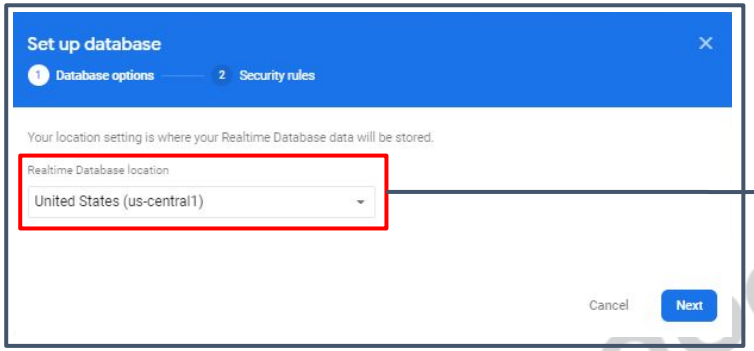


In the **sidebar**, select the **Build** menu then select **Realtime Database**



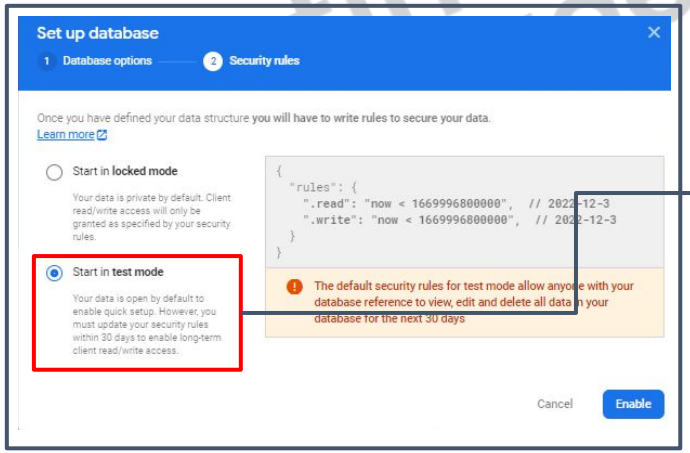
Then select **Create Database**

Browser



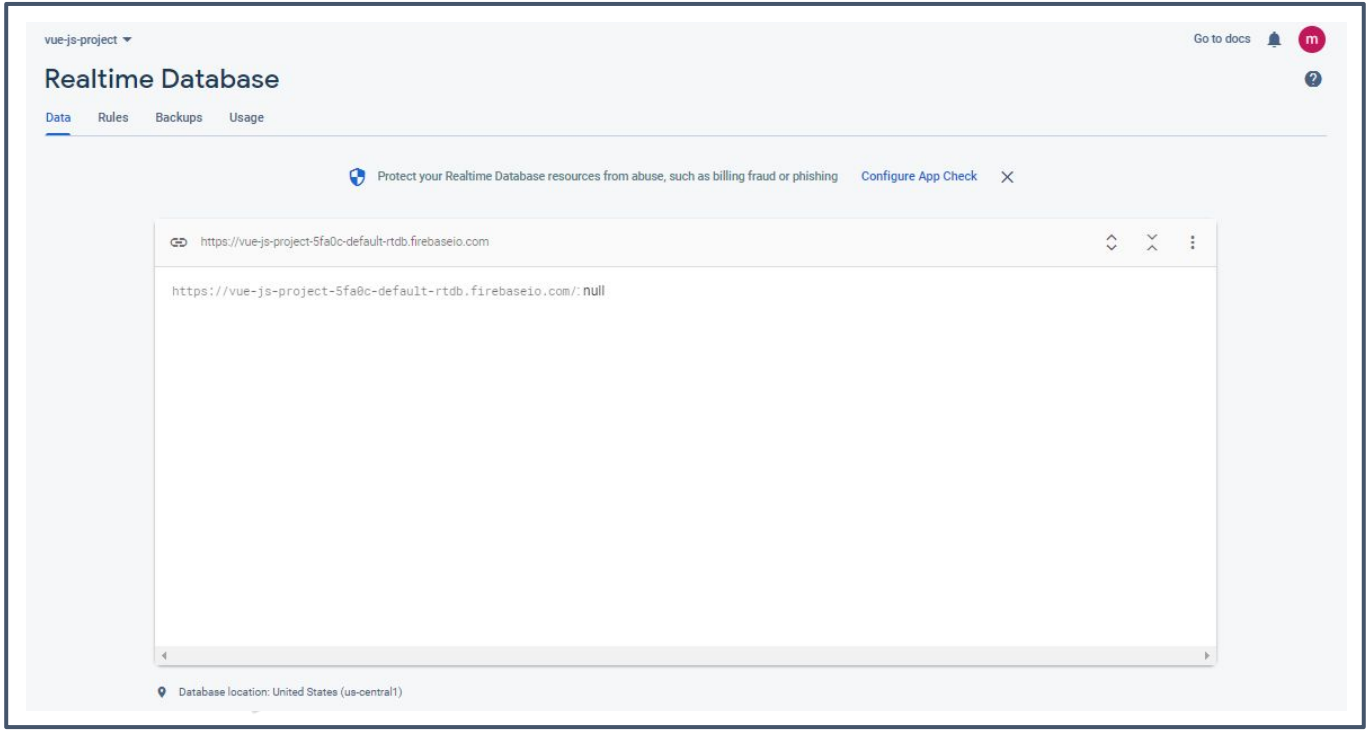
Set the **Realtime Database location** to **United State (us-central1),** the select **next**

Browser



Select **Start in test mode** then select **enable.**

Browser



Finally a display like this will appear. If a display like this appears, then we have successfully created a **Realtime Database on Firebase.**

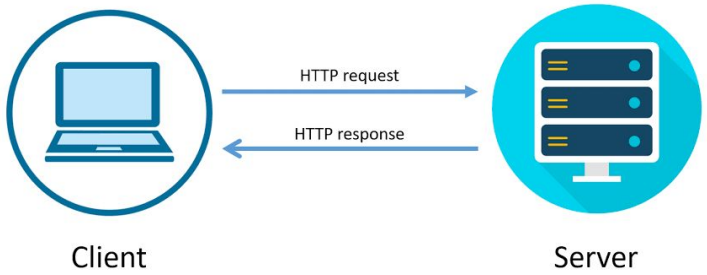
Axios

15 Minutes

timedoor academy PRO

What is **Axios?**

Axios is a **library** that's used to do a ***HTTP Request***. ***HTTP Request*** is a process of **requesting data** from the **client (Browser)** to the **server**.



Axios allows us to make **requests** to an **endpoint**. **Endpoints** can be external APIs or custom built **backends**.

Axios can also use several methods that can be used to make requests, namely as follows.

- **GET**
  Request is used to fetch or request data.

- **POST**
  Request is used to create new data

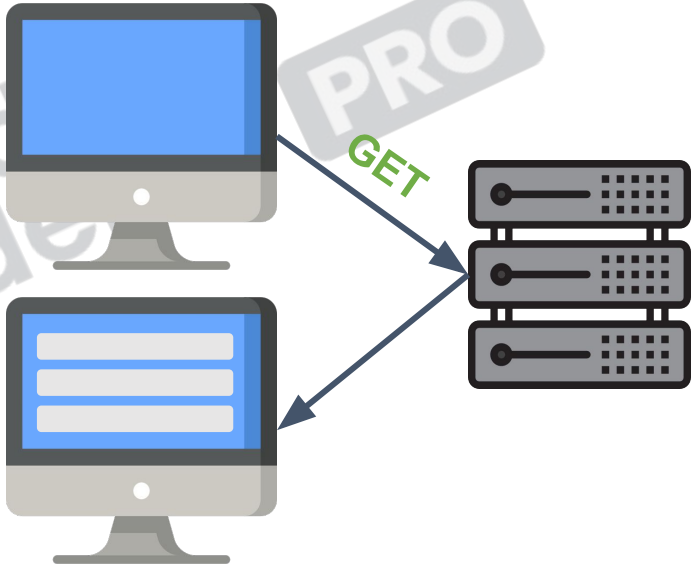
- **PUT**
  Request is used to update data

- **DELETE**
  Request is used to delete data.

timedoor academy PRO

By making a **request**, the **API** is **expected** to be able to carry out a process according to the **requested request**.

Look at the example to the side. If you make a **GET** *request*, you are expected to get some data to display.



Axios needs to be installed in the application used by running the following code in the terminal.

```
npm install axios
```

Always ***import axios*** every time you use it.

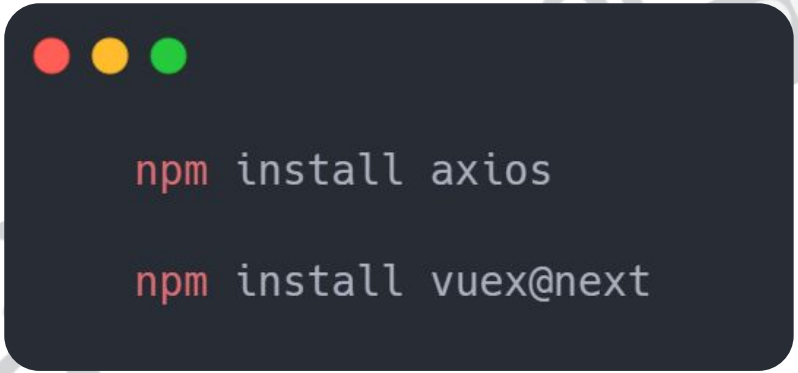
```
import axios from "axios"
```

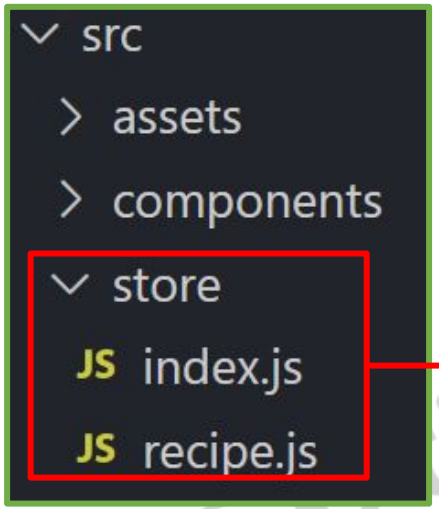
Practice Displaying The Recipe Data Stored in The Firebase

30 Minutes

Ahmed Adel Academy PRO

In this exercise, we will display recipe data stored in **Firebase**. We'll be using **Vuex** and **Axios** during this exercise. Please run the following command in the terminal of the **Tasty Recipe project.**





Create a new **folder** named **store** in the **src folder**. Add two new files in the **store folder** named **index.js** and **recipe.js**.

All code related to managing **recipe data** such as retrieving data from **firebase**, adding new recipes, updating recipe data, and deleting data will be placed in the **recipe.js file.**

After that, add these two codes in the **main.js file.**

src > main.js

```
import { store } from './store/index'

app.use(store)
```

src > store > index.js

```
import { createStore } from "vuex";
import recipe from "./recipe";

export const store = createStore({
    modules: {
        recipe,
    },
})
```

In the **index.js file** add the following code to add the module recipe that was created.

src > store > recipe.js

```
import axios from "axios";

export default {
    namespaced: true,
    state() {
        return {
            recipes: []
        }
    },
    getters: {},
    mutations: {},
    actions: {}
}
```

Add the following code to the **recipe.js file.**

The recipes state will be used to store the entire data obtained from **Firebase**.

Add a **mutation** named **setRecipeData. This mutation** is used to update the contents of the **recipes state..**

src > store > recipe.js

```
mutations: {
     setRecipeData(state, payload) {
          state.recipes = payload
     }
}
```

Recipe data is stored on the firebase so the code will run **asynchronously**.
Add an **action** named **getRecipeData** with the following code structure.

src > store > store > recipe.js

```
actions: {
     async getRecipeData({commit}) {
          try {

          } catch (err) {
               console.log(err)
          }
     }
}
```

Async/Await because the process runs **asynchronously**.

src > store > store > recipe.js

```
actions: {
   async getRecipeData({commit}) {
      try {
            const { data } = await axios.get(
                "https://recipe-vue-batch2-default-rtdb.firebaseio.com/recip
                es.json")
      } catch (err) {
            console.log(err)
      }
   }
}
```

Add the following code to the **try block. The get method** is used to fetch data from firebase.

src > store > store > recipe.js

```
actions: {
     async getRecipeData({commit}) {
          try {
               const { data } = await axios.get(
                    "https://recipe-vue-batch2-default-rtdb.firebaseio.com/
               recipes.json")

          } catch (err) {
               console.log(err)
          }
     }
}
```

recipes.json refers to where recipe data is stored in the realtime database.

src > store > store > recipe.js

```
const { data } = await axios.get(
     "https://recipe-vue-batch2-default-rtdb.firebaseio.com/recipes.json")

const arr = []
for( let key in data ) {
     arr.push({id: key, ...data[key]})
}

commit("setRecipeData", arr)
```

Add the following code to the **try block**. This code will be used to process data obtained from **firebase** before it is stored in **state**.

Next, call the **setRecipeData** mutation to store the obtained recipe data in the **recipe state.**

src > components > pages > HomePage.vue

```
<script setup>
import RecipeList from "../recipe/RecipeList.vue";
import RECIPE_DATA from "../../recipe.js"
const recipeList = RECIPE_DATA
</script>
```

Previously, recipe data was stored in the **recipe.js file**. Since the recipe data is now stored on **firebase**, remove the following code.

src > components > pages > HomePage.vue

```
<script setup>
import RecipeList from "../recipe/RecipeList.vue";
import { onMounted, ref } from "vue";
import { useStore } from "vuex";

const store = useStore();
const recipeListStatus = ref(false);
const recipeList = ref();
</script>
```

Add the following code to the **HomePage.vue component.**

src > components > pages > HomePage.vue

```
onMounted(async () => {
  try {
    await store.dispatch("recipe/getRecipeData");
    recipeListStatus.value = true;
    recipeList.value = store.state.recipe.recipes;
  } catch (error) {
    console.log(error);
  }
});
```

Add the following code to the **HomePage.vue component.**

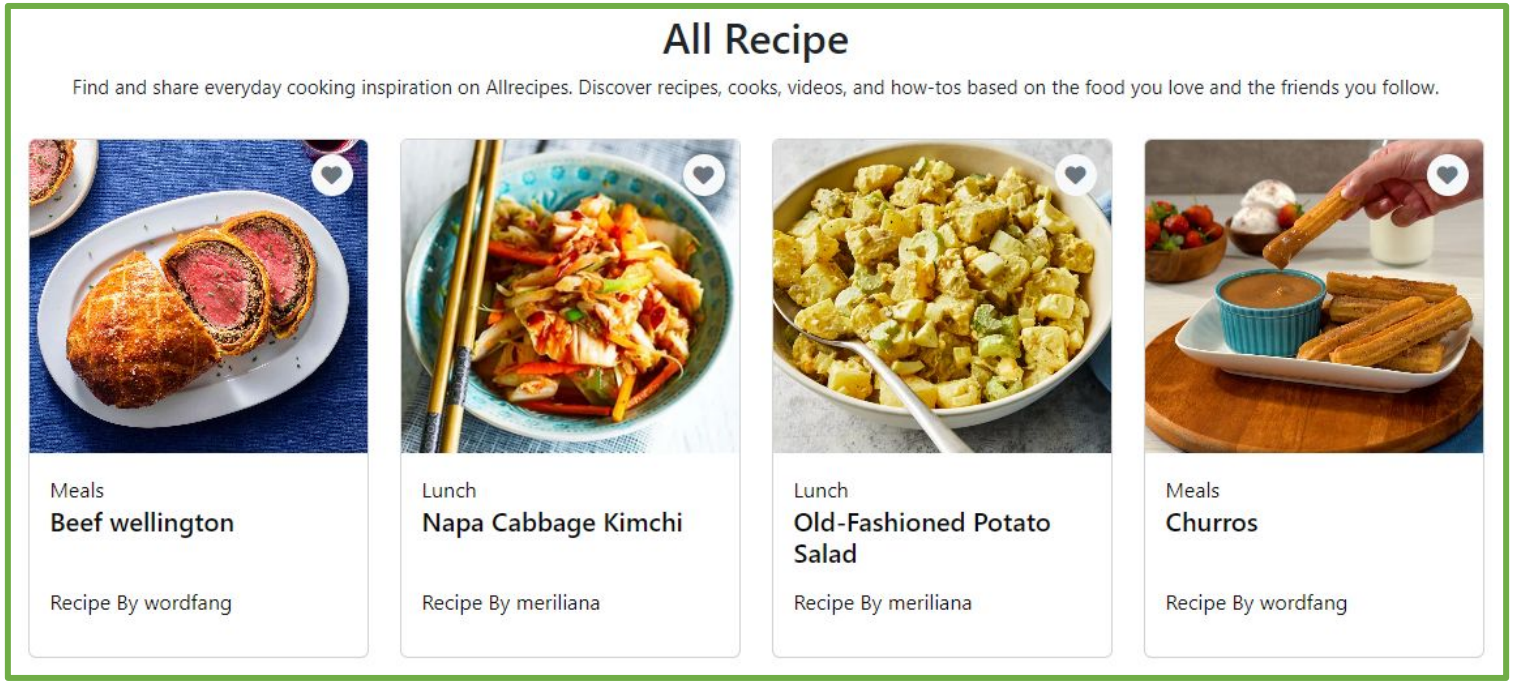
src > components > pages > HomePage.vue

```
<recipe-list
    :recipes="recipeList"
    v-if="recipeListStatus">
</recipe-list>
```

Add the following code to the **recipeListStatus** component call.

recipeListStatus is valued **false** when the **recipes state is empty** and valued **true** when the **recipes state is filled.**

Make sure the **recipe data** successfully appears on the **main web page.**



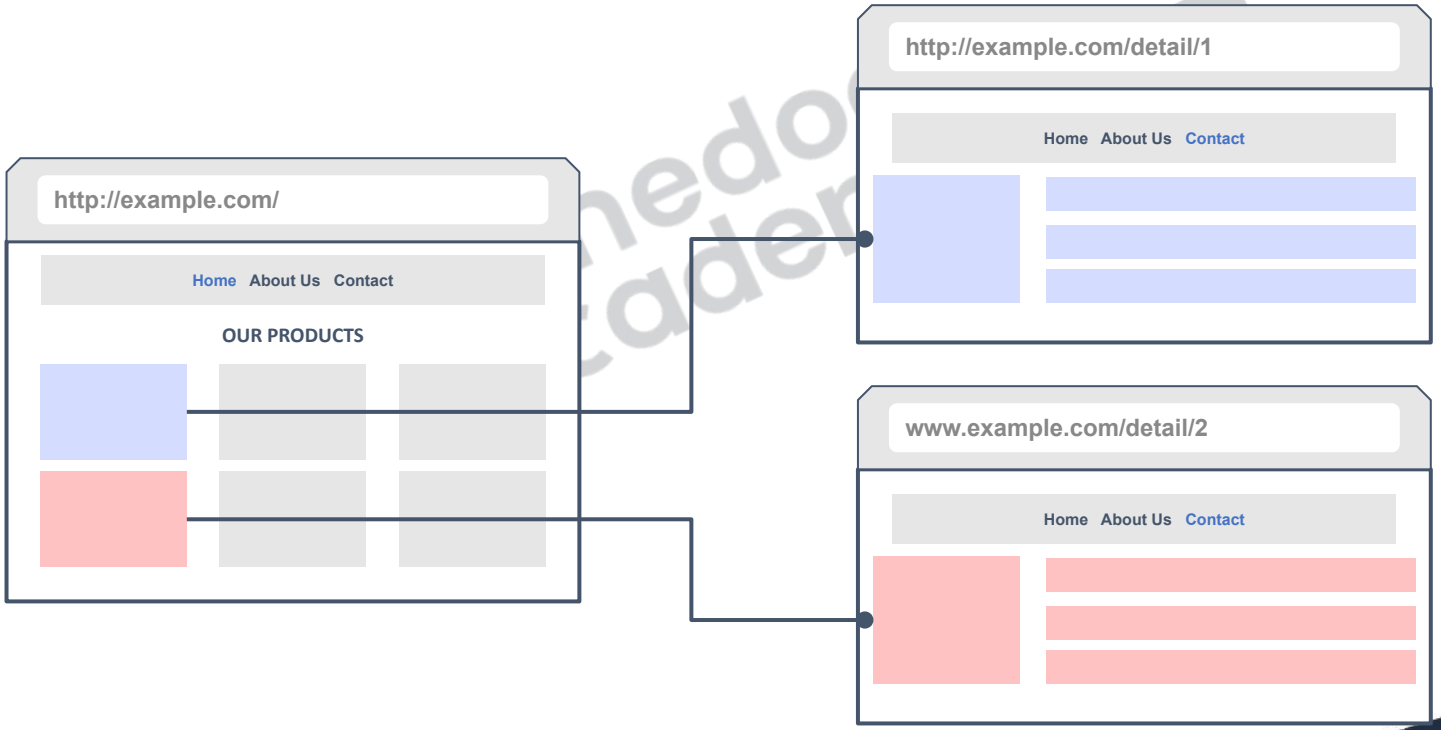
Dynamic Route

10 Minute

timedoor academy PRO

Dynamic Route

Dynamic route is a condition where the **website** opens a **page** with the same **component** but with different data. **Dynamic routes** are often used to open a detail page.



Dynamic route uses **params** in the **property path**. **Params** is symbolized by the **colon symbol (:).**

```
{ path: '/user/:id', name: 'userPage', component: User}
```

Params

If the **route** created is a **Dynamic Route**, then the **to attribute** can be written in the following way.

```
:to="'/url/' + id"
```

How to write the to attribute when filled with the path property

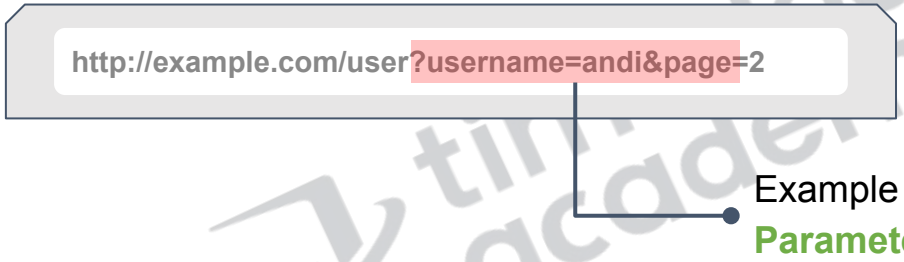
```
:to="{ name: route-name, params: id }"
```

How to write the to attribute when filled with the property name

timedoor academy PRO

Query Parameter

Query Parameter is an **optional value** that appears after the **question mark (?)** on a **url**.



Query Parameter is usually used when there is a process of filtering or searching data.

```
:to="{ name: route-name, params: id, query: { data } }"
```

Query Parameter is written like this. The `data` can be replaced with the data used.

timedoor academy PRO

Vue Router uses **`useRoute()`** and **`useRouter()`** which is a function owned by **composition**. These two functions can be used to access **route** and **router** information between components.

`useRouter()`

There are several common *methods* provided by **`useRouter()`** which are as followed:

- **`push`** : used to navigate to a new route.
- **`replace`** : used to replace the current route with a new route.
- **`go`** : used to navigate to a specific historical position associated with the current route.

`useRoute()`

Is used to access **current route** information. The information provided is in the form of an object that contains properties such as **`path`**, **`parameter`**, **`query`**, and **`meta`**.

tinedoor academy PRO

useRoute() and **useRouter()** is called using this way.

Component

```
<script setup>
import { useRoute, useRouter } from "vue-router"

const route = useRoute()
const router = useRouter()
</script>
```

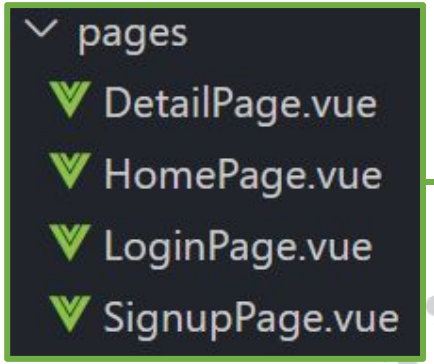
After that you can use commands like **route.params or router.push** to use the functionality of this **Vue Router.**

Practice Adding Detail Page Part 1

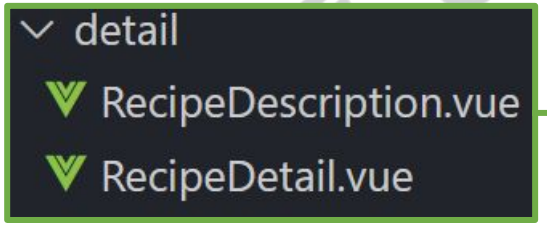
30 Minutes

Timedoor Academy PRO

Now we will add a **detail page** to the **recipe web page**. Please open the **recipe project**, then follow the steps below.



Create a new file called **DetailPage.vue** in the pages folder.



Add **two new files** in the **detail folder** like the image to the side.

timedoor academy PRO

Copy the code in the following **link https://bit.ly/3MT45F6** then paste it in the **DetailPage.vue component.**

Call the **RecipeDescription.vue component** in the **RecipeDetail.vue component** like so.

src > components > detail > RecipeDetail.vue

```
<template>
    <div class="col-12 col-sm-8">
        <recipe-description></recipe-description>
    </div>
</template>

<script setup>
import RecipeDescription from "./RecipeDescription.vue"
</script>
```

Add the following code to the **DetailPage.vue component.**

src > components > pages > DetailPage.vue

```
<template>
    <main>
        <div class="container-md my-5 py-5" >
            <div class="row my-4">
                <div class="col-12 col-sm-2"></div>
                <recipe-detail></recipe-detail>
                <div class="col-12 col-sm-2"></div>
            </div>
        </div>
    </main>
</template>

<script setup>
import RecipeDetail from "../detail/RecipeDetail.vue"
</script>
```

Add the following code to **routes.js. This route** will be used by the detail page of the selected recipe.

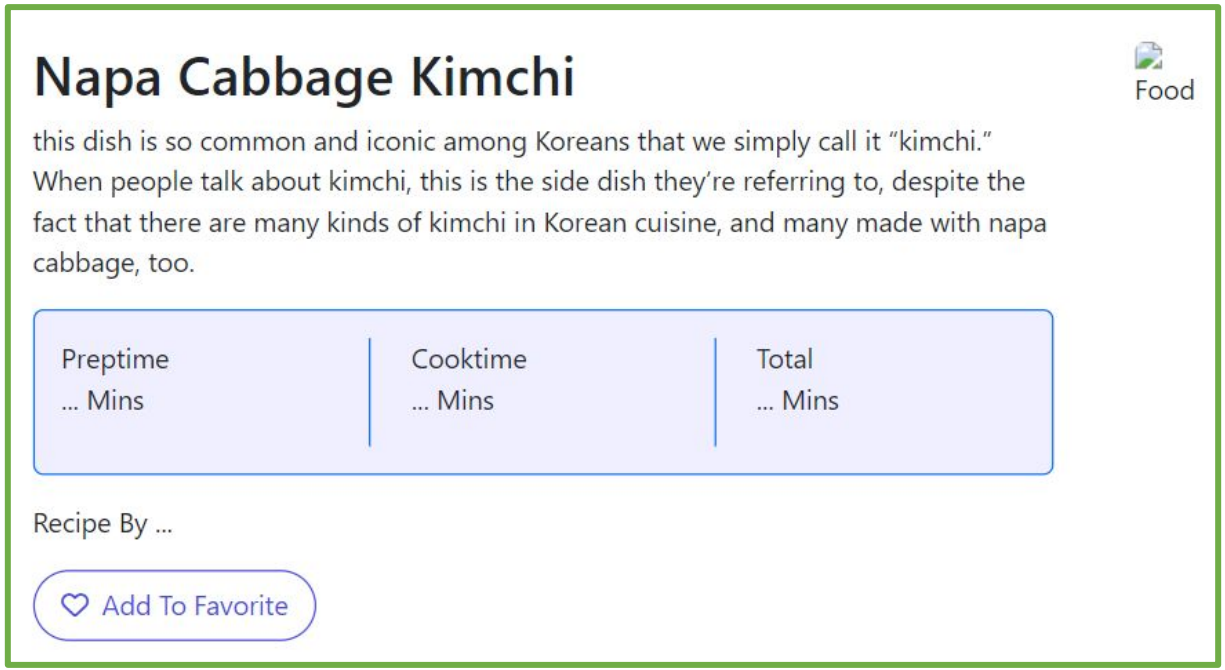
src > routes.js

```
import DetailPage from "./components/pages/DetailPage.vue"

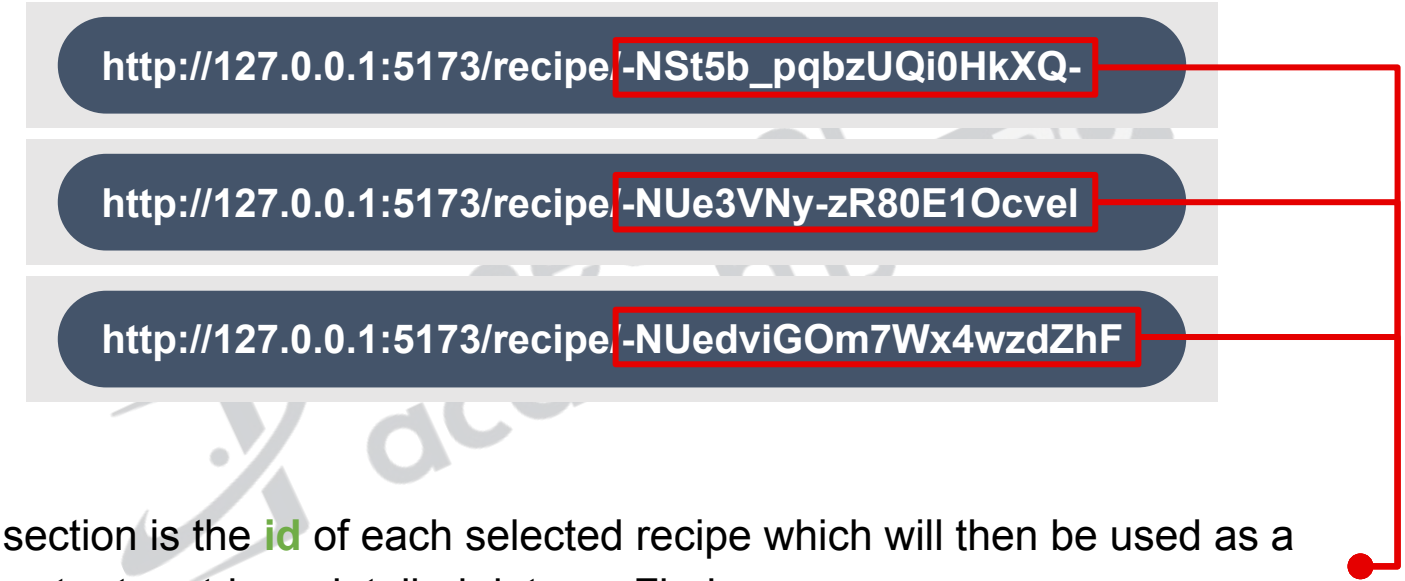
export const routes = [
      ....
  { path: "/recipe/:id", name: "detailPage", component: DetailPage }
];
```

The detail page uses a **dynamic route** and the **parameters** used are the **id** of each selected recipe.

Now try to **choose one of the recipes**. Make sure the details page opens with a view like the following.



The URL of the details for each recipe will vary. An example of the **URL** is as follows.



This section is the **id** of each selected recipe which will then be used as a parameter to retrieve detailed data on Firebase.

Add new **states** and **mutations** that are used to store recipe detail data as followed.

src > store > recipe.js

```
state() {
    return {
        ...
        recipeDetail: {}
    }
},
mutations: {
    ...
    setRecipeDetail(state, payload) {
        state.recipeDetail = payload
    }
},
```

Add a new **action** called **getRecipeDetail** as follows. This **action** will be used to **retrieve** recipe data that matches the **id** of the **selected recipe**.

src > store > recipe.js

```
async getRecipeDetail({commit}, payload) {
    try {

    } catch (err) {

    }
}
```

Then add this code to the **try block.**

src > store > recipe.js

```
async getRecipeDetail({commit}, payload) {
    try {
        const { data } = await axios.get(
            `https://recipe-vue-batch2-default-rtdb.fir
        ebaseio.com/recipes/${payload}.json`)

        commit("setRecipeDetail", data)
    } catch (err) {

    }
}
```

The payload parameter is filled with the **id** of the recipe whose details will be displayed. This process will only return **1** matching **recipe data.**

Add the following code to the **RecipeDetail.vue component.**

src > components > pages > RecipeDetail.vue

```
<script setup>
import { onMounted } from "vue"
import { useRoute } from "vue-router"
import { useStore } from "vuex"
import RecipeDescription from "./RecipeDescription.vue"

const store = useStore()
const route = useRoute()

onMounted(async () => {
    await store.dispatch(
        "recipe/getRecipeDetail", route.params.id
    )
})
</script>
```

src > components > pages > RecipeDetail.vue

```
<script setup>

...

onMounted(async () => {
    await store.dispatch(
        "recipe/getRecipeDetail", route.params.id
    )
})
</script>
```

The **`route.params.id`** code is used to grab the **recipe ID** in the **URL**, then sent when the **getRecipeDetail** action is executed.

Add the following code to the **RecipeDescription.vue component**. The following code is used to fetch recipe detail data from the state.

src > components > pages > RecipeDescription.vue

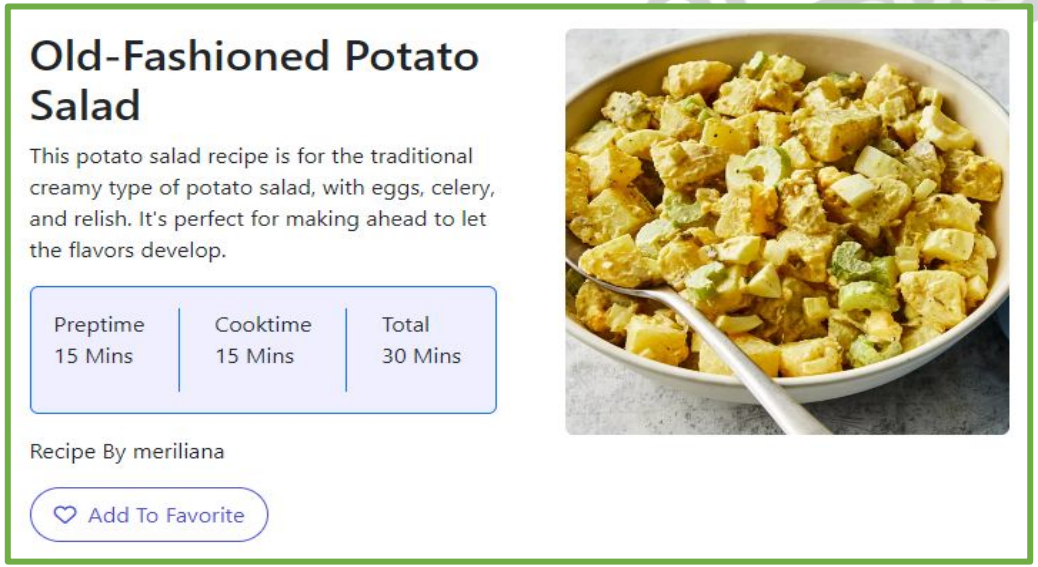
```
<script setup>
import { computed, reactive } from "vue";
import { useStore } from "vuex";

const store = useStore()

const recipeDetail = computed(() => {
  return store.state.recipe.recipeDetail
})
</script>
```

Challenge

Try using the expression and call the computed that was created before to display recipe details in the template so that it looks like the following.

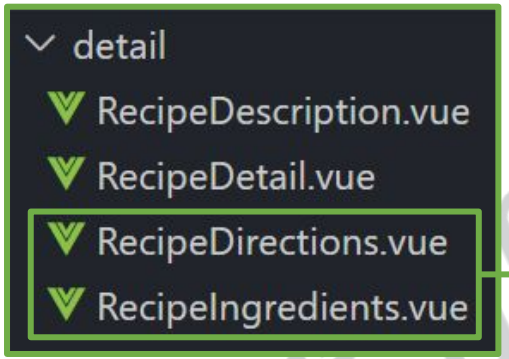


Practice Displaying Recipe Detail in the Detail Page 2

30 Minutes

Timeout academy PRO

Now we continue to complete the **details page** that we have successfully created before.



Add **two new files** to the **detail folder** like the image to the side..

medoor academy PRO

Add the following code to the **RecipeIngredient.vue component.**

src > components > detail > RecipeIngredients.vue

```
<template>
  <div class="my-3">
    <div class="card">
      <div class="card-header">
        <p class="fs-4 my-0 fw-semibold">Ingredients</p>
      </div>
      <div class="card-body">
        <ul class="mb-0">
          <li></li>
        </ul>
      </div>
    </div>
  </div>
</template>
```

Add the following code to the **RecipeDirections.vue component.**

src > components > detail > RecipeDirections.vue

```
<template>
  <div class="my-3">
    <div class="card">
      <div class="card-header">
        <p class="fs-4 my-0 fw-semibold">Directions</p>
      </div>
      <div class="card-body">
        <div>
          <p class="my-2 fs-5 fw-semibold">Step ...</p>
          <p class="my-2"></p>
        </div>
      </div>
    </div>
  </div>
</template>
```

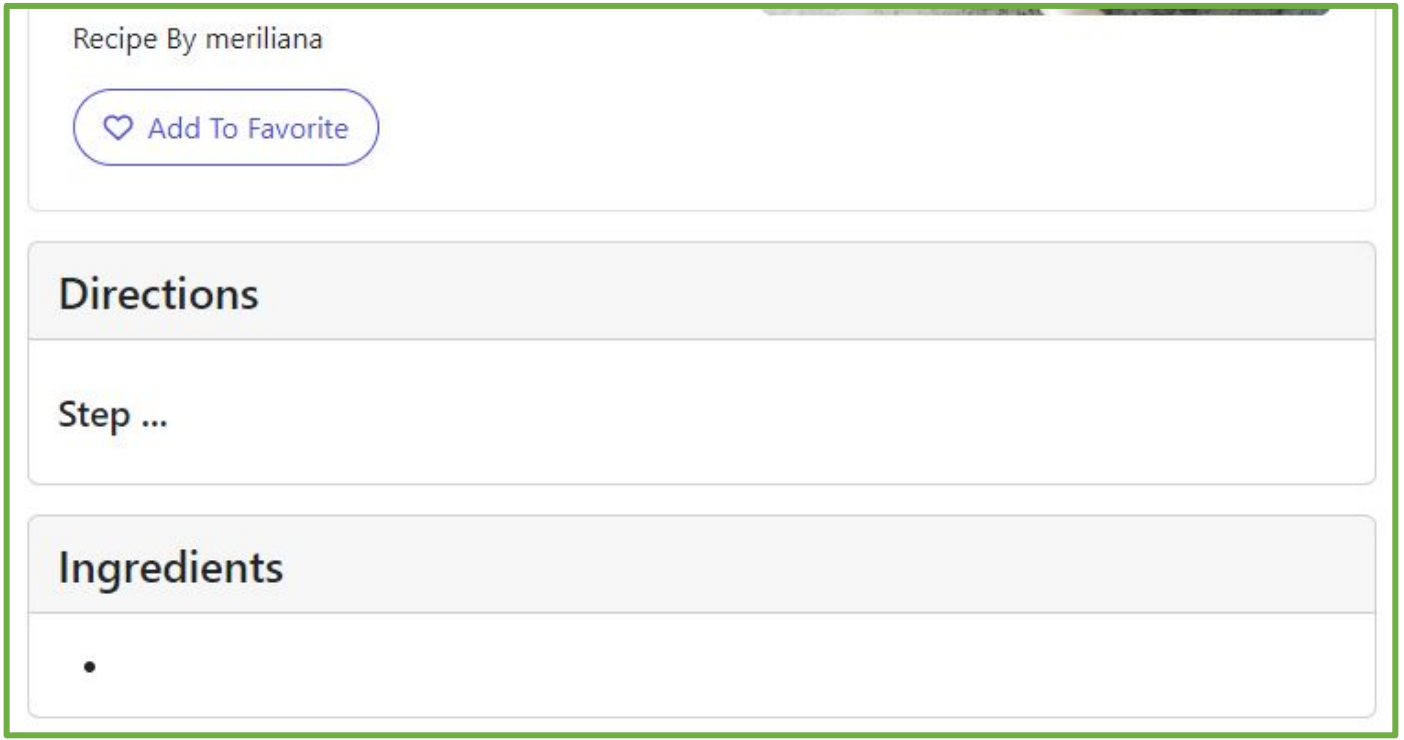
Call the **RecipeDirections.vue** and **RecipeIngredients.vue component** in the **RecipeDetail.vue component.**

src > components > detail > RecipeDetail.vue

```
<template>
    <div class="col-12 col-sm-8">
        <recipe-description></recipe-description>
         <recipe-ingredients></recipe-ingredients>
        <recipe-directions></recipe-directions>
    </div>
</template>

<script setup>
...
import RecipeDirections from "./RecipeDirections.vue"
import RecipeIngredients from "./RecipeIngredients.vue"
...
</script>
```

Make sure the **detail page** looks like this.



Add the following code to the **RecipeIngredients.vue** and **RecipeDirections.vue** components.

RecipeIngredients.vue & RecipeDirections.vue

```
<script setup>
import { computed } from "vue"
import { useStore } from "vuex"

const store = useStore()

const recipeDetail = computed(() => {
  return store.state.recipe.recipeDetail
})
</script>
```

Change the <li> tag in the **RecipeIngredients.vue** component to look like this.

src > components > detail > RecipeIngredients.vue

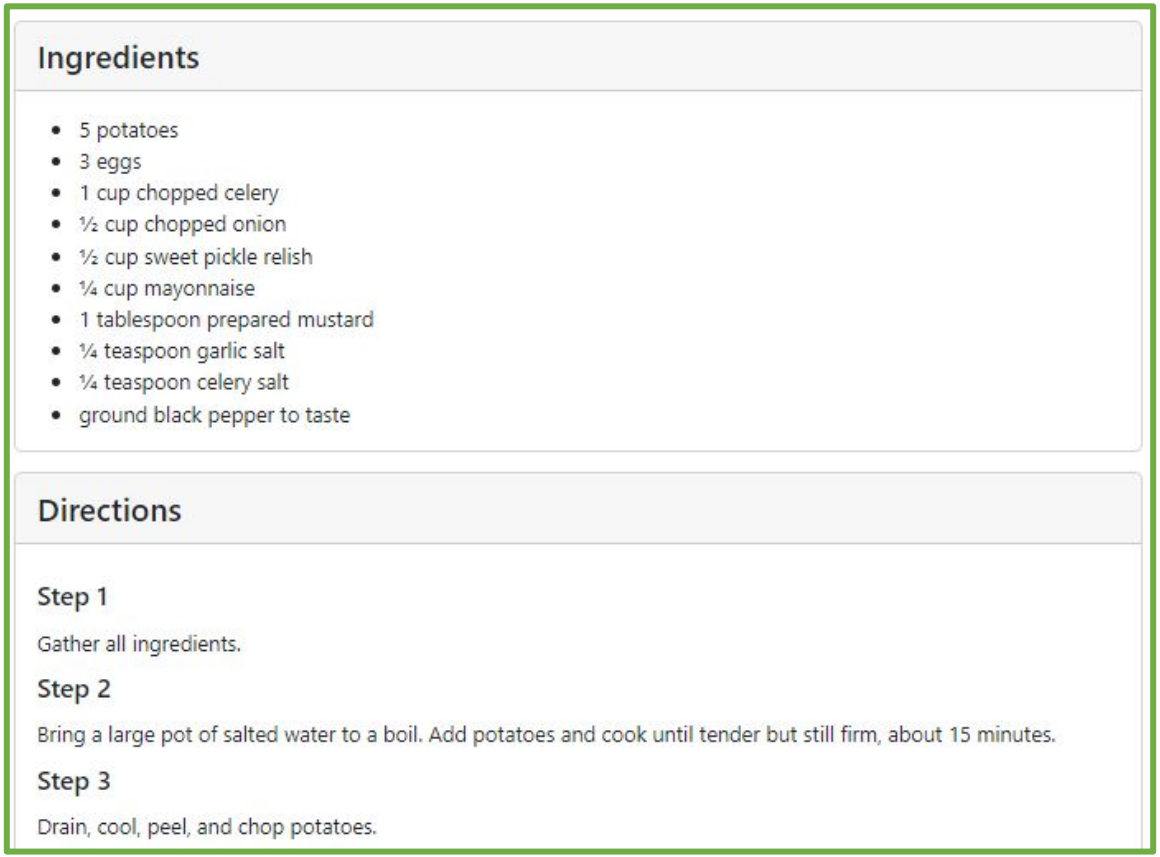
```
<li v-for="(ingredient, index) in recipeDetail.ingredients"
    :key="index">
    {{ ingredient }}
</li>
```

Change the **div tag** inside of the **card-body class** in the **RecipeDirections.vue component** to be like this.

src > components > detail > RecipeIngredients.vue

```
<div class="card-body">
     <div v-for="(direction, index) in recipeDetail.directions"
     :key="index">
          <p class="my-2 fs-5 fw-semibold">
               Step {{ index + 1 }}
          </p>
           <p class="my-2">{{ direction }}</p>
         </div>
</div>
```

Make sure the end result looks like this.



Ingredients

- 5 potatoes
- 3 eggs
- 1 cup chopped celery
- ½ cup chopped onion
- ½ cup sweet pickle relish
- ¼ cup mayonnaise
- 1 tablespoon prepared mustard
- ¼ teaspoon garlic salt
- ¼ teaspoon celery salt
- ground black pepper to taste

Directions

Step 1

Gather all ingredients.

Step 2

Bring a large pot of salted water to a boil. Add potatoes and cook until tender but still firm, about 15 minutes.

Step 3

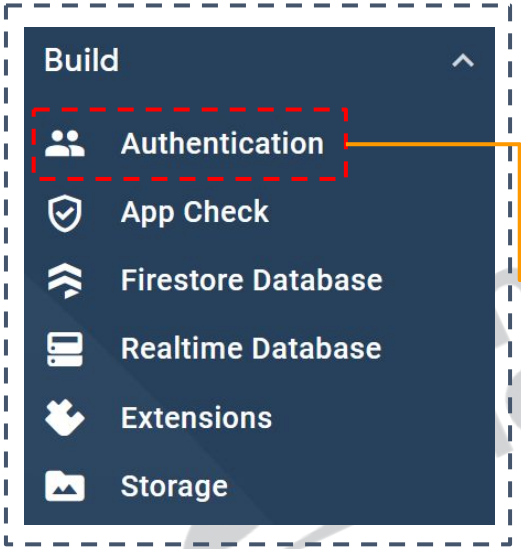
Drain, cool, peel, and chop potatoes.

Preparation

5 Minutes

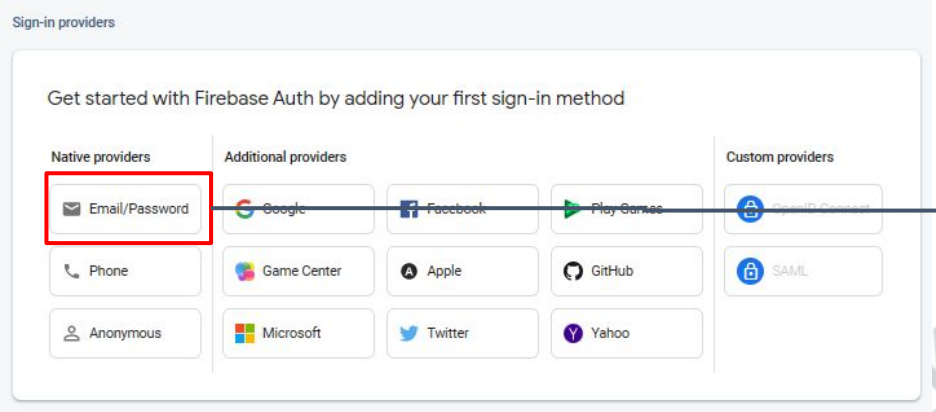
Timedoor academy PRO

First, we need to **configure** the **Firebase** that we use.

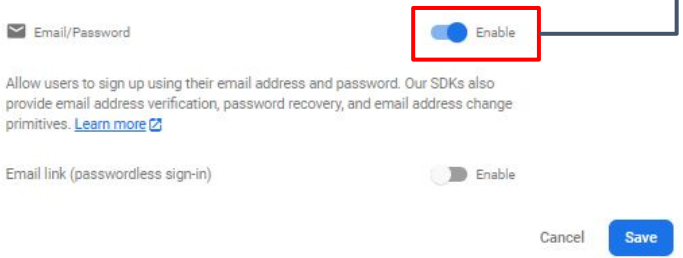


In the **Build menu**, select **Authentication** so that the **project** we are creating can use the **authentication** feature. Next select **Get Started**.

Codedoor Academy PRO

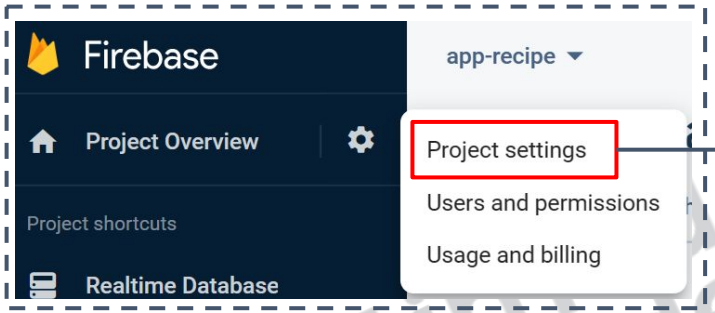


In the **Sign-in provider** select **Email/Password.**

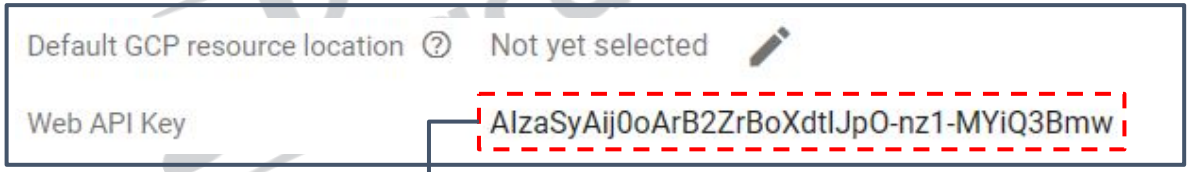


Then enable this part.

In order to be able to **Login** and **Signup** we need an **API Key** from the **firebase project** used.



Select **Project Settings**



Web API Key will be used during the registration/login process

The **Sign Up** and **Login** process on **Firebase** uses an **HTTP Request** with the **POST Method.**

Sign Up Endpoint

```
https://identitytoolkit.googleapis.com/v1/accounts:signUp?key=[API_KEY]
```

Login Endpoint

```
https://identitytoolkit.googleapis.com/v1/accounts:signInWithPassword?key=[API_KEY]
```

These **two endpoints** will later be used in the **HTTP Request** process during **Signup** and **Login.** To make it clearer, you can also see the documentation via the following link: **https://bit.ly/FirebaseAuthRESTAPI**

timedoor academy PRO

Web Cookies

5 Minutes

Timedoor academy PRO

Cookies

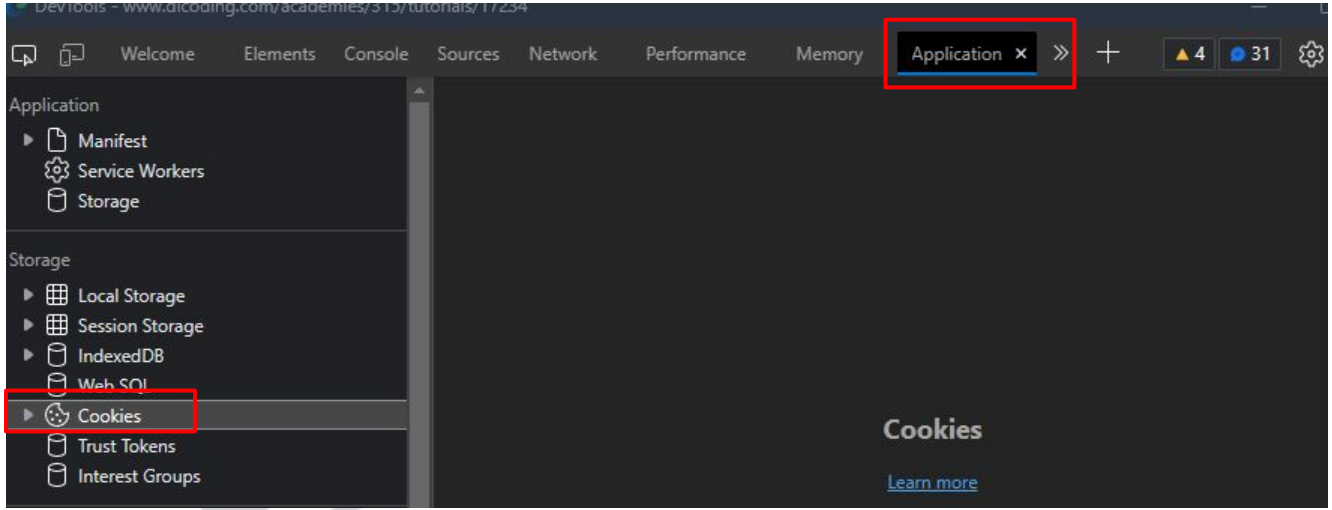
Cookies are **data stored** inside of a **small text file** in a **computer** that's **received** from a **website** and sent back to the **visited site.**

There are several **functions** from **cookies** which are :

- To store login information
- To save the settings of visited websites
- To display advertisements according to visitor browsing activity.

timedoor academy PRO

Same as Web storage, it **can be seen using DevTools** on the application tab **(Google Chrome)** or the storage tab **(Mozilla Firefox)**



To be able to use **cookie,** we need to **install** it by using the **`npm install js-cookie --save` command.**

```
// Import Module
import Cookies from 'js-cookie'

// Menyimpan Cookies
Cookies.set('name', 'value')

// Mengakses Cookies
Cookies.get('name')

// Menghapus Cookies
Cookies.remove('name')
```

Several **methods** can be used to **store, access, and delete cookies.**

ed eor adem y PRO

Practice Adding the Register Feature Part 1

30 Minutes

Mentor Academy PRO

Now we will add **register** and **login** features to the recipe web page that has been created.

Run the following code on the terminal.

```
npm install js-cookie
```



Add a **new file** called **auth.js in the store folder**. The **auth.js file** will be used to handle the authentication process.

academy PRO

Add the following code to the **index.js file** contained in the **store folder.**

src > store > index.js

```
import { createStore } from "vuex";
import recipe from "./recipe";
import auth from "./auth";

export const store = createStore({
    modules: {
        recipe,
        auth
    },
})
```

Add the following code to the **auth.js file.**

src > store > auth.js

```
import axios from "axios";
import Cookies from "js-cookie";
export default {
    namespaced: true,
    state() {
        return {

        }
    },
    mutations: {},
    actions: {}
}
```

So that the data entered in the form can be saved, add the following code to the **BaseInput.vue component.**

src > components > ui > BaseInput.vue

```
<input
      :class="[{ 'd-none': isImage}, 'form-control']"
      :type="type"
      :id="identity"
      :placeholder="placeholder"
      :readonly="readonly === '1'"
      :value="modelValue"
      @input="$emit('update:modelValue', $event.target.value)"
/>

<script setup>
defineProps({
...
  modelValue: { type: [String, Number]}
});
</script>
```

Sekarang tambahkan **object** bernama **signupData** dengan properti seperti berikut. **Object** ini akan digunakan untuk menyimpan data yang dimasukan pada **form**.

src > components > auth > WebSignup.vue

```
<script setup>
...
import { reactive } from "vue";

const signupData = reactive({
  firstname: "",
  lastname: "",
  username: "",
  email: "",
  password: "",
  confirmationPassword: "",
  isLogin: false,
  imageLink: "",
});
</script>
```

Add the following code in the **input component** section to enter the **profile photo.**

src > components > auth > WebSignup.vue

```
<base-input
      type="file"
      identity="recipeImage"
      label="Profile Photo"
      isImage="true"
      @input="checkImage">
            ...
</base-input>
```

The password must consist of a **minimum** of 8 letters. Now we will add a warning if the number of letters in the **password** is less than 8.

Add a **keyup event** to the **BaseInput.vue component**. This event will be executed every time a key on the keyboard is pressed.

src > components > ui > BaseInput.vue

```
<input
      :class="[{ 'd-none': isImage}, 'form-control']"
      :type="type"
      :id="identity"
      :placeholder="placeholder"
      :readonly="readonly === '1'"
      :value="modelValue"
      @input="$emit('update:modelValue', $event.target.value)"
      @keyup="$emit('keyInput', $event.target.value)"/>
```

Then add the following code in the input component for the password.

src > components > auth > WebSignup.vue

```
<div class="my-4">
     <base-input type="password" identiy="password"
          placeholder="Your password" label="Password"
          v-model="signupData.password" @keyInput="passwordCheck">
     </base-input>
     <p class="text-danger mt-1 fw-medium" style="font-size: 11px">
          The password field must be at least 8 characters
     </p>
</div>
```

Add the following code which will be used to check the character length of the **password**.

src > components > auth > WebSignup.vue

```
<script setup>
import { reactive, ref } from "vue";

const passwordStatusDisplay = ref("none")

const passwordCheck = () => {
  if ( signupData.password.length < 8 ) {
    passwordStatusDisplay.value = "block"
  } else {
    passwordStatusDisplay.value = "none"
  }
}
</script>
```

Then in the following **<p> tag**, add a **style attribute** like this. The **passwordStatusDisplay** property has a value of **none** if the password is more than 8 characters and a block value if the password is less than 8.

src > components > auth > WebSignup.vue

```
<div class="my-4">
     <base-input ...></base-input>
     <p class="text-danger mt-1 fw-medium"
          style="font-size: 11px"
          :style="{ display: passwordStatusDisplay }">
          The password field must be at least 8 characters
     </p>
</div>
```

The display when the password is less than 8 characters



The display when the password is more or equal to 8 characters



Now we will add a warning if the **password** and **confirmationPassword** do not match. Add the following code.

src > components > auth > WebSignup.vue

```
<div class="my-4">
    <base-input ... v-model="signupData.confirmationPassword"
        @keyInput="confirmationPasswordCheck"></base-input>
    <p class="text-danger mt-1 fw-medium" style="font-size: 11px">
        The password confirmation does not match
    </p>
    <p class="text-success mt-1 fw-medium" style="font-size: 11px">
        The password confirmation does match
    </p>
</div>
```

Then add the following code which is used to check whether the **password** is the same as **confirmationPassword.**

src > components > auth > WebSignup.vue

```
<script setup>
const confirmPasswordDoesNotMacth = ref("none");
const confirmPasswordMacth = ref("none");

const confirmationPasswordCheck = () => {
  if (signupData.confirmationPassword === "") {
    confirmPasswordDoesNotMacth.value = "none";
    confirmPasswordMacth.value = "none";
    return;
  }

  if (signupData.password !== signupData.confirmationPassword) {
    confirmPasswordDoesNotMacth.value = "block";
    confirmPasswordMacth.value = "none";
    return;
  }

  confirmPasswordDoesNotMacth.value = "none";
  confirmPasswordMacth.value = "block";
};
</script>
```

Then in the following **<p> tag**, add a **style attribute** like this.

src > components > auth > WebSignup.vue

```
<div class="my-4">
    <base-input ...></base-input>
    <p class="text-danger mt-1 fw-medium" style="font-size: 11px"
        :style="{ display: confirmPasswordDoesNotMacth }">
        The password confirmation does not match
    </p>
    <p class="text-success mt-1 fw-medium" style="font-size: 11px"
        :style="{ display: confirmPasswordMacth }">
        The password confirmation does match
    </p>
</div>
```

The display when password and confirmation password match



The display when password and confirmation password do not match



Now we will activate the **Profile Photo** feature. First, add the following code.

src > components > auth > WebSignup.vue

```
<div class="my-4">
      <base-input type="file" identity="recipeImage" label="Profile Photo"
            isImage="true" @input="checkImage">
            <div class="border p-1 mt-2 rounded-circle">
                  <img :src="signupData.imageLink" class="rounded-circle"
                        width="140" height="150" style="object-fit: cover"/>
            </div>
            ...
      </base-input>
</div>
```

Then add the following code. This code will be used to store the selected image file in the **imageLink** property.

src > components > auth > WebSignup.vue

```
const checkImage = (e) => {
  const file = e.target.files[0];
  const reader = new FileReader();
  reader.readAsDataURL(file);

  reader.addEventListener("load", () => {
    signupData.imageLink = reader.result;
  });
};
```

Now try selecting an image, and make sure the page looks like the following.



Practice Adding the Register Feature Part 2

30 Minutes

Mimedor academy PRO

Previously we have completed half of the **register features**. Now we will add code so that the entered register data can be stored in the **firebase.**

src > components > auth > WebSignup.vue

```
<form class="mt-3" @submit.prevent="register">
    ...
</form>
```

In the form tag, add the following code. **@submit.prevent** prevents the web page from reloading when the register button is clicked.

Now we will set up a **store** to **manage** register data. In the **auth.js file**, add the following **state.**

src > store > auth.js

```
state() {
    return {
        token: null,
        tokenExpirationDate: null,
        userLogin: {},
        isLogin: false,
    };
}
```

token : to store the tokens obtained when logging in/registering
tokenExpirationDate : to save the expiration time of the token
userLogin : data from users who are logged in/registered
isLogin : login status true/false

When **registering or logging in**, the two processes will generate a **token**. The token will determine the success of the user to **login/register**. Add the following **mutations** which will be used to store **tokens** in **state** and **cookies**.

src > store > auth.js

```
mutations: {
    setToken(state, { idToken, expiresIn }) {
      state.token = idToken;
      state.tokenExpirationDate = expiresIn;
      Cookies.set("tokenExpirationDate", expiresIn);
      Cookies.set("jwt", idToken);
    },
}
```

Apart from tokens, we also need to store **data** from **users** who have successfully **logged in/registered.** Add the following **mutations.**

src > store > auth.js

```
  mutations: {
    setUserLogin(state, { userData, loginStatus })
{
      state.userLogin = userData;
      state.isLogin = loginStatus;
    },
  },
```

Now, we'll add an **action** that is used to handle the **register** process. Add an **action** named **getRegisterData** as follows.

src > store > auth.js

```
actions: {
   async getRegisterData({ commit, dispatch }, payload) {
      const APIkey = "AIzaSyAZw4XY-USmeiN9j1cLmxJ8W8mu_l2a19U";
      const authUrl =
      "https://identitytoolkit.googleapis.com/v1/accounts:signUp?key=";
      try {
            const { data } = await axios.post(authUrl + APIkey, {
                  email: payload.email,
                  password: payload.password,
                  returnSecureToken: true,
            });
      } catch (err) {
            console.log(err);
      }
   },
}
```

Ganti dengan **Web API Key** yang diperoleh pada saat persiapan.

Add the following code to the **try block** after the **POST** process. These codes are used to store **tokens** in **state** and **cookies** and call the **addNewUser action** to save new user data to **firebase.**

src > store > auth.js

```
commit("setToken", {
   idToken: data.idToken,
   expiresIn: new Date().getTime() + Number.parseInt(data.expiresIn) * 1000
})

const newUserData = {
     userId: data.localId, firstname: payload.firstname,
     lastname: payload.lastname, username: payload.username,
     email: payload.email, imageLink: payload.imageLink,
};

Cookies.set("UID", newUserData.userId);
await dispatch("addNewUser", newUserData);
```

Add a new **action** called **addNewUser** which is used to add new user data to **firebase**.

src > store > auth.js

```
async addNewUser({ commit, state }, payload) {
  try {
    const { data } = await axios.post(
`https://recipe-vue-batch2-default-rtdb.firebaseio.com/user.json?auth=${
state.token}`, payload);
    commit("setUserLogin", {userData: payload, loginStatus: true})
  } catch (err) {
    console.log(err);
  }
},
```

Now we will create a **function** that will be called when the **register** button is pressed. Add the following code to the **WebSignup.vue component.**

src > components > auth > WebSignup.vue

```
import { useStore } from "vuex";
import { useRouter } from "vue-router";

const store = useStore()
const router = useRouter()

const register = async () => {

};
```

Add the following code to the **function register**. The *conditional* code is used to abort the r**egistration process** if the **password** and **confirmation password** do not match.

src > components > auth > WebSignup.vue

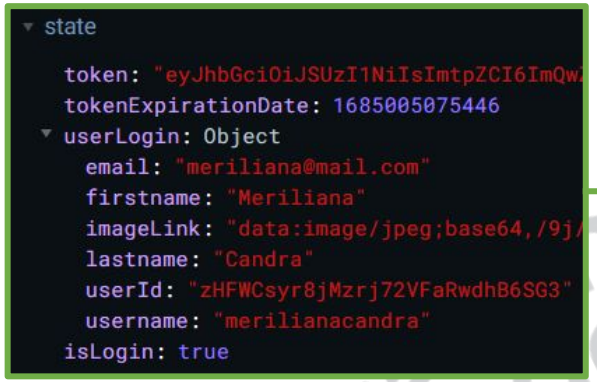
```
if (
  signupData.password !== signupData.confirmationPassword ||
  signupData.password.length < 8
) {
  signupData.confirmationPassword = "";
  signupData.password = "";
  confirmPasswordDoesNotMacth. value = "none";
  confirmPasswordMacth. value = "none";
}
```

Now add an else block. The code in the else block will call the **getRegisterData action** and then run the **router.push("/")** code which is used to open the **home page** when the **action** process is complete.

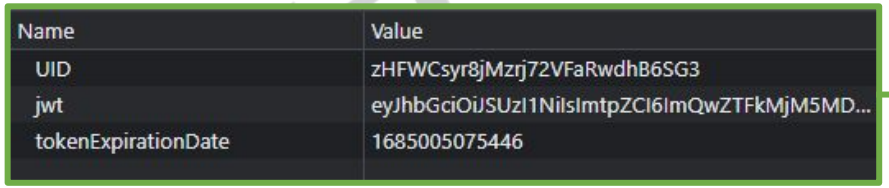
src > components > auth > WebSignup.vue

```
if (
  ...
} else {
  await store.dispatch("auth/getRegisterData", signupData);
  router.push("/");
}
```

Sekarang cobalah untuk melakukan **register**. Apabila proses sudah selesai cobalah cek beberapa bagian berikut ini untuk memastikan bahwa proses **register** sudah berhasil dilakukan.

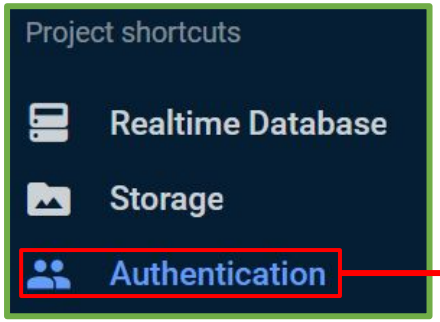


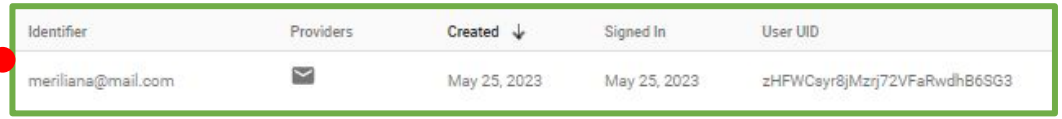
Cek pada **Vue.js Devtools**, lalu state **token**, **tokenExpirationDate**, **userLogin**, dan **isLogin** sudah terisi seperti contoh berikut.



Name	Value
UID	zHFWCsyr8jMzrj72VFaRwdhB6SG3
jwt	eyJhbGciOiJSUzI1NiIsImtpZCI6ImQwZTFkMjM5MD...
tokenExpirationDate	1685005075446

Data **token**, **tokenExpirationDate**, dan **user id** sudah tersimpan pada **cookies**.





The newly registered account already appears in **Firebase Authentication.**



The new user data is already stored in **Firebase**.

Apabila seluruh bagian sudah sesuai, maka proses *register* sudah berhasil dilakukan.

medoor academy PRO

Dynamic Component

5 Minutes

Timedoor academy PRO

DYNAMIC COMPONENT

Dynamic components are used to change components based on certain conditions. Dynamic components can be created using a **<component></component>** element with an **is attribute.**

```
<component :is="componentName"></component>
```

Can be a **variable** that holds the **component** name.

timedoor academy PRO

Each ***component*** that will be used as a ***dynamic component*** must be declared as follows.

Component

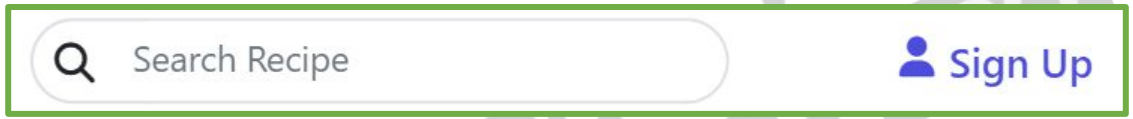
```
<script setup>
const components = {
  'first-component': FirstComponent,
  'second-component': SecondComponent
}
</script>
```

Practice Setting the Profile Menu on the Navbar

20 Minutes

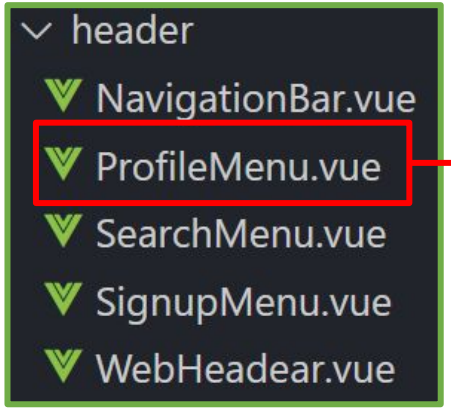
Timedoor academy PRO

Now we will change the appearance of the ***Navbar***. When the ***user*** has not ***registered/logged in,*** the display of the ***Navbar*** will be like the following.



However, if the ***user*** has successfully ***registered/login***, the display of the ***Navbar*** will be as follows.





Add a new **header folder** component named **ProfileMenu.vue.**

Copy the code contained in the following link then place it in the **ProfileMenu.vue file.**

https://bit.ly/3BVcxNx

timedoor academy PRO

Add the following code to the **NavigationBar.vue component.**

src > components > header > NavigationBar.vue

```
<script setup>
...
import ProfileMenu from "./ProfileMenu.vue";
import { computed, ref, watch } from "vue";
import { useStore } from "vuex";

const menuComponent = ref("signup-menu");
const store = useStore();

const components = {
  'signup-menu': SignupMenu,
  'profile-menu': ProfileMenu
}
</script>
```

The components object is used to store components that will alternate, namely the **SignupMenu.vue** and **ProfileMenu.vue components.**

Still in the **script tag**, add the following code.

src > components > header > NavigationBar.vue

```
const getToken = computed(() => {
  return store.state.auth.token;
});
```

The getToken computed function retrieves the token stored in state. The token will be a marker whether the user has **logged in/registered** or not.

Still in the **script tag,** add the following code.

src > components > header > NavigationBar.vue

```
if (!getToken.value) {
  menuComponent.value = "signup-menu";
} else {
  menuComponent.value = "profile-menu";
}
```

This code is used to check the value returned by the **getToken computed**. If **no token** is found then the **SignupMenu.vue component** will be displayed, if a **token is found** then the **ProfileMenu.vue component** will be displayed.

Still in the **script tag**, add the following code.

src > components > header > NavigationBar.vue

```
watch(getToken, (newValue, oldValue) => {
  if (!newValue) {
    menuComponent.value = "signup-menu";
  } else {
    menuComponent.value = "profile-menu";
  }
});
```

Watch functions to notice changes to **getToken**. It is used to change the components that should be displayed on the **Navbar**.

In the template tag, erase this code.

src > components > header > NavigationBar.vue

```
<signup-menu></signup-menu>
```

Then change the code to the one below.

src > components > header > NavigationBar.vue

```
<component :is="components[menuComponent]"></ component>
```

Now try to **register**, make sure the **profile** menu appears after the **registration** process is complete.

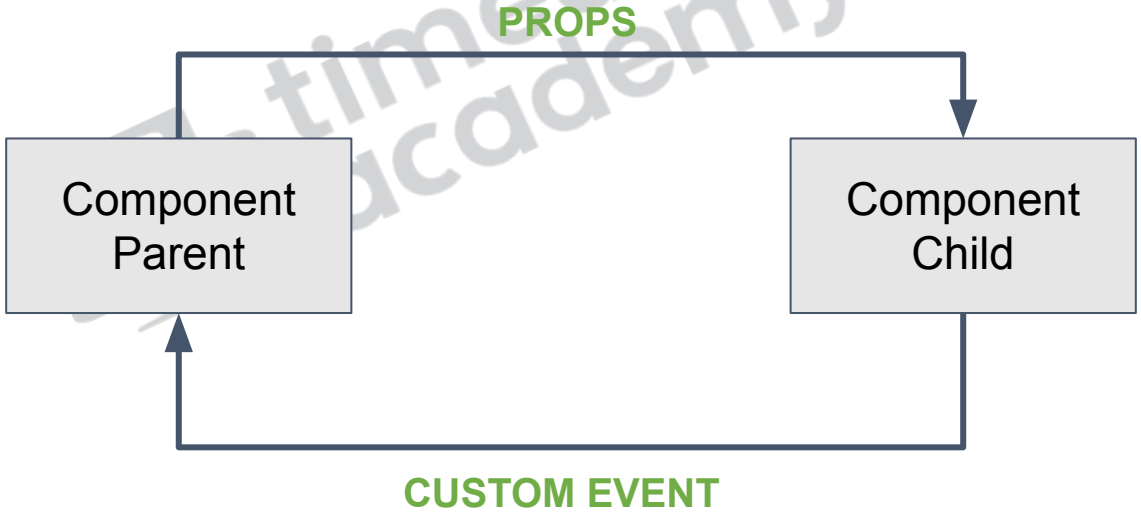
Custom Event

5 Menit

imedoor academy PRO

Previously we have learned to send data from the **parent component** to the **child component** using **props**. Then what if we want to send data from the **child component to the parent**?

Sending data from the **child component to the parent component** can be done using **Custom Event**.



timedoor academy PRO

Component

```
<script setup>
const emit = defineEmits(['customEventName'])

const functionName = param => {
  emit("customEventName", param)
}
</script>
```

Declare every **custom event** made

How to run **custom event.** **Params** can be changed to the data that will be sent to the parent.

To make sure the **parent component** can **receive the custom event**, in the **component** that sends the data, add the following **code**.

```
@customEventName="action"
```

Name of the **custom event** received

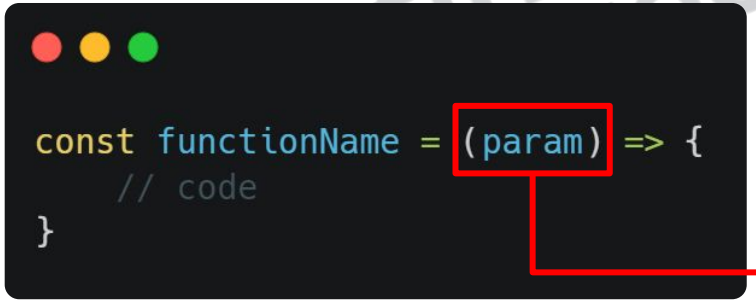
The action to be performed when the **custom event** occurs. For example, like calling a **function**.

timedoor academy PRO

Then how to retrieve the **data** sent from the **child component**?

- If the **action** is performed by calling a **function**, the **data** will be stored in the **parameters** of the **function**.

```
@customEventName="functionName"
```



The sent data will be stored in the **parameter** that will be run when the **custom event** occurs.

- Other than using **method**, another way to do it is through **$event**.

```
@customEventName="propertyName = $event"
```

property that is used to store **data** received.

$event carries the data sent from tne **child component.**

timedoor academy PRO

Practice Adding the Login Feature

2 Minutes

Timedoor Academy PRO

Now we will activate the ***login*** feature, so that ***users*** can ***log in*** with an account that has been registered.

Add an ***object*** named **loginData**. ***This object*** will be used to save the ***login*** data inserted by the ***user***.

src > components > auth > WebLogin.vue

```
<script setup>
...
import { reactive } from "vue";

const loginData = reactive({
  email: "",
  password: "",
  isLogin: true
})
</script>
```

Add a **v-model** like the following step so that every data implemented can be saved in the ***object*** that has been made previously.

src > components > auth > WebLogin.vue

```
<div class="my-4">
     <base-input
          type="usernmae" identiy="username"
             placeholder="Your username" label="Email"
             v-model="loginData.email"></base-input>
</div>
<div class="my-4">
     <base-input
          type="password" identiy="password"
             placeholder="Your password" label="Password"
             v-model="loginData.password"></base-input>
</div>
```

Now, we'll add an **action** that is used to handle the **login** process. Add an **action** named **getLoginData** like the following.

src > store > auth.js

```
 async getLoginData({ commit, dispatch }, payload) {
   const APIkey = 'AIzaSyAZw4XY-USmeiN9j1cLmxJ8W8mu_l2a19U';
   const authUrl =
"https://identitytoolkit.googleapis.com/v1/accounts:signInWithPassword?key="
   try {
     const { data } = await axios.post(authUrl + APIkey, {
      email: payload.email, password: payload.password, returnSecureToken: true,
     });
     commit("setToken", {
       idToken: data.idToken,
       expiresIn: new Date().getTime() + Number.parseInt(data.expiresIn) * 1000
     });
     await dispatch("getUser", data.localId)
   } catch (err) {
       console.log(err)
   }
},
```

Ganti dengan **Web API Key** yang diperoleh pada saat persiapan.

Add a **new action** named **getUser** which functions to retrieve ***user*** data based on the ***user*** who is currently logged in.

src > store > auth.js

```
async getUser({ commit }, payload) {
  try {
    const { data } = await axios.get(
`https://recipe-vue-batch2-default-rtdb.firebaseio.com/user.json`
    for (let key in data) {
      if (data[key].userId === payload) {
         Cookies.set("UID", data[key].userId)
         commit("setUserLogin",
              { userData: data[key], loginStatus: true }
          )
       }
     }
  } catch (err) {
       console.log(err)
  }
}
```

Now we will create a **function** that will be executed when the **login** button is pressed. Add the following **code** to the **WebLogin.vue** component.

src > components > auth > WebLogin.vue

```
...
import { useStore } from "vuex";
import { useRouter } from "vue-router";

const store = useStore()
const router = useRouter()

const login = async () => {
  await store.dispatch("auth/getLoginData", loginData);
  router.push("/")
}
```

In the form tag, asd the following code.

src > components > auth > WebLogin.vue

```
<form @submit.prevent="login">
    ...
</form>
```

Now try to ***login*** with the account that has been registered before.

Practice Adding the Logout Feature

20 Minutes

Timedoor academy PRO

In the previous exercise, we added a **login** feature to the **web recipe page**. Now we will add a **logout** feature to the **web recipe page**.

Add a new **mutation** named **setUserLogout**. This mutation will delete data related to authentication in **state** and **cookies**.

src > components > auth > WebLogin.vue

```
setUserLogout(state) {
    state.token = null
    state.userLogin = {}
    state.isLogin = false
    state.tokenExpirationDate = null
    Cookies.remove("jwt");
    Cookies.remove("tokenExpirationDate");
    Cookies.remove("UID");
}
```

In the **ProfileMenu.vue** component, add the **click event** in the **logout list** like below.

src > components > header > ProfileMenu.vue

```
<ul class="dropdown-menu">
    <li class="dropdown-item">My Profile</li>
    <li class="dropdown-item">Favorited Recipes</li>
    <li class="dropdown-item">My Recipes</li>
    <li><hr class="dropdown-divider" /></li>
    <li class="dropdown-item" @click="logout">Logout</li>
</ul>
```

Still in the **ProfileMenu.vue** component, add the following **code** to the **script tag.**

src > components > header > ProfileMenu.vue

```
<script setup>
import { useRouter } from "vue-router";
import { useStore } from "vuex";

  const store = useStore()
  const router = useRouter()

  const logout = () => {
    store.commit("auth/setUserLogout")
    router.push("/")
  }
</script>
```

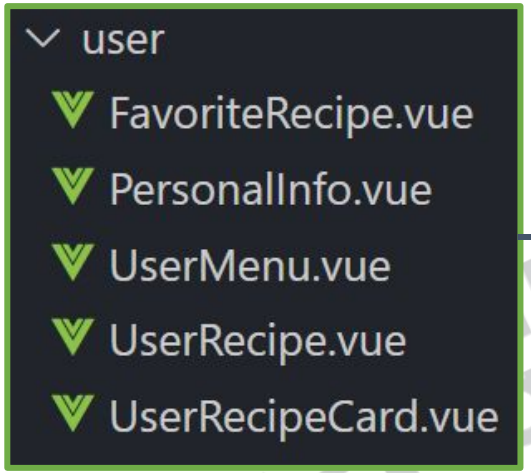
Now try pressing the **logout button**. Make sure the **logout button** is working properly.

Practice Add a User Page Part 1

30 Menit

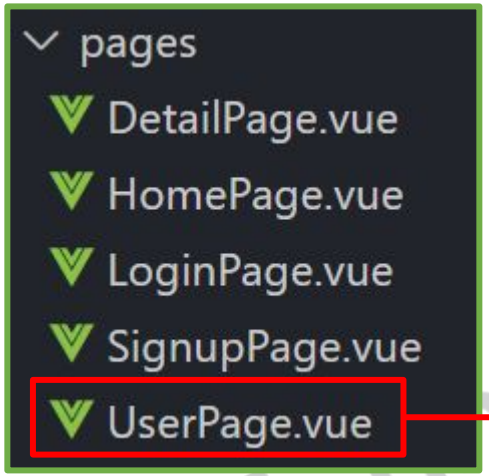
Innovator academy PRO

Now we will add a user page. The user page will contain the user's **personal data**, **recipes created** by the user, and **recipes liked** by the user.



Add new **5 components** baru in the **user folder** with the names like the image to the left.

Copy paste the following code provided by the link: **https://bit.ly/3OCQIKn** then place it in each made component.



Add a new file in the **pages folder** named **UserPage.vue**.

Next, we will call several **components** that is needed for the **user page**.

Add the following **code** to the **UserPage.vue component.**

src > components > pages > UserPage.vue

```
<template>
    <div class="container-md my-5 py-5">
        <div class="row">
            <user-menu></user-menu>
        </div>
    </div>
</template>

<script setup>
import UserMenu from "../user/UserMenu.vue"
</script>
```

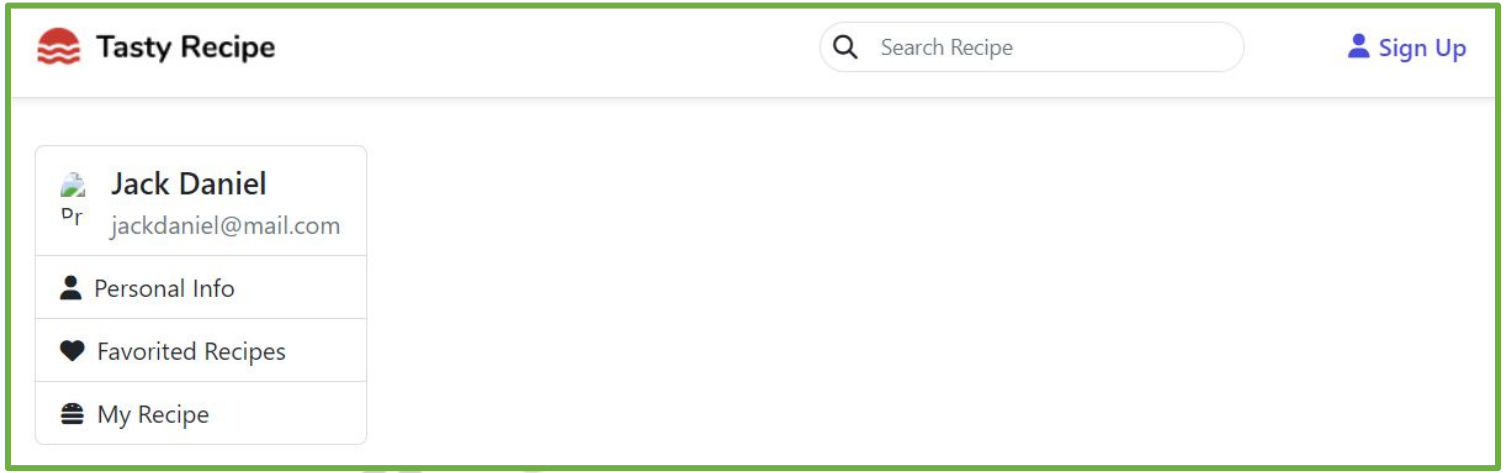
Add a new **route** for the **user page** like the following

src > routes.js

```
...
import UserPage from "./components/pages/UserPage.vue"

export const routes = [
  ...
  { path: "/user", name: "userPage", component: UserPage }
];
```

Open the **user page** with **route /user**, make sure the **web page** looks like the following.



Now add the highlighted codes into the **UserMenu.vue component.**

src > components > user > UserMenu.vue

```
<li class="list-group-item user-menu"
     @click="menuClicked('personal-info')">
     <i class="fa-solid fa-user pe-2"></i>Personal Info
</li>
<li class="list-group-item user-menu"
     @click="menuClicked('favorite-recipes')">
     <i class="fas fa-heart pe-2"></i>Favorited Recipes
</li>
<li class="list-group-item user-menu"
     @click="menuClicked('user-recipe')">
     <i class="fa-solid fa-burger pe-2"></i>My Recipe
</li>
```

Still in the **UserMenu.vue component**, add the following code in the **script tag**.

src > components > user > UserMenu.vue

```
<script setup>
const emit = defineEmits(['changeComponent'])

const menuClicked = option => {
  emit("changeComponent", option)
}
</script>
```

The menuClicked function is called by every menu in the user page that can be clicked on. This function will run the emit command

In the **UserPage.vue component,** add the highlighted codes.

src > components > pages > UserPages.vue

```
<template>
    <div class="container-md my-5 py-5">
        <div class="row">
            <user-menu @changeComponent="$router.push($event)"></user-menu>
                <div class="col-lg-9">
                    <component :is="component[getRoute]"></component>
                </div>
        </div>
    </div>
</template>
```

Then in the **UserPage.vue** component, add the highlighted code,

src > components > pages > UserPages.vue

```
<script setup>
import UserMenu from "../user/UserMenu.vue"
import FavoriteRecipe from "../user/FavoriteRecipe.vue"
import PersonalInfo from "../user/PersonalInfo.vue"
import UserRecipe from "../user/UserRecipe.vue"
</script>
```

The **FavoriteRecipe.vue**, **PersonalInfo.vue**, and **UserRecipe.vue** component will display according to the menu selected in the **user page.**

Still in the **script tag,** add the following code

src > components > pages > UserPages.vue

```
import { useRoute } from "vue-router"
import { computed } from "vue"

const route = useRoute();

const component = {
    "personal-info": PersonalInfo,
    "favorite-recipes": FavoriteRecipe,
    "user-recipe": UserRecipe
}

const getRoute = computed(() => {
    return route.params.component
})
```

Next, change the **path** for the **user** to be like this.

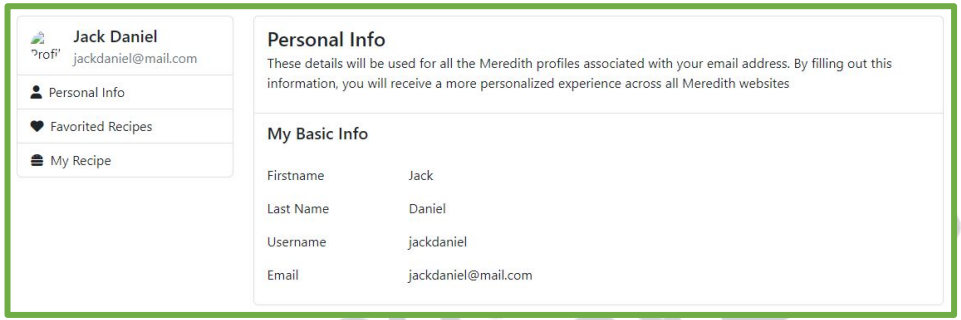
src > routes.js

```
...
export const routes = [
  ...
  { path: "/user/:component", name: "userPage", component: UserPage }
];
```

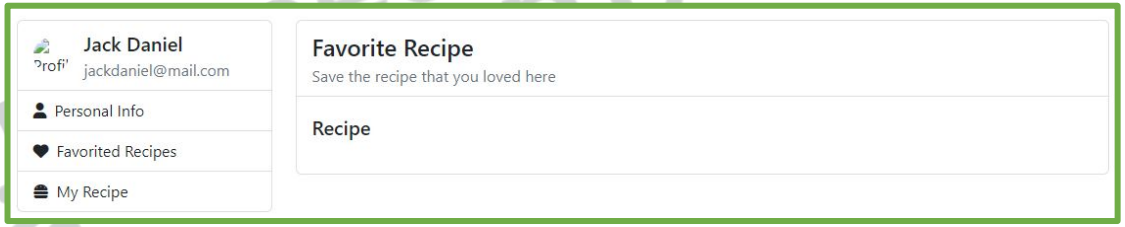
The **User page** will use a **dynamic route**. The **route** will adjust to the menu selected on the **user page.**

The user page will look like this.

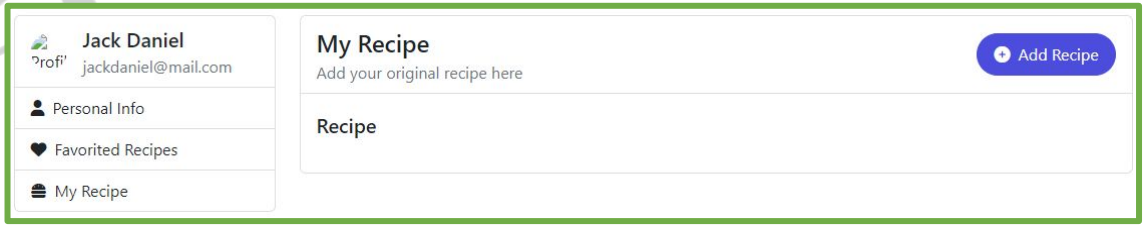
Personal Info



Favorite Recipe



My Recipe

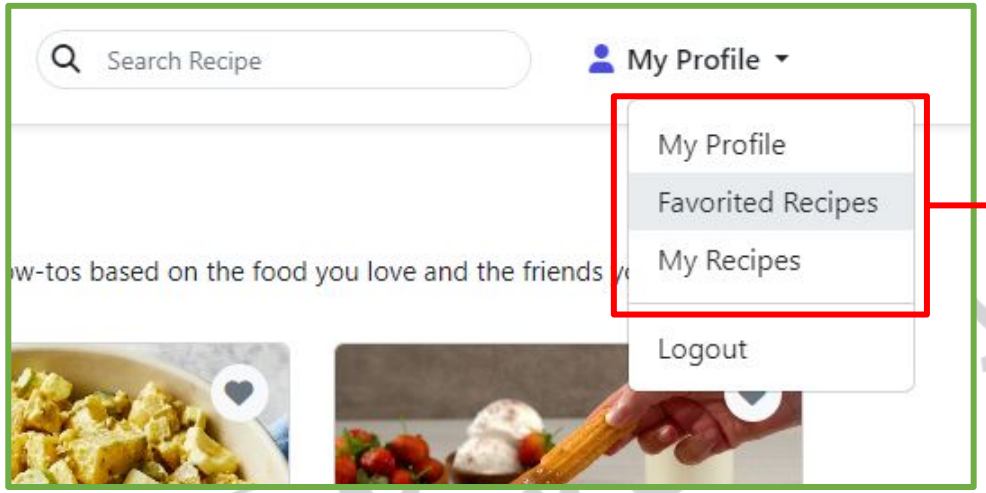


Lastly in the **ProfileMenu.vue component**, change the **li tag** with **router-link** like below.

src > components > header > ProfileMenu.vue

```
<ul class="dropdown-menu">
     <router-link to="/user/personal-info" class="dropdown-item">
          My Profile</router-link>
     <router-link to="/user/favorite-recipes" class="dropdown-item">
          Favorited Recipes</router-link>
     <router-link to="/user/user-recipe" class="dropdown-item">
          My Recipes</router-link>
     ...
</ul>
```

Now try to **login**.



After the **login** process is successful, try to enter the user page via the following menu. Make sure the display that appears on the **user page** is in accordance with the **selected menu.**

Middleware

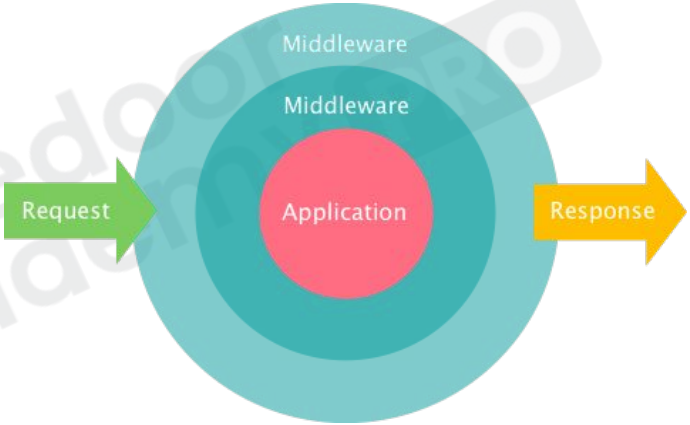
5 Minutes

Ahmedou academy PRO

What is Middleware?

Middleware is a **software** that is in between the **operation system** and **application** that is run within.

One example of using **middleware** is in sharing **user access rights** or providing **protection** for a **page** so that it can only be **accessed** by **registered users**.



`beforeEnter`

`beforeEnter` used to apply a check or condition before accessing a **route. An example would be** if the user has not logged in, then the user cannot enter the page to add a new product.

src > routes.js

```
{
    path: "/router",
    name: "component",
    component: Component,
    beforeEnter: (to, from, next) => {
        ...
    },
}
```

beforeEnter consists of three parameters which are **to**, **from**, **next**.

- **`to` :** destination *route* that is directed by the *user*. This object consists of properties such as `path`, `name`, `params`, and `query`.

- **`from` :** *Route* that is benign left by the *user*. This object consists of properties such as `path`, `name`, `params`, and `query` from the *previous route.*

- **`next` :** Function used to control the navigation flow. This function must be called to allow or prevent navigation to the destination *route*. `next()` to allow to the destination route, `next('/other-route')` to direct to another route, atau `next(false)` to cancel navigation.

Practice Adding User Page Part 2

30 Menit

Innovator academy PRO

Now we will continue to create the **user page** that was created **earlier**.

Add the following code to the **PersonalInfo.vue component.**

src > components > user > PersonalInfo.vue

```
<script setup>
import { computed } from "vue";
import { useStore } from "vuex";

const store = useStore()
const userData = computed(() => {
  return store.state.auth.userLogin
})
</script>
```

This code is used to retrieve user data that is currently **logged in**. **User data** is stored in the **userLogin state.**

In the **template**, fix the **code** in the highlighted part.

src > components > user > PersonalInfo.vue

```
<div class="d-flex mt-4">
     <div class="pe-4 me-4">
          <p>Firstname</p>
          <p>Last Name</p>
          <p>Username</p>
          <p>Email</p>
     </div>
     <div class="ps-4 ms-4">
          <p>{{ userData.firstname }}</p>
          <p>{{ userData.lastname }}</p>
          <p>{{ userData.username }}</p>
          <p>{{ userData.email }}</p>
     </div>
</div>
```

Just like the **PersonalInfo.vue** component, the **UserPage.vue** component also requires data from the **user** who is currently **logged in**. Add the following code.

src > components > pages > UserPage.vue

```
<script setup>
import { computed } from "vue";
import { useStore } from "vuex";

...

const store = useStore()
const userData = computed(() => {
  return store.state.auth.userLogin
})
</script>
```

Change the **code** in the **template** like the following.

src > components > pages > UserPage.vue

```
<div class="d-flex align-items-center">
    <img :src="userData.imageLink" alt="Profile"
        width="36" height="36"
        class="rounded-circle" style="object-fit: cover"
    />
    <div class="ps-3">
        <p class="my-0 fs-5 fw-semibold">
            {{ userData.firstname}} {{ userData.lastname }}
        </p>
        <p class="my-0 fs-6 text-secondary">
            {{ userData.email }}
        </p>
    </div>
</div>
```

Make sure the **user page** look exactly like this.



User data must be in accordance to the **user** that is **logged in**.

Practice Adding Access Rights

30 Minutes

Administrator academy PRO

The access rights of the user are determined from the **login status** of the **user.** Add the following code to the **routes.js** file.

src > routes.js

```
import Cookies from "js-cookie"
import { store } from "./store/index";

const checkAuth = () => {
  const jwtCookie = Cookies.get("jwt");
  const expirationDate = Cookies.get("tokenExpirationDate");
  const userId = Cookies.get("UID");
};
```

The checkAuth function is used to check whether there is an **active token** in **cookies. The token** will determine whether the user is currently **logged in** or not.

Add the following **code** in the **checkAuth function**.

src > routes.js

```
if (jwtCookie) {
     if (new Date().getTime() < +expirationDate) {
          store.commit("auth/setToken", {
               idToken: jwtCookie,
               expiresIn: expirationDate,
          });
          store.dispatch("auth/getUser", userId);
          return true;
     } else {
          store.commit("auth/setUserLogout");
          return false;
     }
} else {
     return false;
}
```

Afterwards, add this **code** to the ***route*** of the **Home Page.**

src > routes.js

```
{
    path: "/",
    name: "homePage",
    component: HomePage,
    beforeEnter: () => {
      checkAuth();
    },
},
```

Before the **home page** is displayed, **the checkAuth function** will be run. This is so that when there is a **reload process** on the **home page**, the **user** will remain in the **login status** if there is an **active token**.

Add the following **code** to the ***route*** of the **User page.**

src > routes.js

```
{
    path: "/user/:component",
    name: "userPage",
    component: UserPage,
    beforeEnter: (to, from, next) => {
      checkAuth() ? next() : next({ name: "login" });
    },
}
```

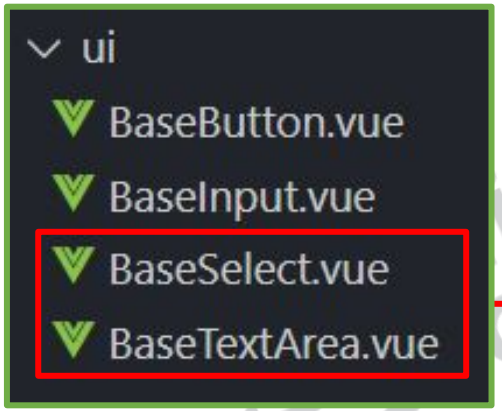
The user page can only be accessed if the **user** is **logged in.** If the **user** has not **logged in,** they will be directed to the **Login Page.**

Practice Adding New Recipe Page Part 1

30 Minutes

Timedoor academy PRO

We're going to add some of the necessary components to create this page.



Add a new component in the **ui folder** called the **BaseSelect.vue** and **BaseTextArea.vue** **component.**

Add the following code to the **BaseSelect.vue component**. This **component** is used when it requires **input** in the form of a **select**.

src > components > ui > BaseSelect.vue

```
<template>
  <select class="form-select" :id="id"
    @input="$emit('update:modelValue', $event.target.value)">
    <option v-for="(item, index) in data" :key="index" :value="item"
      :selected="modelValue === item">
      {{ item }}
    </option>
  </select>
</template>
```

Still in the **BaseSelect.vue component**, add a following code in the **script tag.**

src > components > ui > BaseSelect.vue

```
<script setup>
defineProps({
  data: { type: Array, require: true },
  id: { type: String, require: true },
  modelValue: { type: [String, Number] },
});
</script>
```

Add the following code in the **BaseTextArea.vue component. This component** is used to create **textarea**.

src > components > ui > BaseTextArea.vue

```
<template>
  <div>
    <label :for="identity" class="fw-semibold">{{ label }}
          <span style="color: #cb3a31">*</span>
    </label>
    <textarea class="form-control" :id="identity"
      @keyup="emit('update:modelValue', $event.target.value)"
      v-text="modelValue"
    ></textarea>
  </div>
</template>
```

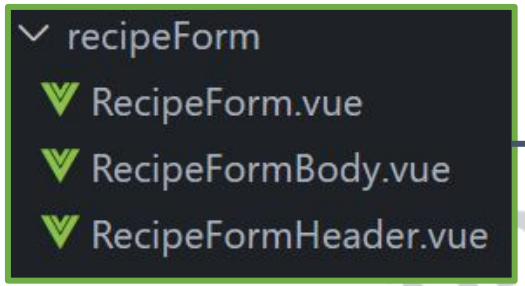
Still in the **BaseTextArea.vue component**, add the following code in the **script tag.**

src > components > ui > BaseTextArea.vue

```
<script setup>
defineProps({
  identity: { type: String, require: true },
  label: { type: String, require: true },
  placeholder: { type: String, require: true },
  modelValue: { type: String }
});

let emit = defineEmits(['update:modelValue']);
</script>
```

Now we will create a component that will be displayed on the page for adding a new recipe.



Add three new components in the recipeForm folder called **RecipeForm.vue**, **RecipeFormBody.vue**, and **RecipeFormHeader.vue**.

Afterwards, copy and paste the code for **RecipeFormBody.vue** and **RecipeFormHeader.vue** that has been provided in the link below: **https://bit.ly/45Zff2D**

We will now complete the code in the **RecipeFormBody.vue** component. Call the component in the missing part of the code with the following conditions.

No	Component	Props Value	Slot
1	BaseInput.vue	**type**=“file” **identity**=”recipeImage” **label**=“Photo Recipe”	-
2	BaseInput.vue	**type**=“text” **identity**=”recipeTitle” **placeholder**=“Give your recipe a title” **label**=“Photo Recipe”	-
3.	BaseTextArea.vue	**identity**=”recipeDescription” **label**=“Description” **placeholder**=“Share story behind recipe and what is make it special”	-
4.	BaseSelect.vue	**:data**= “[‘Breakfast’, ‘Lunch’, ‘Dinner’, ‘Meals’,’Snacks’]”	

Timedoor academy PRO

No	Component	Props Value	Slot
5	BaseInput.vue	**type**=“number” **identity**=”prepTime” **placeholder**=“0” **label=**”Prep Time”	-
6	BaseInput.vue	**type**=“number” **identity**=”cookTime” **placeholder**=“0” **label=**”Cook Time”	-
7	BaseInput.vue	**type**=“number” **identity**=”totalTime” **placeholder**=“0” **label=**”Total Time”	-
8	BaseInput.vue	**type**=“text” **identity**=”ingredient” **placeholder**=“Ex: 1 cup sugar”	-

timedoor academy PRO

No	Component	Props Value	Slot
9	BaseButton.vue	**class=**“new-ingredient-btn px-3 py-2” **type=**”button”	Add More
10	BaseInput.vue	**type=**“text” **identity=**”direction” **placeholder=**“Ex: 1 cup sugar”	-
11	BaseButton.vue	**class=**“new-ingredient-btn px-3 py-2” **type=**”button	Add More
12	BaseButton.vue	**class=**“cancel-btn px-3 py-2 ms-1”	Cancel
13	BaseButton.vue	**class=**“submit-recipe-btn px-3 py-2 ms-1”	Submit

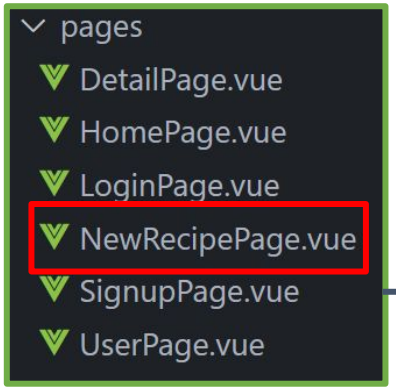
timedoor academy PRO

After completing the code in the **RecipeFormBody.vue** component, we will now complete the **RecipeForm.vue** component. Add the following code to the **RecipeForm.vue** component.

src > components > recipeForm > RecipeForm.vue

```
<template>
    <ul class="list-group">
        <recipe-form-header></recipe-form-header>
        <recipe-form-body></recipe-form-body>
    </ul>
</template>

<script setup>
    import RecipeFormHeader from "./RecipeFormHeader.vue"
    import RecipeFormBody from "./RecipeFormBody.vue"
</script>
```



Add the new component into the **pages folder** called **NewRecipePage.vue**

Add the following code to the **routes.js file.**

src > routes.js

```
import NewRecipePage from "./components/pages/NewRecipePage.vue";

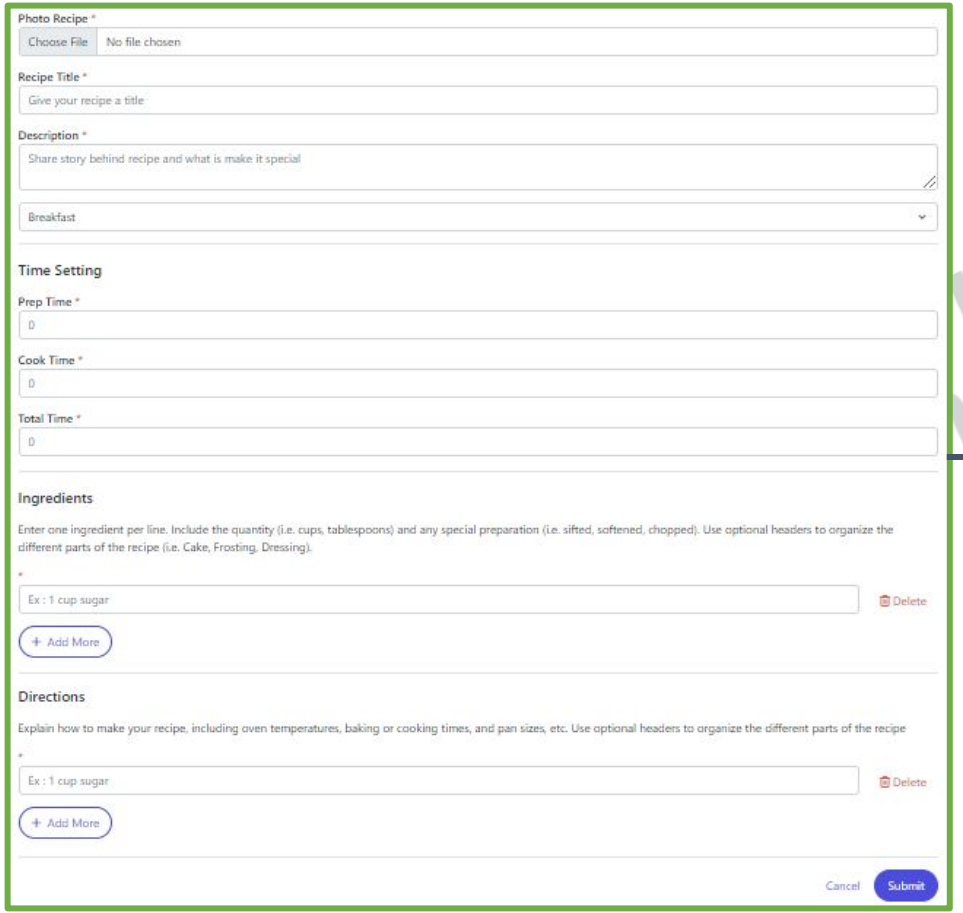
{ path: "/new-recipe", name: "newRecipePage", component: NewRecipePage,
    beforeEnter: (to, from, next) => {
      checkAuth() ? next() : next({ name: "login" });
    },
},
```

Lastly, add the following code into the **NewRecipePage.vue component.**

src > components > pages > NewRecipePage.vue

```
<template>
  <div class="container-md my-5 py-5">
    <recipe-form></recipe-form>
  </div>
</template>

<script setup>
import RecipeForm from "../recipeForm/RecipeForm.vue";
</script>
```



Open the **New Recipe page**, make sure the appearance of the **page** is as follows.

Practice Adding New Recipe Page Part 2

30 Minutes

Timedoor academy

Now we'll add the code so that the input form used to add a new recipe works. First, in the **BaseButton.vue** component, add the following code.

src > components > ui > BaseButton.vue

```
<template>
  <button class="btn rounded-pill" @click="$emit('clickButton')">
    <slot></slot>
  </button>
</template>
```

This code works so that the **button** can function when it is clicked.

Now, we need to add a variable that will be used to store the data entered in the form. Add the following code to the **RecipeFormBody.vue** component.

src > components > recipeForm > RecipeFormBody.vue

```
<script setup>
...
import { reactive } from "vue";
const recipeData = reactive({
  imageLink: "",
  name: "",
  description: "",
  category: "",
  prepTime: 0,
  cookTime: 0,
  totalTime: 0,
  ingredients: [],
  directions: [],
});

const ingredientCount = ref(1);
const directionCount = ref(1);
</script>
```

Add the following code **RecipeFormBody.vue component.**

src > components > recipeForm > RecipeFormBody.vue

```
<base-input type="file" identity="recipeImage"
      label="Photo Recipe" @input="checkImage"></base-input>

<base-input type="text" identity="recipeTitle"
      placeholder="Give your recipe a title" label="Recipe Title"
      v-model="recipeData.name"></base-input>

<base-text-area identity="recipeDescription" label="Description"
      placeholder="Share story behind recipe and what is make it special"
      v-model="recipeData.description"></base-text-area>

<base-select :data="['Breakfast', 'Lunch', 'Dinner', 'Meals', 'Snacks']"
      v-model="recipeData.category"></base-select>
```

src > components > recipeForm > RecipeFormBody.vue

```
<base-input type="number" identity="prepTime" placeholder="0"
      label="Prep Time" v-model="recipeData.prepTime"></base-input>

<base-input type="number" identity="cookTime" placeholder="0"
      label="Cook Time" v-model="recipeData.cookTime"></base-input>

<base-input type="number" identity="totalTime" placeholder="0"
      label="Total Time" v-model="recipeData.totalTime"></base-input>
```

Now, we will add the **code** so that the **user** can add more than one ingredient.

src > components > recipeForm > RecipeFormBody.vue

```
<base-button
     class="new-ingredient-btn px-3 py-2"
     @clickButton="addIngredient">
     <i class="fa-solid fa-plus me-2"></i>Add More
</base-button>
```

Add the following ***custom event*** code to the **Add More** button from the **ingredients.**

Then add a **function** called `addIngredient` in the **script tag**, where this **function** will call the **Add More** button for the **ingredient** to click on.

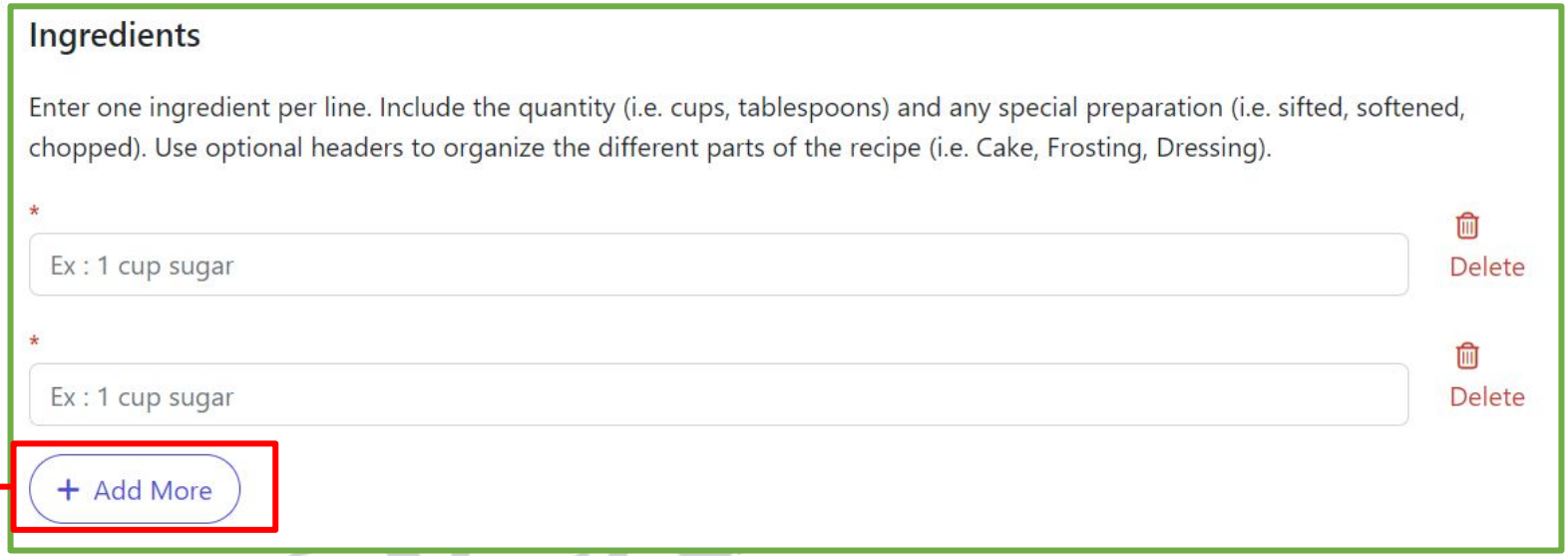
src > components > recipeForm > RecipeFormBody.vue

```
const addIngredient = () => {
  ingredientCount.value++;
};
```

After that, add this code to the ***ingredients*** part.

src > components > recipeForm > RecipeFormBody.vue

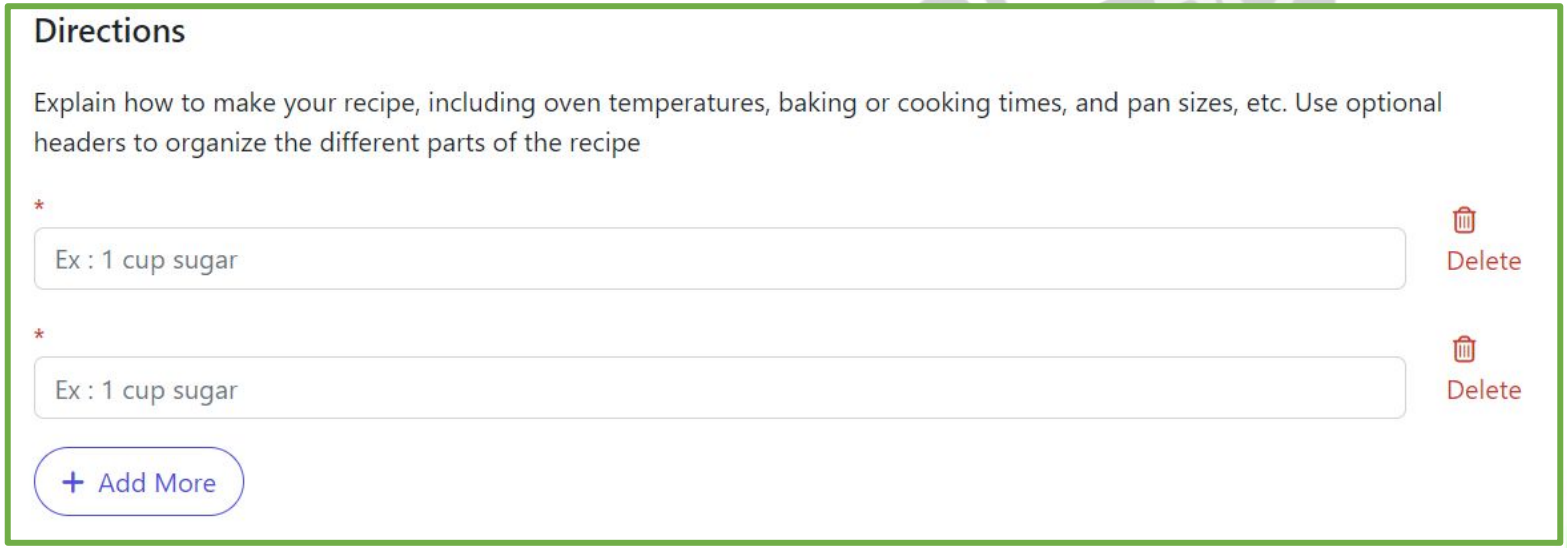
```
<div class="mb-3 row" v-for="count in ingredientCount" :key="count">
     <div class="col-lg-11 col-11">
          <base-input
               type="text" identity="ingredient"
               placeholder="Ex : 1 cup sugar"
               v-model="recipeData.ingredients[count - 1]">
          </base-input>
     </div>
      ...
</div>
```



Finally, try clicking the **Add More** button and make sure the number of **ingredient input forms** continues to grow.

Now use the same method as in the **Ingredients** section to activate the **Add More** button in the **Directions** section.

Make sure the result is like the **following**.



Next, we will activate the **Delete** button in the **Ingredients** and **Directions** section. Add the following code to the **Delete** button.

src > components > recipeForm > RecipeFormBody.vue

```
<div class="col-lg-1 col-1 col-form-label
     align-self-end delete-ingredient" style="color: #cb3a31"
     v-if="ingredientCount > 1" @click="deleteIngredient(count - 1)">
          <i class="fa-regular fa-trash-can px-1"></i>
          <span class="d-none d-md-inline">Delete</span>
</div>

<div class="col-lg-1 col-1 col-form-label
     align-self-end delete-ingredient" style="color: #cb3a31"
     v-if="directionCount > 1" @click="deleteDirection(count - 1)">
          <i class="fa-regular fa-trash-can px-1"></i>
          <span class="d-none d-md-inline">Delete</span>
</div>
```

Then add the following **two functions** to the **script tag**. The following function will be **executed** when the **Delete** key is pressed.

src > components > recipeForm > RecipeFormBody.vue

```
const deleteIngredient = (index) => {
  recipeData.ingredients.splice(index, 1);
  ingredientCount.value--;
};

const deleteDirection = (index) => {
  recipeData.directions.splice(index, 1);
  directionCount.value--;
};
```

Total Time will immediately add up t**he Prep Time and Cook Time.** Add the following code to the **BaseInput.vue component.**

src > components > ui > BaseInput.vue

```
<input :class="[{ 'd-none': isImage}, 'form-control']"
      :type="type" :id="identity" :placeholder="placeholder"
       :readonly="readonly === '1'" :value="modelValue"
       @input="$emit('update:modelValue', $event.target.value)"
       @keyup="$emit('keyInput', $event.target.value)"
       @focus="$emit('totalTimeFocus', $event.target.value)"
/>
```

Then in the **input component** for **total time,** add the following code.

src > components > recipeForm > RecipeFormBody.vue

```
<base-input type="number" identity="totalTime"
     placeholder="0" label="Total Time" v-model="recipeData.totalTime"
     @totalTimeFocus="totalTime" readonly="1">
</base-input>
```

Add the following **function** to the **script tag.**

src > components > recipeForm > RecipeFormBody.vue

```
const totalTime = () => {
  recipeData.totalTime = parseInt(recipeData.prepTime) +
parseInt(recipeData.cookTime)
}
```

Lastly, we'll create a function that is used to handle the selected recipe image. Add the following code in the **script tag.**

src > components > recipeForm > RecipeFormBody.vue

```
const checkImage = (e) => {
  const file = e.target.files[0];
  const reader = new FileReader();
  reader.readAsDataURL(file);

  reader.addEventListener("load", () => {
    recipeData.imageLink = reader.result;
  });
};
```

Practice Adding New Recipe Page Part 3

30 Minutes

Timedoor academy

In the previous exercise we completed the code related to the **input form** on the Add a new recipe page. Now we will **save all data** entered into firebase.

Add a new **mutation** named **setNewRecipe** in the **recipe.js file.**

src > store > recipe.js

```
setNewRecipe(state, payload) {
    state.recipes.push.payload;
}
```

Still in the **recipe.js file**, add a new **action** called **addNewRecipe**. **This action** will be used to store data in the **firebase**.

src > store > recipe.js

```
async addNewRecipe({ commit, rootState }, payload) {
     const newData = {
          ...payload,
          username: rootState.auth.userLogin.username,
          createdAt: Date.now(),
          likes: ["null"],
          userId: rootState.auth.userLogin.userId,
     };
}
```

Still in the **addNewRecipe action**, Add the following code.

src > store > recipe.js

```
try {
      const { data } = await axios.post(
      `https://recipe-vue-batch2-default-rtdb.firebaseio.com/recipes.json?
      auth=${rootState.auth.token}`, newData);

      commit("setNewRecipe", { id: data.name, ...newData });
} catch (err) {
      console.log(err)
}
```

Add the following code in the **form tag** of the **RecipeFormBody.vue component.**

src > components > recipeForm > RecipeFormBody.vue

```
<form @submit.prevent="addNewRecipe">
      ...
</form>
```

Then add the following **code** in the berikut pada **script tag.**

src > components > recipeForm > RecipeFormBody.vue

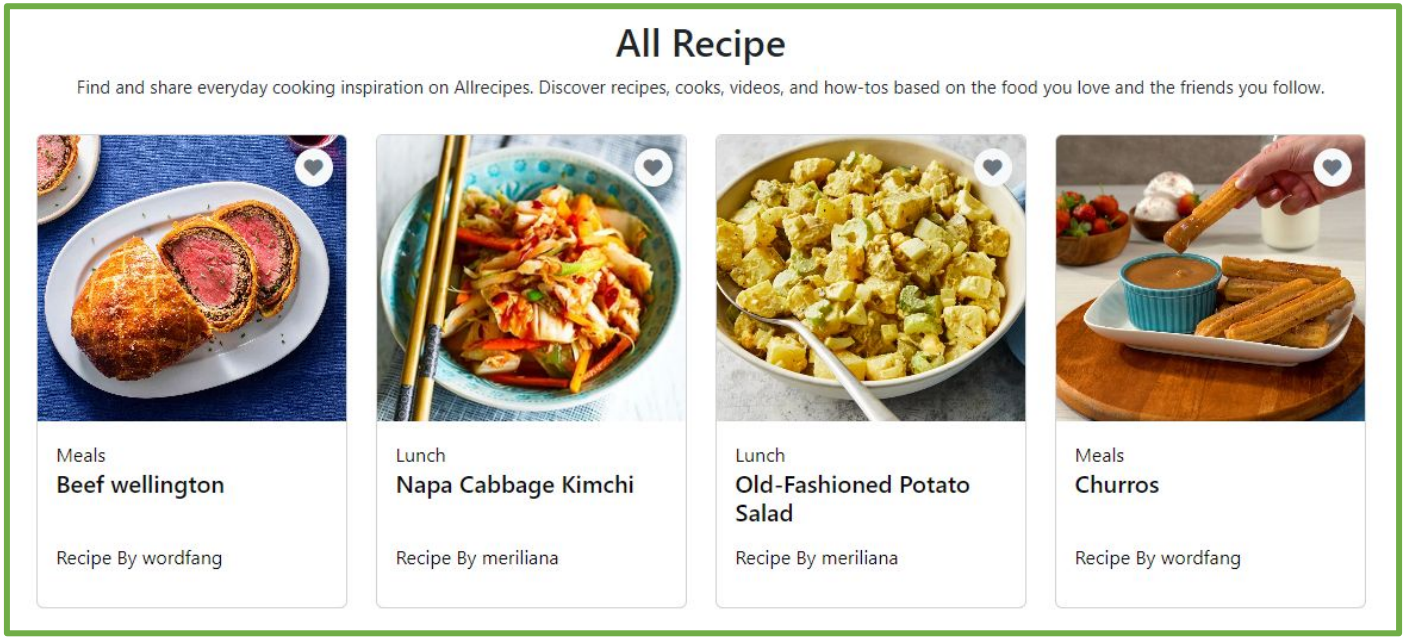
```
const store = useStore()
const router = useRouter()

const addNewRecipe = async () => {
  await store.dispatch("recipe/addNewRecipe", recipeData);
  router.push("/user/user-recipe");
}
```

Now try to add new recipe data. Make sure that every **new recipe** that is added, the new data is successfully entered into the **firebase**.



Try opening the **home page**. Make sure the new data added appears on the **home page**.



Practice Displaying Recipes to User Pages

30 Minutes

Timeador academy PRO

Every **new recipe** that is added will appear on the **user page**. Now we will add a **code** to **display** the recipe on the **user page.**

In the **UserRecipeCard.vue** component, add the following code

src > components > user > UserRecipeCard.vue

```
<script setup>
defineProps({
  recipe: { type: Object, require: true },
  buttonName: { type: Array, require: true }
})
</script>
```

Still in the **UserRecipeCard.vue** component, add the following codes.

src > components > user > UserRecipeCard.vue

```
<div class="d-flex justify-content-between">
      <div>
            <p class="mb-0">{{ recipe.category }}</p>
            <div class="h-50">
                  <h4 class="fs-5 mb-0">{{ recipe.name }}</h4>
            </div>
            <slot></slot>
      </div>
      <div class="d-flex">
            <img :src="recipe.imageLink" :alt="recipe.name"
                  width="90" height="80" class="rounded"
                  style="object-fit: cover">
      </div>
</div>
```

Still in the **UserRecipeCard.vue** tcomponent, add the following code.

src > components > user > UserRecipeCard.vue

```
<div class="d-flex justify-content-between mt-3 pt-3 border-top">
     <button class="btn delete-btn px-3 py-2 rounded-pill">
          {{ buttonName[0] }}
     </button>
     <button class="btn edit-btn px-3 py-2 rounded-pill">
          {{ buttonName[1] }}
     </button>
</div>
```

Now add this code to the **UserRecipe.vue component.**

src > components > user > UserRecipe.vue

```
<script setup>
import { computed } from "vue";
import { useStore } from "vuex";

const store = useStore();

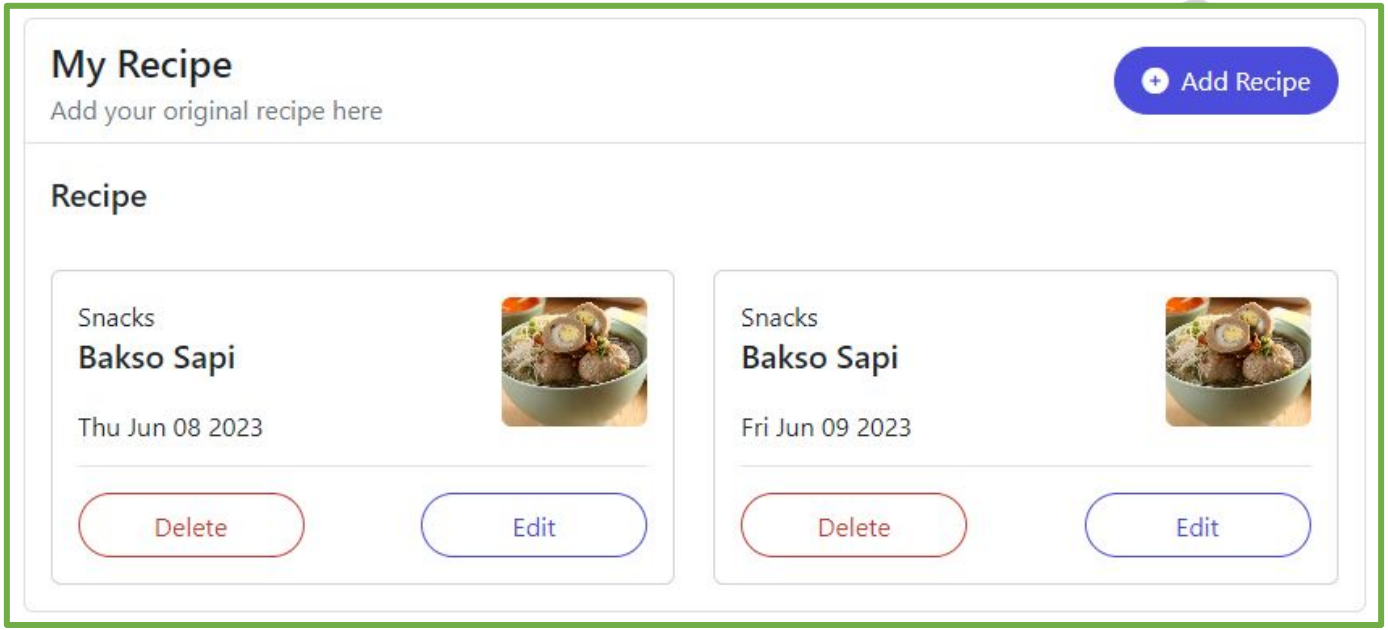
const recipes = computed(() => {
  const allRecipe = store.state.recipe.recipes;
  const userId = store.state.auth.userLogin.userId;
  return allRecipe.filter((recipe) => recipe.userId === userId);
});
</script>
```

Call the **UserRecipeCard.vue component** in the **UserRecipe.vue** like as followed.

src > components > user > UserRecipe.vue

```
<li class="list-group-item">
     <p class="mt-2 mb-4 fs-5 fw-semibold">Recipe</p>
     <div class="row">
          <user-recipe-card
               v-for="recipe in recipes"
               :key="recipe.id"
               :recipe="recipe"
               :buttonName="['Delete', 'Edit']"
          >
               <p>{{ new Date(recipe.createdAt).toDateString() }}</p>
          </user-recipe-card>
     </div>
</li>
```

Select the **My Recipe** menu located on the **User page**. Make sure the **page displays** the recipe made by the user who is currently **logged in**.



Practice Adding the Delete Feature

20 Minutes

Timedoor academy

Now, we will enable the feature to delete recipes. Add this code to the **recipe.js file.**

src > store > recipe.js

```
async deleteRecipe({ dispatch, rootState }, payload) {
     try {
          const { data } = await axios.delete(
               `https://recipe-vue-batch2-default-rtdb.firebaseio.com/
               recipes/${payload}.json?auth=${rootState.auth.token}`);
          await dispatch("getRecipeData")
     } catch (err) {
          console.log(err);
     }
},
```

The Delete method will be used to delete the data from the **firebase**.

Add the following code in the **UserRecipeCard.vue component.**

src > components > user > UserRecipeCard.vue

```
<div class="d-flex justify-content-between mt-3 pt-3 border-top">
     <button class="btn delete-btn px-3 py-2 rounded-pill"
          @click="$emit('btnRemowe')">
          {{ buttonName[0] }}
     </button>
     <button class="btn edit-btn px-3 py-2
          rounded-pill">
          {{ buttonName[1] }}
     </button>
</div>
```

Add the following code in the **UserRecipe.vue copmponent.**

src > components > user > UserRecipe.vue

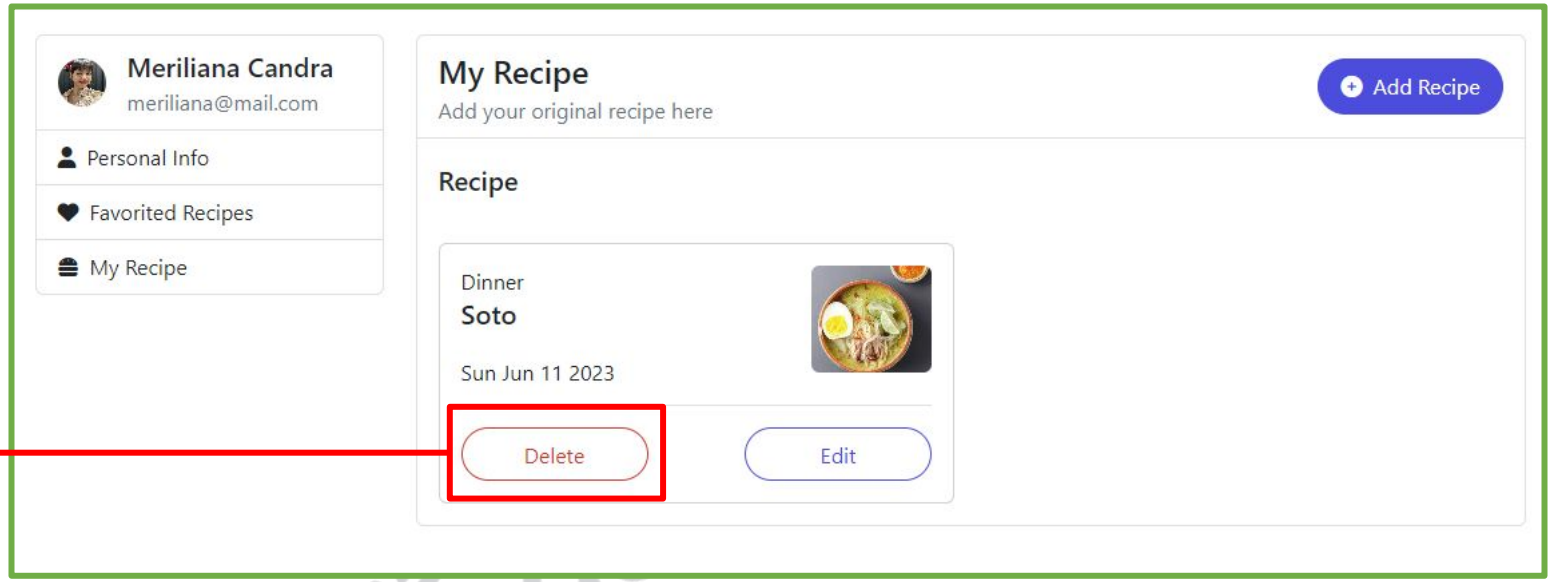
```
<user-recipe-card
      v-for="recipe in recipes"
      :key="recipe.id"
      :recipe="recipe"
      :buttonName="['Delete', 'Edit']"
      @btnRemowe="deleteRecipe(recipe.id)"
>
      <p>{{ new Date(recipe.createdAt).toDateString() }}</p>
</user-recipe-card>
```

Add a **function** named **deleteRecipe** in the **UserRecipe.vue component.**

src > components > user > UserRecipe.vue

```
const deleteRecipe = async (id) => {
  await store.dispatch("recipe/deleteRecipe", id);
}
```

The deleteRecipe function will be called on when the **Delete** button is pressed.



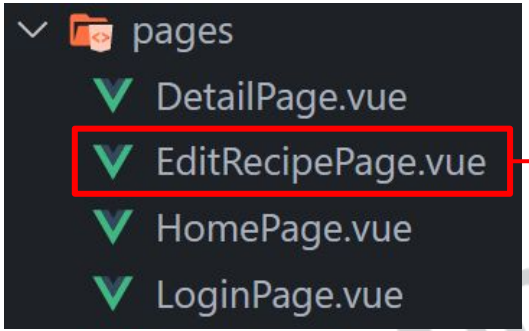
Now try pressing **Delete**. Make sure the selected data is successfully deleted.

Practice Adding the Edit Feature Part 1

30 Minutes

Timedoor academy

Now we will add a feature to update recipes created by users.



First, in the page folder add a new component named **EditRecipePage.vue.**

timedo academy PRO



Add a new route for edit page. Use the path **"/recipe/edit/:id**". The recipe edit page can only be accessed if the user has logged in before.

Add the following code to the **EditRecipePage.vue** *component*.

src > components > page > EditRecipePage.vue

```
<template>
  <main>
    <div class="container-md my-5 py-5">
      
    </div>
  </main>
</template>
```

Panggil *component* RecipeForm.vue disini.



Tugasmu sekarang yaitu tambahkan 4 variabel baru pada *component* EditRecipePage.vue dengan ketentuan sebagai berikut.

1. Call the `useStore()` function and save it in a variable called `store` .
2. Call the `useRoute()` function and save it in a variable called `route`.
3. A variable called `detailData` with an empty value (`ref()`).
4. A variable called `isLoading` with a false value.

Still in the **EditRecipePage.vue** *component*, Add `onMounted` complete the following code.

src > components > page > EditRecipePage.vue

```
onMounted(async () => {
```

1. Call the isLoading variable then fill it with a true value I
2. Use await then call the getRecipeDetail action and payload value which is the id from the selected recipe.(Gotten from dynamic route)
3. Save the detailed recipe information contained in the recipeDetail state and then save it in the detailData variable.
4. Call the isLoading variable then fill it with a false value.

```
});
```

src > components > page > EditRecipePage.vue

```
<recipe-form
    v-if="detailData && !isLoading" :isEdit="true">
</recipe-form>
```

Add the following code in the calling of the ***RecipeForm.vue*** *component* that's located in the ***EditRecipePage.vue*** *component.*



Now the **RecipeForm.vue** component can accept a prop named **isEdit.** Add the code to receive the prop in the **RecipeForm.vue** component with the following conditions.

- Prop name is isEdit
- The data type received is Boolean
- The default value is false

src > components > recipeForm > RecipeForm.vue

```
<template>
    <ul class="list-group">
        <recipe-form-header></recipe-form-header>
        <recipe-form-body :isEdit="isEdit"></recipe-form-body>
    </ul>
</template>
```

Still in the **RecipeForm.vue** *component*, change the calling of the **RecipeFormBody.vue** component to be like this.

Then add the following code into the **RecipeFormBody.vue** *component*.

src > components > recipeForm > RecipeFormBody.vue

```
const props = defineProps({
  isEdit: { type: Boolean, default: false },
});
```

Add the following code into the **RecipeFormBody.vue** *component*.

src > components > recipeForm > RecipeFormBody.vue

```
onMounted(() => {
  if (props.isEdit) {
    recipeData.value = store.state.recipe.recipeDetail;
    ingredientCount.value = recipeData.value.ingredients.length;
    directionCount.value = recipeData.value.directions.length;
  }
});
```

This code is used to store **recipe data** that will be updated. This process can only be run if the **`isEdit` props is true**.

Practice Adding the Edit Feature Part 2

30 Menit

Timedoor Academy

Now, we will add **action** to update data in firebase. ***Method* PUT** is used to update the data.

src > store > recipe.js

```
async updateRecipe({ dispatch, rootState }, { id, newRecipe }) {
  try {
     const { data } = await axios.put(
      `https://recipe-vue-batch2-default-rtdb.firebaseio.com/
      recipes/${id}.json?auth=${rootState.auth.token}`, newRecipe);

     await dispatch("getRecipeData")

  } catch (error) {
     console.log(error);
  }
},
```

In *component* **RecipeFormBody.vue** there is function namely `addNewRecipe`. Update the content of the function with the following requirements.



1. Add the condition **`If props isEdit is true`** then **`call updateRecipe action`**.

```
await store.dispatch(
      "recipe/updateRecipe",
      { id: route.params.id, newRecipe: recipeData.value}
);
```

2. Make **route variable** because recipe id comes from **route params.**
3. Add else block, where else block will executew action **addNewRecipe.**
4. After condition block, add code for moving to **User Recipe** page.

Now try to **update** one of the recipe data. Make sure **Edit** Procces is running well.

